LEPL Kutaisi International University

School of Computer Science

Bachelor's Program in Computer Science

Giorgi Bakradze, Sandro Lortkipanidze, Lado Turmanidze

Implementation of a MIPS CPU, Its Peripherals, and Interrupts

The thesis is completed for obtaining the academic degree of
Bachelor of Computer Science

Bachelor's Thesis Supervisor

Prof. Dr. Walter F. Tichy

Co-supervisor

Prof. Dr Wolfgang J. Paul

Kutaisi International University, 2026

Declaration

I, the author of this bachelor's thesis, declare that the thesis is my own original work and does not contain any material previously published, submitted for publication, or presented for defense by other authors, unless properly referenced or cited in accordance with academic standards.

Authors: Giorgi Bakradze, Sandro Lortkipanidze, Lado Turmanidze

Date: April 27, 2026

Abstract

This thesis presents the design and implementation of a MIPS CPU in Verilog, closely following the formal hardware specification from the KIU Computer Architecture course. The processor is implemented on an Altera Cyclone V FPGA board and supports a subset of the MIPS ISA. The work is divided into three parts: the sequential CPU core with all fundamental modules (adder, arithmetic unit, ALU, shifter, branch condition evaluator, instruction decoder, and program counter logic); the integration of on-board SRAM and LPDDR2 memory instead of simulated register memory; and the addition of interrupt support, including illegal instruction detection, misalignment faults, and system calls. The design is validated through ModelSim simulation waveforms and on-board testing, and is further exercised on multiplication, division, and bubble sort algorithms written in MARS assembly. Results confirm correct execution of all supported instructions on real hardware.

Keywords: MIPS, FPGA, Verilog, CPU design, interrupts, SRAM, LPDDR2, hardware description language, Altera Cyclone V.

0 Contents

1 Introduction **1**

2 Adder Module **6**

3 Arithmetic Unit **6**

4 Arithmetic Logic Unit (ALU) **9**

5 Shifter **11**

5.1 fixed distance shifter 12

5.2 Barrel Shifter using fixed distance shifters 13

6 Branch Condition Evaluation Unit(BCE) **15**

7 Auxiliary Modules **18**

7.1 Immediate Extension Unit 18

7.2 Splitter module(used in instruction decoder) 18

7.3 General Purpose Register File (GPR) 19

7.4 ALUDecoderBridge 20

7.5 Shifter.v module 22

7.6 Helper modules for decoding stage 22

7.7 Helper module for writeback stage 23

8 Modules For Decoding Stage **23**

8.1 Next PC environment 26

8.2 Master Module that handles Decoder Stage 29

9 Execute Stage **29**

10 Memory **30**

10.1 Register Memory 32

11 CPU (Master Module to rule them all) **33**

12 SRAM **37**

12.1 Introduction . . . 37

12.2 Setup . . . 37

12.2.1 IP Catalog . . . 37

12.2.2 IP creation wizard . . . 38

12.3 Usage . . . 40

12.4 Memory Initialization . . . 41

12.4.1 Wizard configuration . . . 41

12.4.2 .Mif or .Hex? . . . 42

12.4.3 Hex files . . . 42

12.4.4 Mif files . . . 42

12.5 Memory Content Editor . . . 45

12.6 MIPS CPU specific . . . 47

13 Off-Chip SRAM **49**

13.1 Setup . . . 49

13.1.1 Module Specification and Example . . . 52

13.2 What if we want to make it work for 32 bits? . . . 53

13.2.1 Overview . . . 53

13.3 How we do it . . . 54

13.4 Module source code . . . 54

14 LPDDR2 RAM **56**

14.1 Logic behind the IP . . . 58

14.2 Logic behind our Implementation . . . 59

14.3 The IP provided . . . 60

14.4 Our abstraction . . . 63

14.5 How we use it (Memory master module) . . . 69

14.5.1 LPDDR2 specific . . . 70

14.5.2 SRAM specific and common . . . 71

14.5.3 Code . . . 72

14.6 User Guide for LPDDR2 Memory . . . 73

14.6.1 Setting Up the IP Core . . . 73

14.6.2 Integrating the Memory Controller . . . 74
14.6.3 Compiler Directive for Memory Selection . . . 77

15 Interrupts **80**
15.1 Changes to the Original Code . . . 80
15.2 Find First One Circuit (FF1) . . . 84
15.2.1 Parallel Prefix Circuit . . . 84
15.2.2 $n-1$ NOT gates . . . 86
15.3 Special Purpose Register . . . 87
15.4 Interrupt Controller . . . 91
15.5 ‘main_interrupt.v‘ . . . 93
15.6 Updated ‘master.sv‘ and ‘memory_master.v‘ . . . 97

16 Further testing of the design **103**
16.1 A little introduction to Signal Tap Logic Analyzer . . . 103
16.2 Multiplication Test . . . 105
16.3 Division Test . . . 107
16.4 Bubble Sort Test . . . 112

17 Hardware Implementation Structure **115**
17.1 Design Rationale . . . 116
17.2 Core Components . . . 116
17.2.1 Execute Components (`parts/execute/`) . . . 116
17.2.2 Memory System (`parts/memory/`) . . . 117
17.2.3 General-Purpose Registers (`parts/writeback/`) . . . 117
17.3 Interface Communication . . . 117
17.4 Utility Components . . . 117
17.5 Memory Options . . . 118
17.6 Sequential vs. Pipelined Operation . . . 118

18 Conclusion **118**

19 Appendix - TestBench Codes **119**

0 List of Figures

1 Overview of the design. Each of the parts will be explained along the way. 5
2 adder diagram from the book 7
3 adder waveform, for testbench code check appendix (Chapter 19-testbench codes) . 7
4 n-au diagram 8
5 Data paths of an n-arithmetic unit 9
6 Waveforms for Arithmetic Unit, check chapter 19 for test bench. 9
7 ALU specification(from the book, page 95, table 7) 11
8 test pattern for all possible values of af and i row 1: operand a, row 2: operand b, row 3: af, row 4: i, row 5: output result, row 6: ovfalu. click to check ALU test bench 11
9 construction of an (n, i)-logical right shifter ([5] page 97, chapter 7.4) 12
10 waveforms for fixed distance shifter, row 1: input a, row 2: input s, row 3: output res. click here to check fixed distance shifter test bench 13
11 construction using stack of fixed distance shifters ([5] page 97, chapter 7.4) 14
12 waveforms of shift right logical module, row 1: input a, row 2: input b (shift amount), row 3: res. click here to check srl.v test bench 15
13 specification of BCE, chapter 7.5, page 99 16
14 circuit of auxiliary signal computation (page 99 from the [5]) 16
15 Waveforms for BCE unit. See BCE module test bench. 16
16 division of instructions (page 113 from the [5], chapter 8.2.1) 19
17 GPR file diagram(page 75, chapter 6.4 three port RAM for GPR) 21
18 Instruction Decoder Diagram(page 129, chapter 8.3.6) 24
19 Next PC circuit from [5] 28
20 waveforms for master module, check chapter 19 for testbench 36
21 IP Catalog and its dropdown menu 38
22 IP instance creation. Name would be written next to "CPU/" in the example above. 39
23 First page of the wizard 39
24 Memory init tab in the wizard, already configured to load initial values. 41
25 Freshly opened Memory Content Editor 45
26 Memory Content Editor, connected but with no data loaded 46

27 The IP Catalog entries for off-board SRAM 50
28 Name the module 50
29 Timing diagram showing typical usage and how internal wires behave. 56
30 State machine of the E bit 57
31 LPDDR2 RAM on the chip 59
32 General layout of the hardware, how we implement the memory 60
33 Avalon Memory interface. Times shown are not accurate, they're just for visual interpretation. 62
34 The State machine 64
35 The final interface we create. Simpler than the original. 66
36 Memory partitioning 69
37 Interrupts handled by our design 80
38 Testbench for updated 'Type' function 81
39 Recursive construction of an n-bit parallel prefix circuit of the function $\circ$ for an even n 84
40 Output in ModelSim's Transcript window 85
41 Output in ModelSim's Transcript window 86
42 Special Purpose Register file 87
43 Special Purpose Registers 87
44 SPR Testbench Waveform 89
45 *jisr* and interrupt level computation 91
46 'interrupt_controller.v' testbench 92
47 Output of above testbench in ModelSim 96
48 Computation of the execute bit with interrupts 97
49 The new *pc* environment supporting interrupts and return from exception 97
50 First part of *ill* waveform 99
51 First part of *ill* output 99
52 Second part of *ill* waveform 99
53 Second part of *ill* output 99
54 *malf* waveform 100
55 *malf* output 100
56 *misals* waveform 101

57 *misals* output 101
58 *syscall* output 101
59 Test for move instructions 102
60 Clock selection UI. We have selected a 50MHz clock for the device. This affects the frequency of data fetch from the device. 104
61 Signal tap nodes in use. As you can see, some wires are red - Quartus has optimized them, removed them. Adding /* synthesis keep */ would fix that. 104
62 Result of the multiplication algorithm (multiplying 5 and 7) 106
63 Result of the multiplication algorithm (multiplying 5 and 7) 106
64 Final Content of Registers 107
65 Final Content of Registers 107
66 Computation of Division we divide 100 by 5 110
68 Result shown output on HEX display (Which has the register 5 mapped to it). Keep in mind that MARS and Signal Tap show results in Hexadecimal, hardware shows in decimal. 111
67 Computation of Division we divide 100 by 5, results in register 5. Left - Signal Tap (registers sorted descending, from 31 to 0), Right - MARS Simulator (registers ascending) 111
69 Sorted Data in Hexadecimal format stored in registers [2-18]. initial data to be sorted is: [64, 34, 25, 12, 22, 11, 90, 45, 33, 21, 65, 89, 44, 1, 99, 13] 113
70 Results on Board observed from Signal Tap 114

1 Introduction

This thesis aims to develop a gold-standard design for the hardware lab that closely mirrors the lecture slides at KIU (Kutaisi International University), with minimal deviations. The design is tested on the multiplication and division algorithms discussed in the lectures and is assembled using MARS. Additionally, the design is extended with the interrupt support. Furthermore, the study explores methods for accessing external SRAM and DRAM on the board. DRAM access can be achieved either by utilizing an available DRAM controller as an IP core or by investing significant effort, as demonstrated in a similar lab at Tel Aviv University led by Guy Even.

This thesis heavily relies on [5]. Alongside simulation, the design was compiled onto the FPGA board Altera Cyclone V. In the text, we refer to the Cyclone V Altera board simply as "the board." For further details, refer to [1]. We also refer to our GitHub repository; interested readers can clone the repository and run the complete processor [3]. Whenever we reference numbered lemmas, we refer to [5]. Whenever we mention MARS simulator, we refer to [4].

At KIU, first-semester students learn how to precisely specify hardware and, based on those specifications, prove its correctness. In the first semester, students design a sequential hardware machine based on a subset of the MIPS ISA in a modular fashion, then prove its correctness using fundamental concepts which are later generalized to more complex hardware in subsequent years. These core concepts include simulation relations, induction, and Boolean algebra (or in some cases, first-order logic). Most predicate evaluations in fundamental modules such as the ALU and BCE are based on theoretical results from [5]. In the hardware lab, taught after the Computer Architecture course, this sequential design is implemented, with memory modeled as a separate Verilog module. Based on our results, others can extend the system with disk access, a memory management unit, and a simple form of an operating system, which is addressed in the second semester at KIU. Currently, other bachelor theses are being developed based on this foundation.

One of the main contributions of this thesis is the integration of memory on the FPGA board, replacing the previously compiled Verilog memory module. The project consists of three main parts:

1. **Basic Processor Design:**

   The initial design and fundamental modules, which represent a direct mapping from the design in [5] (described using Boolean algebra) to Verilog syntax, formed the starting point of this thesis. This part, along with the documentation, was completed by Giorgi Bakradze.

Subsequently, Sandro Lortkipanidze made significant contributions by implementing the decode and memory stages (initially using a mock memory) and integrating the components to achieve a working simulation of the processor by the end of Chapter 11.

2. **Integration of On-Board Memory:**
   Instead of using a mock memory module, we decided to harness the power of on-board memory (LPDDR2 and SRAM). Sandro Lortkipanidze put considerable effort into this task, overcoming the lack of comprehensive documentation on the IPs and memory structure. He successfully integrated both SRAM and LPDDR2 into the processor design and produced detailed documentation on the process.

3. **Interrupt Support:**
   Later, Lado Turmanidze made modifications to the initial processor design and added interrupt support, which was successfully tested through simulation.

After the project was completed, Sandro Lortkipanidze cleaned up the codebase, resulting in a well-structured GitHub repository [3].

Substantial effort was devoted to testing. Since the design included two types of on-board memory, we used tools like SignalTap for debugging and developed rigorous test cases to ensure that every instruction from the instruction table functioned correctly. Instead of writing machine code by hand, we used the MARS simulator, which allows writing assembly code and generating a `.MIF` file—this file contains the corresponding machine code instructions used to test our CPU.

In the initial processor design, a major challenge was debugging. (As always) Not everything worked out of the box, and substantial amount of time was spent debugging the hardware, which was made even more difficult due to compilation times (Sometimes 30 minutes or more!). Our generation was the first in the university to use Verilog at this scale, hence with not much information to guide us, we had to figure out everything on our own. However, as time went by, we gathered enough experience to easily tackle more and more problems along the way, speeding up the progress significantly.

As for the different kinds of on-board memories - most have proven easy to implement. Implementation of the SRAM (both on-chip and on-board) was as simple as instantiating an IP design and writing logic behind it. LPDDR2, however, didn't provide a simple interface, and on top of that, it introduced an issue of multicycle behavior. The latter was easy to work around, the former, however - it took plenty of reading through many documents, much of which provided either too small

or unnecessarily large information (e.g. hundreds of pages of documentation about implementing LPDDR2 protocol from scratch). In the end, the current solution was reverse engineered from the official Cyclone V example (Original was an LPDDR2 memory test code, hundreds of lines from it have been trimmed down to a single simple verilog state machine which we use).

We start by introducing the fundamental module for our CPU in Chapter 2, which is the adder module. We explain its specifications and how we mapped it into Verilog code. We also introduce the concept of a parameterized module.

Then, in Chapter 3, we introduce the concept of hierarchical design in Verilog. We implement the arithmetic unit based on the adder module, which we instantiate there, and map results from the book to compute flags.

Now that we have the arithmetic unit, we use it to implement the ALU. We explain its specifications and how we mapped it into Verilog, and we test it for all possible values of `af`.

Next, we use the construction of the shifter from [5]. We first implement the fixed-size shifter in Chapter 5.1 and then the barrel shifter in Chapter 5.2. Here, we again use a modular design, instantiating five instances of the fixed-size shifter with different shift lengths and different control bits. This supports `srl`. Verilog also has built-in operators for shifting, which we use in `Shifter.v`. (Eventually, we use this `Shifter.v` module; however, for its own elegance, we included Chapter 5.) In `Shifter.v`, we also include other types of shifts, which became helpful in multiplication. For example, since the MARS assembler stops computation upon detecting overflow, but the multiplication algorithm in [5] relies on a register becoming 0 after overflow, we use `sll` (shift left logical) instead of addition. `sll` does not cause overflow and simply performs bit manipulation to achieve the same result.

Chapter 6 introduces the branch condition evaluation unit. There, we simply map results from [5] to Verilog syntax, include a table of supported instructions, compute predicates based on the results from [5], and present waveforms tested on all branch instructions.

Chapter 7 introduces what we call "auxiliary modules" — modules that are important for cleaner code and are relatively small. There, we also include the general-purpose register file and some helper modules.

In Chapter 8, we introduce modules important for the decode stage. Since the next program counter is also calculated during that stage, we include the Next PC environment there. Chapter 8.2 combines all modules to eventually obtain a unified module that handles instruction decoding.

The chapter that follows describes the next stage of our machine, which is executing the in-

struction (i.e., the execute stage). It utilizes all previously described modules and instantiates them in a careful manner, harnessing the power of modular design to get the desired result.

The chapter that follows discusses the memory module — the memory master, which is instantiated if we are compiling the design for hardware, and the register memory, which is compiled for simulation. Details of this are described in depth in the chapters that follow about SRAM and LPDDR2.

In Chapter 11, we combine every stage of computation into one master module that controls everything. We test it in simulation and present the corresponding waveforms and the machine code used in the mock memory module.

In Chapter 12, we start discussing the first type of memory presented on the FPGA. We provide all necessary details along with implementation challenges. After that, another type of memory is explained — also on the FPGA board — which is LPDDR2.

In Chapter 13, we discuss off-chip SRAM, the second type of SRAM present on the board. There, we cover the setup and wiring details—specifically the baseline file, which is a crucial Verilog file for wiring on the FPGA board.

In Chapter 14, we introduce an additional type of memory present on the FPGA board: LPDDR2. We use it to expand the physical memory storage. The chapter provides an example of how to interface with a more complex IP module in Verilog, along with an abstraction that simplifies this complexity.

In Chapter 15, we provide a modified design that adds interrupt support.

In Chapter 16, we introduce a tool that is extremely useful for on-board debugging, called SignalTap. We then test our processor design for multiplication, division, and bubble sort. We wrote the corresponding code in MARS assembly and used it to get the corresponding machine code and loaded those instructions into the mock memory module for simulation and for on-board testing. We then provided the final testing results.

In Chapter 17, we describe the general structure of our implementation, including how we organized folders to improve code management.

Chapter 18 summarizes our work.

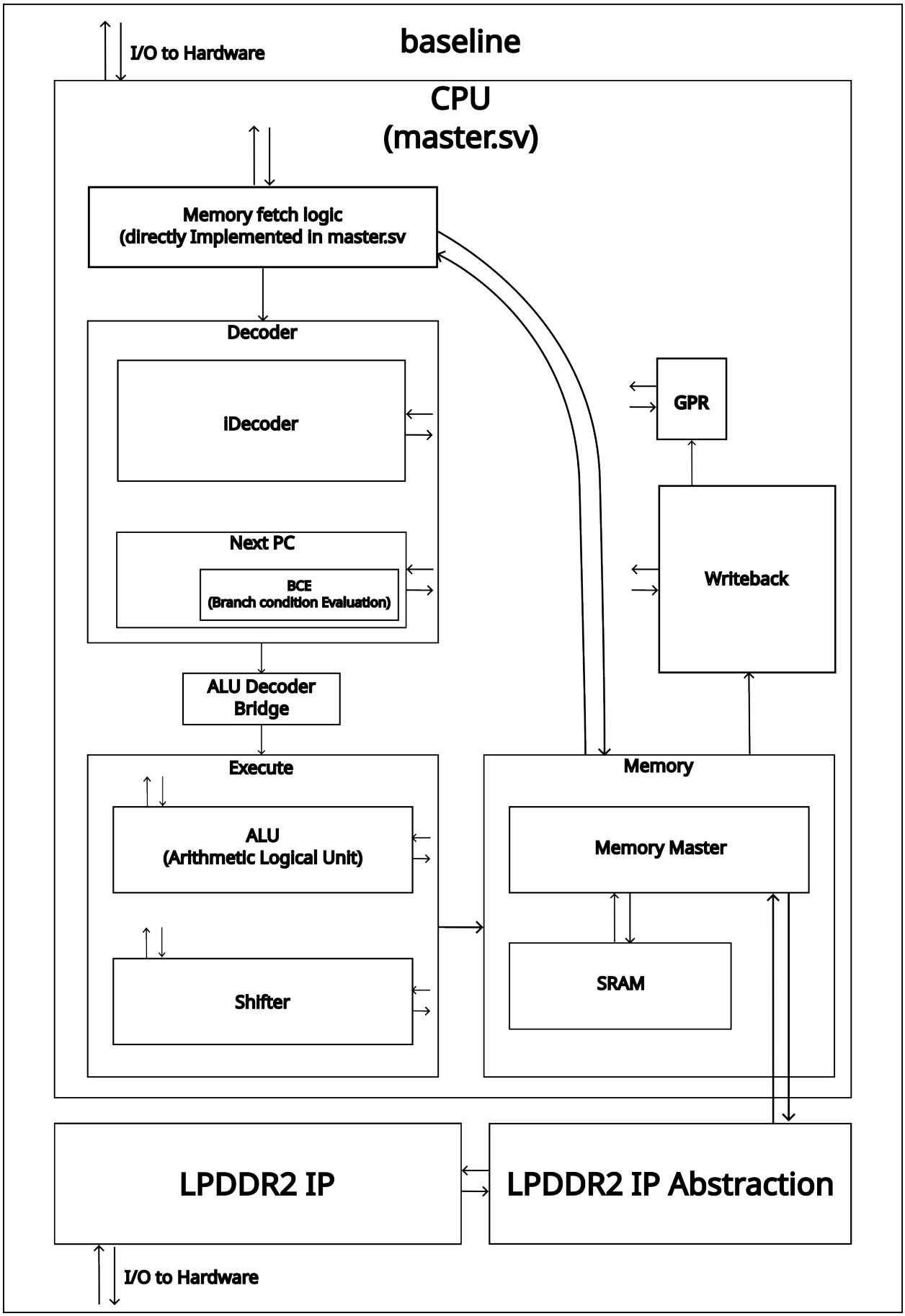


Figure 1: Overview of the design. Each of the parts will be explained along the way.

2 Adder Module

We start with the easiest module so far, the adder module. Before analyzing this module, we want to mention that similar designs exist in hardware labs in Saarbrücken, Tel Aviv, and Zurich.

Specification of the adder: An n-adder is a circuit with inputs $a[n-1:0] \in B^n$ and $b[n-1:0] \in B^n$, $c_0 \in B$ and outputs $c_n \in B$ and $s[n-1:0] \in B^n$ satisfying specifications: $\langle c_n, s[n-1:0]\rangle = \langle a[n-1:0]\rangle + \langle b[n-1:0]\rangle + c_0$. (page 81, chapter 7.1 in [1])

```
module adder #(parameter n = 32) (
input [n-1:0] a,
input [n-1:0] b,
input cin,
output [n-1:0] s,
output cout
);
assign {cout, s} = a + b + cin;
endmodule
```

Code 1: Adder Verilog Code

Below, one can see the general diagram of an adder, which is treated as a black box and can be implemented in various ways. The book provides numerous examples. Notably, our adder module uses the built-in infix `"+"` operator of Verilog, which, depending on time constraints and module size, can have different gate-level mappings when compiled on the FPGA board. Therefore, one can attempt to implement their own adders; however, every instance of the infix `"+"` must be replaced with the output of the corresponding module.

The [5] presents various adder implementations, including the Carry Chain Adder (with $O(n)$ cost and delay; see Lemma 33), the Conditional-Sum Adder (with $O(n^{\log 3})$ cost and $O(\log n)$ delay; see Lemma 34), and the best option, Carry-Look-Ahead Adders (CLAs), where n-CLAs have $O(n)$ cost and $O(\log n)$ delay; see Lemma 36.

As we can see for $0^{31}1$ input for both operands output is $0^{30}10$, for 1^{32} and $0^{31}1$ we get overflow, thus adder module is working correctly on those inputs, as it satisfies the specification for input samples.

3 Arithmetic Unit

The arithmetic unit is a fundamental part of our design, performing essential arithmetic operations such as addition and subtraction. This is the first instance where a hierarchical structure is utilized. Within the arithmetic unit module, we require an adder module to execute arithmetic operations.

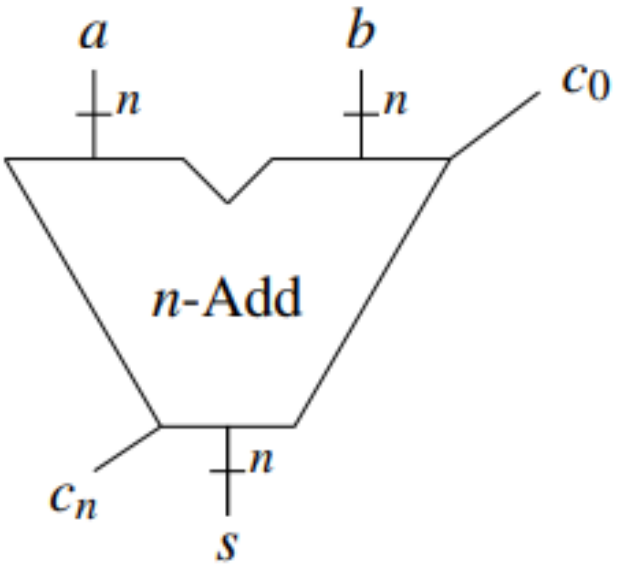


Figure 2: adder symbol ([5], Fig. 32, p. 82)

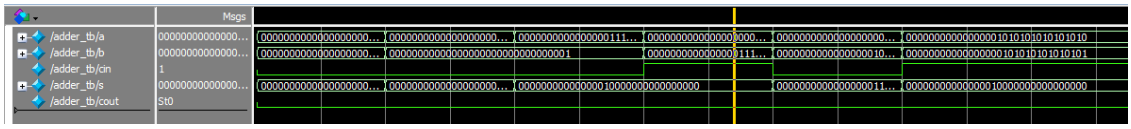


Figure 3: adder waveform, for testbench code check appendix (Chapter 19-testbench codes)

The arithmetic unit is a circuit with the following inputs: operands $a = a[n-1:0] \in B^n$ and $b = b[n-1:0] \in B^n$, a control signal $u \in B$ indicating whether the operands should be treated as unsigned integers, and finally, $sub \in B$, which specifies whether input b should be subtracted from input a. As the reader can see, this specification is directly translated into Verilog, where all the above inputs are declared using the `input` keyword.

The arithmetic unit has the following outputs: $s[n-1:0] \in B^n$, a negative bit $neg \in B$ (as referenced in Chapter 7.2 of the book, page 89), and an overflow bit $ovf \in B$. These variables are preceded by the `output` keyword. If only `output` is specified without explicitly declaring `wire`, the data type is implicitly assumed to be `wire`.

```
1         module au(input  [n-1:0] a,
2         input [n-1:0] b,
3         input sub,
4         input u,
5         output wire [n-1:0] s,
6         output wire neg,
7         output ovf);
8         parameter n = 32;
9
10        wire [n-1:0] d;
11        wire lcin; // last carry in
12        wire cout;
```

```
13
14          assign d=b^{32{sub}}; //negate b for subtraction
15
16          //import 32 bit adder
17          adder #(32) add(
18          .a (a),
19          .b(d),
20          .cin(sub),
21          .s(s),
22          .cout(cout)
23          );
24          assign lcin = s[n-1]^a[n-1]^d[n-1] ;// proof: adder operand bits
                mod 2
25
26          assign neg = (~u & (a[n-1] ^d[n-1] ^cout)) | (u&~cout & sub);//
                lemma 39
27
28          assign ovf = u & (cout ^ lcin); // lemma 40
29          endmodule
```

Code 2: n-au module

figure below is the diagram for n-arithmetic unit:

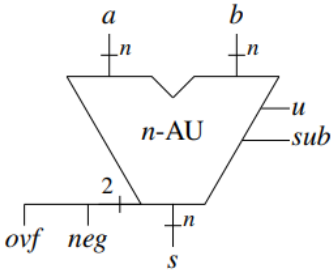


Figure 4: n-AU symbol ([5], Fig. 38, p. 90)

As we can see, the arithmetic unit takes operands a and b as inputs (on which arithmetic operations are performed), along with two control signals, u and sub, and three outputs: ovf, neg, and s.

We create a 32-bit mask: if the sub bit is 1, the mask becomes 1^{32}. XORing this mask with b negates every bit of b. Additionally, when $sub = 1$, the carry-in (cin) is also set to 1, which is justified by Lemma 24 on binary subtraction (see Figure 5).

The diagram below illustrates the logic behind mask, followed by the subsequent instantiation of the adder module. The notation (#32) during instantiation corresponds to the concept introduced earlier, where we instantiate the adder module with a parameter value of 32.

The last three lines compute the flag bits. For a detailed correctness check, refer to Chapter 7.2: the correctness of the neg bit is established in Lemma 39, while the ovf bit computation is justified by Lemma 40.

For initial inputs it is tested for addition operations, that are handled well by our previous adder

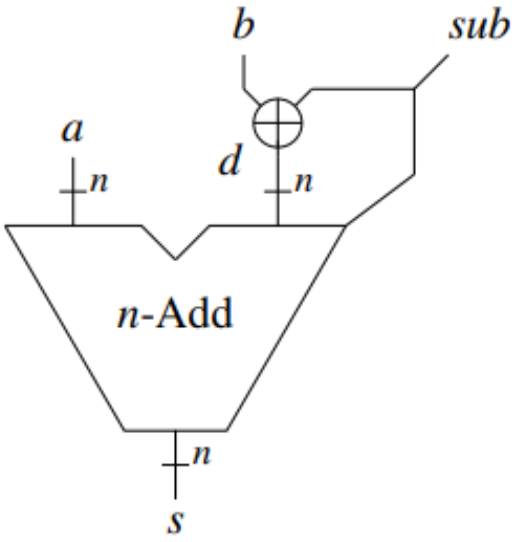


Figure 5: Data paths of an n-arithmetic unit

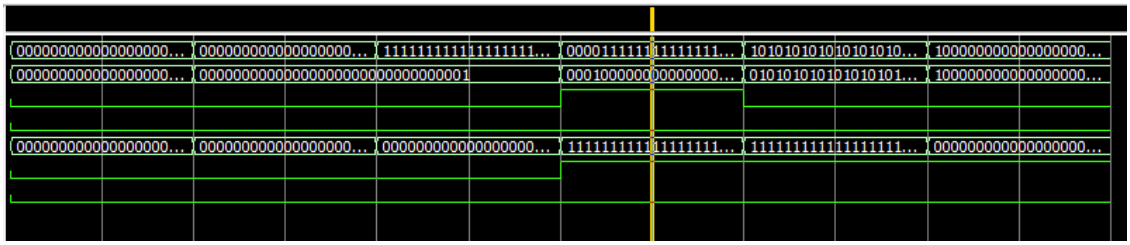


Figure 6: Waveforms for Arithmetic Unit, check chapter 19 for test bench.

module. the fourth input is for subtraction, $0^4 1^{28} - 0^3 10^{28}$ this should give us -1, which is the result in our simulation, since $[1^{32}] = [1^1]$ by sign extension lemma.

4 Arithmetic Logic Unit (ALU)

The ALU has the following inputs: 1) Operand inputs: $a = a[n-1:0] \in B^n$ and $b = b[n-1:0] \in B^n$ 2) Control inputs: $af[3:0] \in B^4$ and $i \in B$, specifying the operation that the ALU performs with the operands.

It has the following outputs: result $alures[n-1:0] \in B^n$, overflow bit $ovfalu \in B$.

As we can see from "Code 3", we have inputs a and b, along with i and af (lines 2-5), and corresponding outputs: $alures$ and $ovfalu$ (lines 6-7). $alures$ is of wire type. There are two general types for output: "output wire" or "output reg". The difference between them is that "reg" can only have a value assigned in an always block, while a wire can only have a value assigned outside an always block (if you try to assign a value inside an always block, you will get a compile-time error). Page 96 of the book justifies the assignment of values on line 25.

```
1         module alu #(parameter n = 32) (
2         input [n-1:0] a,
3         input [n-1:0] b,
4         input [3:0] af,
5         input i,
6         output [n-1:0] alures,
7         output ovfalu
8         );
9
10        wire [n-1:0] au_result;
11        wire au_neg, au_ovf;
12
13
14        au #(n) arithmetic_unit (
15        .a(a),
16        .b(b),
17        .sub(af[1]),
18        .u(af[0]),
19        .s(au_result),
20        .neg(au_neg),
21        .ovf(au_ovf)
22        );
23
24        wire [n-1:0] mux_result;
25        assign mux_result =
26        (af == 4'b0000 || af == 4'b0001 || af == 4'b0010 || af == 4'b0011)
              ? au_result :
27        (af == 4'b0100) ? a & b :
28        (af == 4'b0101) ? a | b :
29        (af == 4'b0110) ? a ^ b :
30        (af == 4'b0111) ? i ? {b[15:0], 16'b0} : ~(a | b) :
31        (af == 4'b1010 || af == 4'b1011) ? {31'b0, au_neg} :
32        {n{1'b0}};
33
34        assign alures = mux_result;
35        assign ovfalu = au_ovf & ~af[3] & ~af[2];
36
37        endmodule
```

Code 3: ALU Verilog Code

Now, to justify mux_result operations, we need to understand how control bits influence the ALU's functionality. The table below shows the ALU specification.

For control bits $af = 0000, 0001, 0010$ or 0011, we have only arithmetic operations, which can be handled by the arithmetic unit instantiated in the ALU module. The result of the arithmetic operation is stored in the variable *au_result* (line 19), so line 14 handles the arithmetic operations. Lines 27-29 handle the second group of operations (logical operations). Line 31 handles test-and-set instructions. Notice that if $a < b \leftrightarrow a - b < 0 \leftrightarrow neg = 1$ (by the ALU construction), *neg* is connected to *au_neg* (line 18). $\{31\{1'b0\}\}$ means 0^{31}, so taking the 0 bit 31 times. $\{0^{31}, au_neg\}$ - the last curly braces are used to concatenate 0^{31} with *au_neg*.

The first 4 tests, which are arithmetic operations on line 23 of the ALU module, are handled by our already tested arithmetic unit. The next 5 cases are logical operations. For example, at $af = 0100$ (third row of the waveform), the operation is an AND operation. We chose inputs such

$af[3:0]$	i	$alures[n-1:0]$	$ovfalu$
0000	*	$a +_n b$	$[a]+[b] \notin T_n$
0001	*	$a +_n b$	0
0010	*	$a -_n b$	$[a]-[b] \notin T_n$
0011	*	$a -_n b$	0
0100	*	$a \wedge b$	0
0101	*	$a \vee b$	0
0110	*	$a \oplus b$	0
0111	0	$\overline{a \vee b}$	0
0111	1	$b[n/2-1:0]0^{n/2}$	0
1010	*	$0^{n-1}([a] < [b] \ ? \ 1:0)$	0
1011	*	$0^{n-1}(\langle a \rangle < \langle b \rangle \ ? \ 1:0)$	0

Figure 7: ALU specification ([5], Fig. 40, Table 7, p. 95)

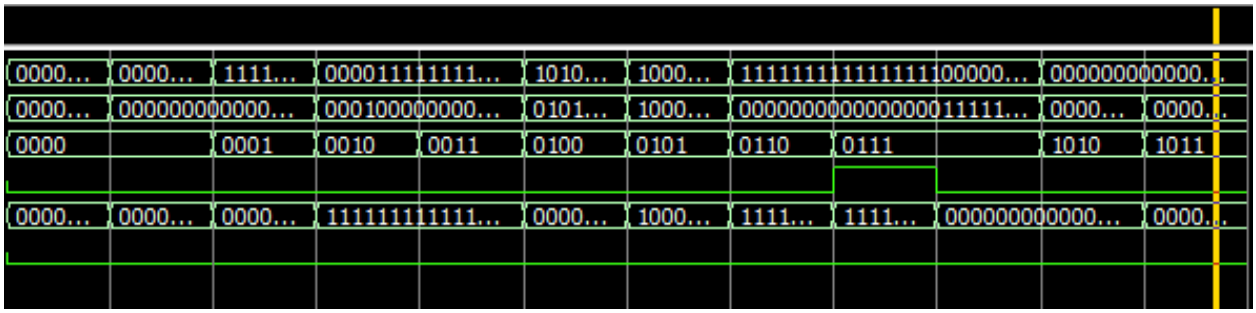


Figure 8: test pattern for all possible values of af and i row 1: operand a, row 2: operand b, row 3: af, row 4: i, row 5: output result, row 6: ovfalu. click to check ALU test bench

that $\forall i, a_i \neq b_i$, so the result is 0. The last 2 operations to be tested are test-and-set operations, which also use the arithmetic unit. Since $a < b \leftrightarrow a - b < 0 \leftrightarrow neg = 1$, for the first case, we took $a = b$, so the result is 0. For the second case, $a = 1$ and $b = 2$, so the result is 1.

5 Shifter

Our final processor will support the SRL (Shift Right Logical) instruction, which is an R-type instruction. For this, we need the corresponding hardware (in Verilog, a separate module that would handle this). To implement this, we use fixed-distance shifters to build a shifter that shifts by the specified input (the "sa" field in the R-type instruction, which denotes the shift amount).

5.1 fixed distance shifter

For $a \in B^n$ and $i \in [n-1]$, we define the logical right shift $srl(a,i)$ of a by i positions as

$$srl(a,i) = 0^i a[n-1:i],$$

thus operand a is shifted by i positions to the right. Unused bits are filled with zeroes.

For a fixed $i \in [0:i-1]$, the circuit satisfies the following specification:

$$a' = \begin{cases} srl(a,i), & \text{if } s = 1, \\ a, & \text{if } s = 0. \end{cases}$$

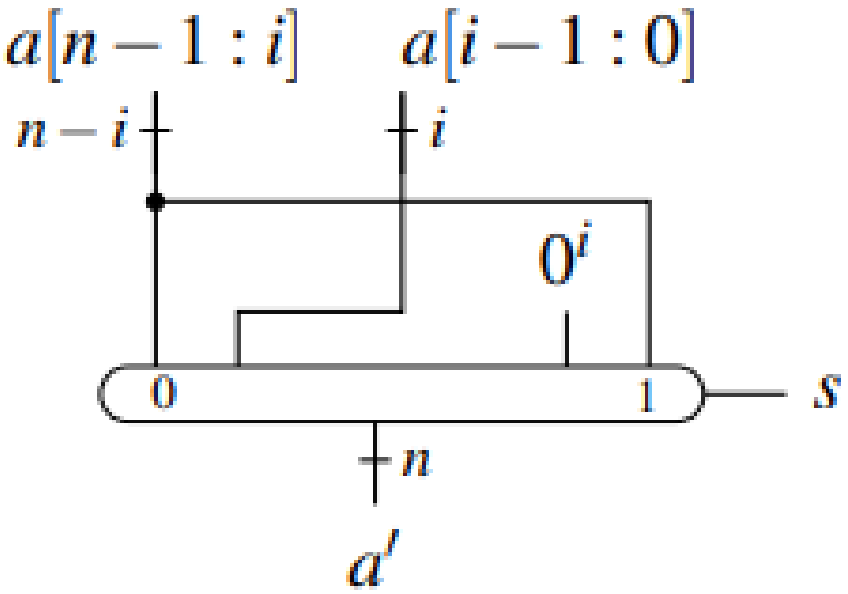


Figure 9: construction of an (n,i)-logical right shifter ([5], Fig. 42, p. 97)

Given below is the code that implements a fixed-distance shifter. It is a parameterized module (which will be instantiated later with various parameter values; see the next subsection). Lines 2-3 declare the inputs (operand a and enable signal s), and line 4 declares the output *res* (the shifted result). Line 7 computes $0^i a[n-1:0]$.

```
1         module srlfix #(parameter i = 16)(
2         input  [31:0] a,
3         input  s,
4         output wire [31:0] res
5         );
6
7         assign res = s == 1'b1 ? ({{i{1'b0}}, a[31:i]}) : a;
8
```

```
9           endmodule
```

Code 4: fixed-size shifter code

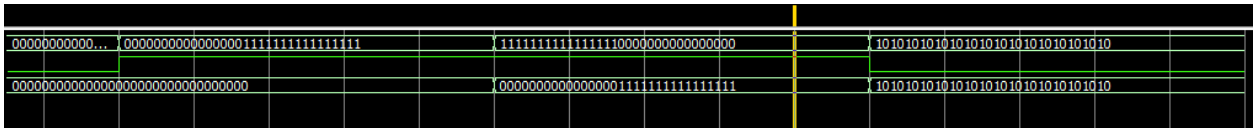


Figure 10: waveforms for fixed distance shifter, row 1: input a, row 2: input s, row 3: output res. click here to check fixed distance shifter test bench

We test for 4 inputs. The first input is 0^{32}, and the enable signal is 0, so the result will also be 0^{32}. The second case is $0^{16}1^{16}$, so if we shift to the right by 16 bits, we get 0^{32}. For the third case, we have $1^{16}0^{16}$, so after shifting, those 1s will move to the leftmost position, giving us $0^{16}1^{16}$. The last case is $(10)^{16}$, and after shifting, we get the same result due to the input pattern. Note that in all cases above, the shift amount is 16. This parameter will vary in the barrel shifter construction.

5.2 Barrel Shifter using fixed distance shifters

An n-logical right shifter, or for short n-SRL, is a circuit with inputs: $a[n-1:0] \in B^n$ and $b[k-1:0] \in B^k$, and output $shres[n-1:0] \in B^n$, satisfying $shres(a,b) = srl(a,\langle b \rangle)$. We build this by stacking (n,i)-SRLs.

We use bits of input b as enable signals. If index i is 0, then that power of 2 is skipped in shifting. Notice that the output of one fixed-distance shifter is used as an input for the next one.

As we've mentioned, we have inputs a (the operand that needs to be shifted, line 3), the shift amount input (line 4), and output res (line 5, the shifted result). $res0, res1, res2, res3$ are intermediate results. Notice the power of the parameter usage here. Lines 9-13 are used with parameter $i = 1$. Initially, the input is a, and the output is stored in $res0$. After that, the input (where $res0$ is connected to a of the $srlfix$ module) is processed, and the output is now stored in $res1$. Finally, with parameter value $i = 16$, the input is $res3$ (the previous fixed-distance shifter result, line 33), and the output is connected to res (declared on line 5). Here, we see the power of the parameterization and hierarchical design. First, we designed the fixed-distance shifter (Section 4.1), and then we instantiated the $srlfix$ module with parameters to simulate the concept in Figure 9.

```
1           module srl(a,b,res);
2
3           input  [31:0] a;
4           input  [4:0] b;
5           output [31:0] res;
6
7           wire [31:0] res0, res1, res2, res3;
8
```

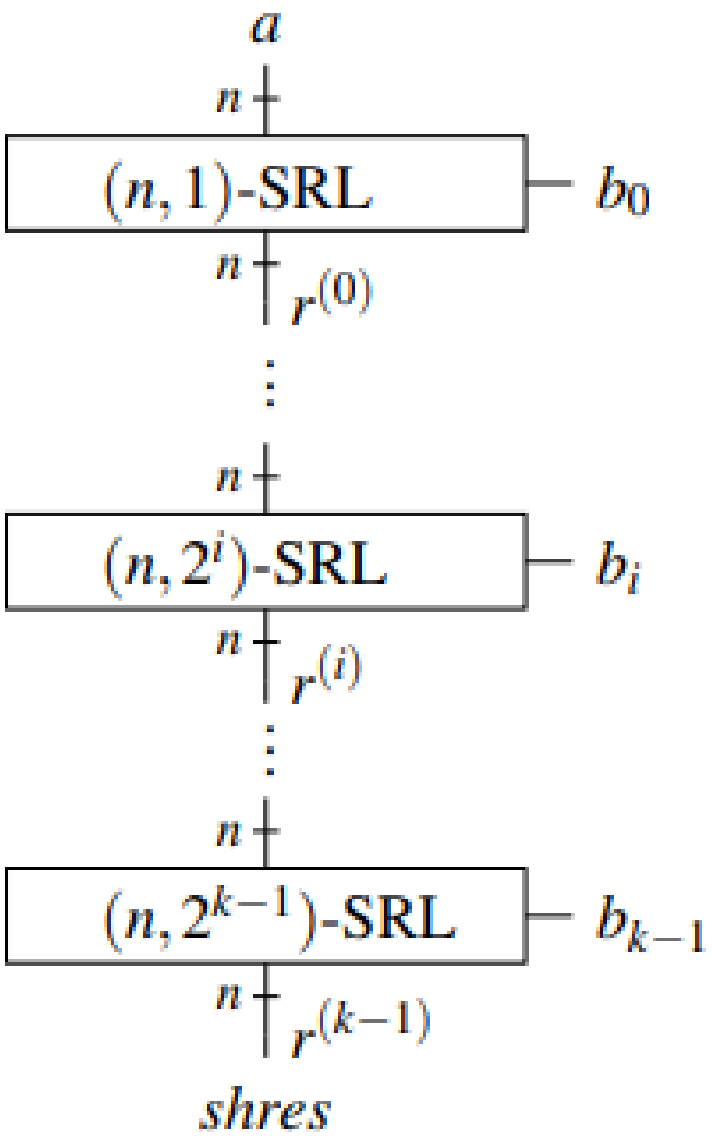


Figure 11: construction using stack of fixed distance shifters ([5], Fig. 43, p. 97)

```
srlfix #(1) s0 (
.a(a),
.s(b[0]),
.res(res0)
);

srlfix #(2) s1 (
.a(res0),
.s(b[1]),
.res(res1)
);

srlfix #(4) s2 (
.a(res1),
.s(b[2]),
.res(res2)
);

srlfix #(8) s3 (
```

```
28          .a(res2),
29          .s(b[3]),
30          .res(res3)
31          );
32
33          srlfix #(16) s4 (
34          .a(res3),
35          .s(b[4]),
36          .res(res)
37          );
38
39          endmodule
```

Code 5: barrel shifter code

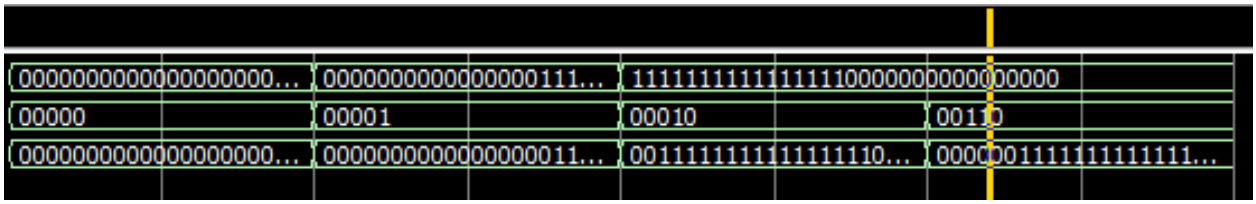


Figure 12: waveforms of shift right logical module, row 1: input a, row 2: input b (shift amount), row 3: res. click here to check srl.v test bench

For the first input, the result is the same since the shift amount is 0. For the second input, we shift right by 1, $\langle 00001 \rangle = 1$, so the shift amount is 1. In the second case, we shift by 2, since $\langle 00010 \rangle = 2$, and we can clearly see 2 extra 0s added on the left. Finally, we use a combination and shift by 6, since $\langle 00110 \rangle = 6$. As we can see, 6 extra 0s are added on the left.

6 Branch Condition Evaluation Unit(BCE)

Branching is an important part of computation; we need to evaluate conditions and, based on that, make decisions. For this, we have a separate piece of hardware called the Branch Condition Evaluation unit (BCE).

An n-BCE is a circuit with inputs: $a[n-1:0] \in B^n$, $b[n-1:0] \in B^n$, and $bf[3:0] \in B^4$, selecting the condition to be tested. The output is $bcres \in B$.

Below is the code for the BCE module. Lines 3-5 declare the inputs as mentioned in the specification above: line 3 declares the first operand, line 4 declares the second operand, and line 5 declares the selector for the condition. Line 6 declares the output. Lines 7-8 declare auxiliary signals. Lines 10-16 contain the Boolean algebra, copied and translated into Verilog code (from page 99 of [5], Chapter 7.5).

```
1           module bce(a,b,bf,bcres);
2
3           input  [31:0] a;
```

$bf[3:0]$	$bcres$
0010	$[a] < 0$
0011	$[a] \geq 0$
100*	$a = b$
101*	$a \neq b$
110*	$[a] \leq 0$
111*	$[a] > 0$

Figure 13: specification of BCE, chapter 7.5, page 99

```
4          input  [31:0] b;
5          input [3:0] bf;
6          output bcres;
7          wire [31:0] d;
8          wire eq, lt,le;
9
10         assign d= (bf[3]&~bf[2]?b:{32{1'b0}});
11         assign eq = (a==d);
12         assign lt= a[31];
13         assign le= lt | (a=={32{1'b0}}); // simplified from book
14         assign bcres = (~bf[3] & ~bf[2] & bf[1] & (bf[0] ^ lt)) |
15         (bf[3] & ~bf[2] & (bf[1] ^ eq)) |
16         (bf[3] & bf[2] & (bf[1] ^ le));
17
18         endmodule
```

Code 6: BCE code

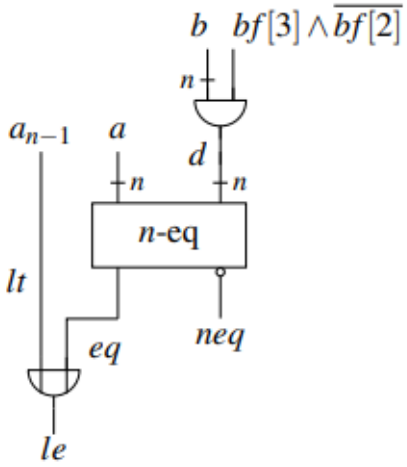


Figure 14: circuit of auxiliary signal computation ([5], Fig. 45, p. 99)

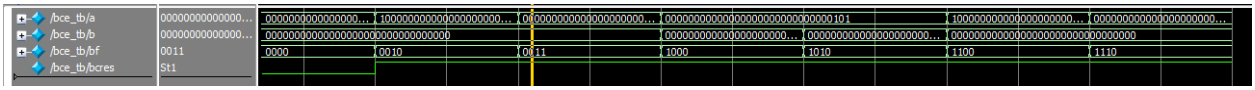


Figure 15: Waveforms for BCE unit. See BCE module test bench.

The input pattern is generated such that for all condition tests, the result is 1. Below are the

inputs for the test bench. As we can see, in all cases, the output should be 1.

```
        // Test case 1: bf = 0010, [a] < 0
        a = 32'b10000000000000000000000000000000; // Negative number
        b = 32'b00000000000000000000000000000000; // Not used in this case
        bf = 4'b0010;
        #10;

        // Test case 2: bf = 0011, [a] >= 0
        a = 32'b00000000000000000000000000000001; // Positive number
        b = 32'b00000000000000000000000000000000; // Not used in this case
        bf = 4'b0011;
        #10;

        // Test case 3: bf = 1000, a == b
        a = 32'b00000000000000000000000000000101;
        b = 32'b00000000000000000000000000000101;
        bf = 4'b1000;
        #10;

        // Test case 4: bf = 1010, a != b
        a = 32'b00000000000000000000000000000101;
        b = 32'b00000000000000000000000000000110;
        bf = 4'b1010;
        #10;

        // Test case 5: bf = 1100, [a] <= 0
        a = 32'b10000000000000000000000000000000; // Negative number
        b = 32'b00000000000000000000000000000000; // Not used in this case
        bf = 4'b1100;
        #10;

        // Test case 6: bf = 1110, [a] > 0
        a = 32'b00000000000000000000000000000001; // Positive number
        b = 32'b00000000000000000000000000000000; // Not used in this case
        bf = 4'b1110;
        #10;
```

Code 7: part of test bench code for BCE

For the first test case, since the most significant bit is 1, $\rightarrow a < 0$ by Lemma 23 (Chapter 4.2, page 38),

$$[a] < 0 \leftrightarrow a_{n-1} = 1 \quad \text{(sign bit)}.$$

For the second test case, $a = 1 \geq 0$, thus the output is 1.

For the third test case, $a = b$ holds, thus the output is again 1. Similarly, for test case 4, $a \neq b$ also holds, thus the output is 1 again.

For test case 5, $a_{n-1} = 1 \leftrightarrow a < 0 \leq 0$, thus the output is again 1.

Finally, for the last test case, $a = 1 > 0$, thus the output is again 1. So, the waveform of the output being 1 all the time is justified by the input pattern.

7 Auxiliary Modules

7.1 Immediate Extension Unit

In our processor, the immediate field in I-type instructions or the index in J-type instructions have a length less than 32. Therefore, the ALU (to appropriately operate on operands) requires a 32-bit length input. For this, we use an extension unit that extends constants based on the control input u. If u is 1, then we have unsigned computation, so we extend with 0 bits. Otherwise, we extend with the most significant bit.

```
1         module ieu #(parameter in = 16,out = 32) (
2         input u,
3         input [in-1:0] immediate_in,
4         output [out-1:0] immediate_out
5         );
6
7         wire bit_to_repeat;
8         wire [(out-in)-1:0] concat;
9
10        assign bit_to_repeat = (u == 1'b0) ? immediate_in[in-1] : 1'b0;
11        assign concat = { (out-in){bit_to_repeat} };
12        assign immediate_out = { concat, immediate_in };
13
14        endmodule
```

Code 8: Extension Unit code

The module is parameterized (default parameter values are 16 and 32; 16 is the length of the immediate field in I-type instructions, and 32 bits is the ALU operand's length). We have two inputs (lines 2-3): u, which decides whether signed or unsigned extension occurs, and immediate_in, which is the input that needs to be extended. Line 4 declares the output (extended result). Line 7 declares an auxiliary variable, which is 0 if the unsigned signal U is 1, and otherwise, it's the most significant bit. Line 11 essentially computes: $bit_to_repeat^{OUT-IN}$, and line 12 finally computes the extended result.

7.2 Splitter module(used in instruction decoder)

This module is based on the subdivision of the bits of the instruction:

It was simply used in the project to make the instruction decoder module more readable and clean. It splits the 32-bit instruction into important fields, which are then used in the instruction decoder to determine the values of certain predicates.

```
1         //Module responsible of splitting up arguments. (rs/rt/rd/etc)
2         //This allows simplification of the ALU as well as some other
             modules.
3         module splitter(
4         input [31:0] instruction_in,
```

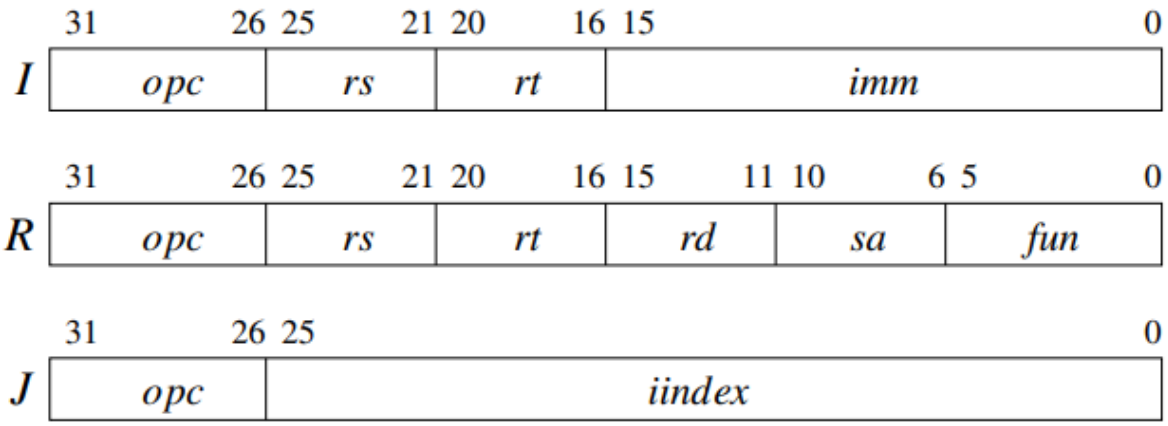


Figure 16: division of instructions ([5], Fig. 55, p. 113)

```
5           output[5:0] opc, fun,
6           output[4:0] rs, rt, rd, sa,
7           output[15:0] imm,
8           output[25:0] iindex
9           );
10
11          assign opc = instruction_in[31:26];
12          assign rs = instruction_in[25:21];
13          assign rt = instruction_in[20:16];
14          assign rd = instruction_in[15:11];
15          assign sa = instruction_in[10:6];
16          assign fun = instruction_in[5:0];
17
18          assign imm = instruction_in[15:0];
19          assign iindex = instruction_in[25:0];
20
21          endmodule
```

Code 9: Splitter unit code

7.3 General Purpose Register File (GPR)

We have the GPR as a separate module, which has three address inputs: two for reading and one for writing, along with write enable, reset, and clock signals. The implementation of this hardware is explained in detail in Chapter 6.4 of the [5]. Our module is the result of mapping the specification into Verilog syntax.

A minor change to the original design, however, is the introduction of the stack pointer, by default pointing at HEX 3FFC. This was done to match MARS Assembler implementation, which uses that same value by default. If we didn't do this, code compiled by MARS would misbehave during loads and saves. By default a stack pointer is used for lw/sw offset, if we set it to 0 like others, generated machine code would overwrite itself by writing data at addresses where the code is.

```
1 module gpr (
2     input clk, rst,
3     input wire we,
```

```
4       input wire [4:0] ra1, ra2, wa,
5       output wire [31:0] rd1, rd2,
6       input wire [31:0] wd,
7       output wire [31:0] register_out
8   );
9
10  reg [31:0] registers [31:1] /* synthesis keep */;
11  integer i;
12
13  assign rd1 = (ra1 == 5'b0) ? 32'b0 : registers[ra1];
14  assign rd2 = (ra2 == 5'b0) ? 32'b0 : registers[ra2];
15  assign register_out = registers[5];
16
17  initial begin
18      for (i = 1; i < 32; i = i + 1)
19          registers[i] = 0;
20      // This is the stack pointer. Implemented to match MARS assembler
21      registers[29] = 11'b01111111111;
22  end
23
24  always @(posedge clk or posedge rst) begin
25      if (rst) begin
26          for (i = 1; i < 32; i = i + 1)
27              registers[i] <= 32'b0;
28          // This is the stack pointer. Implemented to match MARS assembler
29          registers[29] <= 11'b01111111111;
30      end else if (we && wa != 5'b0) begin
31          registers[wa] <= wd;
32      end
33  end
34
35  endmodule
```

Code 10: GPR code

clk, write enable and rst signal are declared as inputs (2-3), line 4 declares input addresses (ra1 and ra2 for reading, wa for writing), line 5(read result stored in those variables), line 6 (input that has to be written if we have write enable). instead of 32 length array, we have 31 cells in our array, each of 32 bit length, if address wants to read from 0, it will always get 0 as a result (lines 13-14). lines 31-33 is for writing logic. line 20 and lines 28-30 are justified by general black-box diagram for the GPR file is in figure 12.

7.4 ALUDecoderBridge

This module is used in `CPU.v` to make the code cleaner and more readable. It simply maps the opcode to the corresponding control signals for the ALU, after which our ALU from `alu.v` is instantiated.

```
1       module alu_decoder_bridge (
2       input [5:0] opcode,
3       input [31:0] a,
4       input [31:0] b,
5       input [3:0] af,
6       input i,
7       output [31:0] result,
```

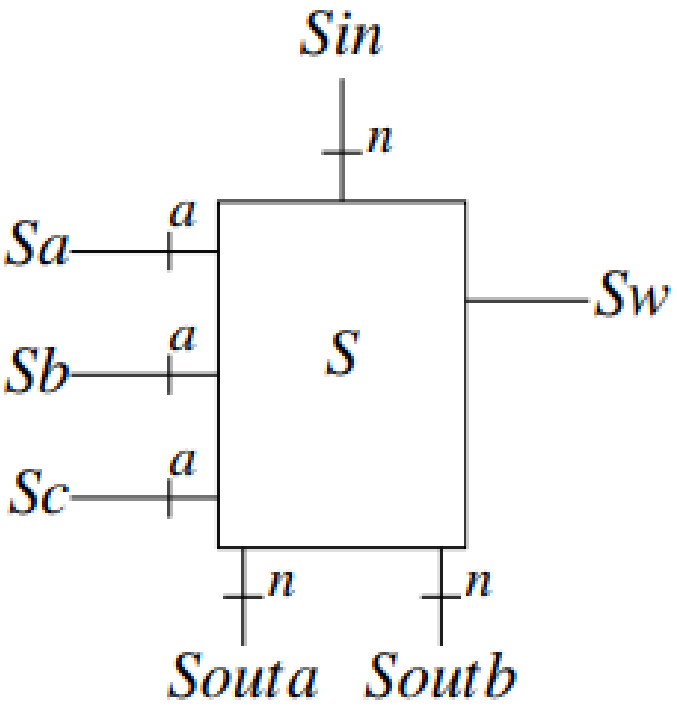


Figure 17: GPR file diagram ([5], Fig. 30, p. 76)

```
8           output ovfalu
9           );
10
11          wire [3:0] af_alu =
12          ~i ? af :
13          opcode == 6'b001001 ? 4'b0001 :
14          opcode == 6'b001000 ? 4'b0000 :
15          opcode == 6'b001011 ? 4'b1011 :
16          opcode == 6'b001010 ? 4'b1010 :
17          opcode == 6'b001100 ? 4'b0100 :
18          opcode == 6'b001101 ? 4'b0101 :
19          opcode == 6'b001110 ? 4'b0111 :
20          opcode == 6'b001111 ? 4'b0111 :
21          opcode == 6'b100011 ? 4'b0001 :
22          opcode == 6'b101011 ? 4'b0001 :
23          4'b1111; // forcing alu to fall back to default.
24
25          alu ula (a,b,af_alu,i,result,ovfalu);
26
27          endmodule
```

Code 11: ALUDecoderBridge

Opcode translation into `af` is done in a brute-force way, simply by considering all opcode possibilities where arithmetic operations are needed.

7.5 Shifter.v module

In the initial design, we intended to have multiple shift modes. Instead of using a barrel shifter, as introduced in Chapter 4, we use built-in operators. One can instantiate a barrel shifter as well; however, in our design decision, we preferred this approach.

```
module shifter (
input  [1:0]  funct,
input  [31:0] a,
input  [4:0]  shamt,
output wire [31:0] res
);

assign res =    (funct == 2'b00) ? (a << shamt) :   // Logical
   left shift
(funct == 2'b01) ? (a >> shamt) :   // Logical right shift
(a >>> shamt);                      // Arithmetic right shift

endmodule
```

Code 12: Shifter.v

7.6 Helper modules for decoding stage

This module interprets 40 bit data that is an output of decoder module (check chapter 7).

```
module decoder_deconcat (
input [39:0] packed_in,
output [3:0] af,
output i,
output alu_mux_sel,
output [2:0] shift_type,
output [4:0] cad,
output gp_we,
output [1:0] gp_mux_sel,
output [3:0] bf,
output [1:0] pc_mux_select,
output mem_wren,
output mem_rren,
output [4:0] rs, rt, rd
);

assign {af, i, alu_mux_sel, shift_type, cad, gp_we,
    gp_mux_sel, bf, pc_mux_select, mem_wren, mem_rren,
    rs, rt, rd} = packed_in;

endmodule
```

Code 13: decoder deconcat.v

This module performs the opposite function of the deconcat module and is used in the main decoder module in Chapter 8.

```
module decoder_concat (
input [3:0] af,
input i,
input alu_mux_sel,
input [2:0] shift_type,
input [4:0] cad,
```

```
    input gp_we,
    input [1:0] gp_mux_sel,
    input [3:0] bf,
    input [1:0] pc_mux_select,
    input mem_wren,
    input mem_rren,
    input [4:0] rs, rt, rd,
    output [39:0] packed_out
);

assign packed_out = {af, i, alu_mux_sel, shift_type, cad, gp_we,
                        gp_mux_sel, bf, pc_mux_select, mem_wren, mem_rren,
                        rs, rt, rd};

endmodule
```

Code 14: decoder concat.v

7.7 Helper module for writeback stage

Given module tells us which data needs to be written in GPR file,

```

        module writeback_master(
        input [31:0] memory_out,
        input [31:0] alu_res,
        input [31:0] shift_res,
        input [31:0] link_addr,
        input [1:0] gp_mux_sel,
        output [31:0] gpr_data_in
        );

        assign gpr_data_in =
        (gp_mux_sel == 2'b00) ? alu_res :
        (gp_mux_sel == 2'b01) ? memory_out :
        (gp_mux_sel == 2'b10) ? shift_res :
        (gp_mux_sel == 2'b11) ? link_addr :
        32'b0;

        endmodule
```

Code 15: writeback_master.v

8 Modules For Decoding Stage

The Instruction Decoder, along with the ALU, is one of the most important modules in our hierarchical design. It generates essential fields and predicates that control the computation process. The schematic representation of the Instruction Decoder is given in Figure 17.

The module utilizes verilog functions to create a more organized and understandable structure. Namely, we have the following functions:

- Type (in: instruction, out: Type indicator bits) - determines what type of an instruction we have. 00 - I-type, 01 - J-type, 10 - R-type. Is only used for calculations inside the module, it is not an output.
- IsWritten (in: opcode, func, out: gpr write enable) - Determines whether GPR writeenable should be on or not.
- WhereFrom (in: opcode, func, out: GPR source select) - determines the source of write in gpr. Assigned to gpr_mux_sel. 00 - ALU, 01 - Memory, 10 - Shifter, 11 - PC.
- PCSelect (in: opcode, func, out: PC source select) - determines the source of next PC. Assigned to pc_mux_sel. 00 - PC+4, 01 - Branching, 10 - J/Jal, 11 - Jr/Jalr.

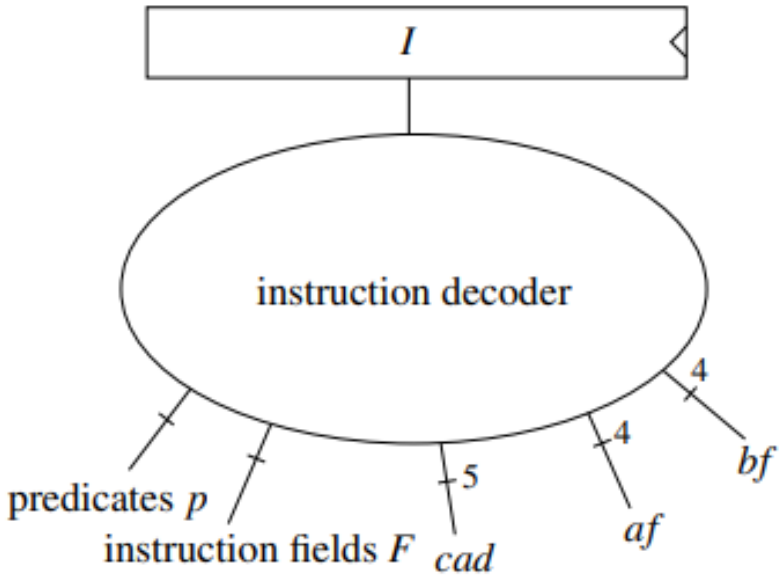


Figure 18: Instruction Decoder Diagram(page 129, chapter 8.3.6)

Results of the hardware are all concatenated on the out_data_packed with the decoder_concat module. This simplifies input/output of the decoder signals. They're subsequently decoded by the decoder_deconcat module and its results are used in execute stage.

The only input for the `i_decoder` module is a 32-bit instruction. The decoded result is packed in a 40-bit signal, which is the output of our module. Line 19 utilizes the `splitter` module to get some essential signals; the `splitter` module is described in Chapter 7.2. Line 21, based on the `Type` function, returns the type of instruction and assigns it to the corresponding wire. Line 22 computes `af` (ALU control signal) based on the results from [5]. The single-bit wire `i` is 0 if we

have an Rtype instruction and 1 otherwise. Line 25 computes where to store the result in gpr, if we have write enable in gpr. Based on the opcode and function code, we determine whether the gpr write enable signal should be on in Line 26. Line 27 computes input data for GPR based on the WhereFrom function. Line 28 computes BCE control bits. Line 29 enables memory write based on the opcode (it is 1 if we have a load instruction), similarly for Line 30. On Line 31, we compute the type of the shift. Line 36 computes cadn; if it's Jalr or jal, then it's 31, otherwise it's cad. We concatenate all of those computed results using the decoder_concat module.

The code that implements the hardware described above is given below:

```
1  module i_decoder(
2  input wire [31:0] instruction,
3  output [4:0] rs, rt,
4  output wire [39:0] out_data_packed
5  );
6
7  wire [5:0] opc, fun;
8  wire [4:0] rd, sa;
9  wire [15:0] imm;
10 wire [25:0] iindex;
11 wire [3:0] af;
12 wire i, alu_mux_sel, gp_we, mem_wren, mem_rren;
13 wire [2:0] shift_type;
14 wire [4:0] cad, cadn;
15 wire [1:0] gp_mux_sel, pc_mux_select;
16 wire [3:0] bf;
17 wire [1:0] ttype;
18
19 splitter splt(instruction, opc, fun, rs, rt, rd, sa, imm, iindex);
20
21 assign ttype = Type(opc);
22 assign af = i ? {(~instruction[28] & instruction[27]), instruction
      [28:26]} : instruction[3:0];
23 assign i = ttype != 2'b10;
24 assign alu_mux_sel = ttype != 2'b10;
25 assign cad = i ? rt : rd;
26 assign gp_we = IsWritten(opc, fun);
27 assign gp_mux_sel = WhereFrom(ttype, opc, fun);
28 assign bf = {instruction[28:26], instruction[16]};
29 assign mem_wren = opc == 6'b101011; // sw instruction
30 assign mem_rren = opc == 6'b100011; // lw instruction
31 assign shift_type = af == 3'b000 ? {ttype != 2'b10, 2'b00} : //
      sll
32 af == 3'b010 ? {ttype != 2'b10, 2'b01} : // srl
33 {ttype != 2'b10, 2'b10}; // sra
34 assign pc_mux_select = PCSelect(opc, fun);
35
36 assign cadn = opc == 6'b000011 || opc == 6'b0 && fun == 4'b1001 ?
      31 : cad; //For Jalr, Jal
37
38 decoder_concat concat_inst (
39 .af(af),
40 .i(i),
41 .alu_mux_sel(alu_mux_sel),
42 .shift_type(shift_type),
43 .cad(cadn),
44 .gp_we(gp_we),
45 .gp_mux_sel(gp_mux_sel),
46 .bf(bf),
```

```
47          .pc_mux_select(pc_mux_select),
48          .mem_wren(mem_wren),
49          .mem_rren(mem_rren),//readenable
50          .packed_out(out_data_packed),
51          .rs(rs),
52          .rt(rt),
53          .rd(rd)
54          );
55
56          function [1:0] Type(input [5:0] instruction);
57          casez (instruction)
58          6'b001???, 6'b1???11, 6'b0001??, 6'b000001: Type = 2'b00;//I-Type
59          6'b00001?: Type = 2'b01;//J-Type
60          default: Type = 2'b10;//R-type
61          endcase
62          endfunction
63
64          function [0:0] IsWritten(input [5:0] opc, input [5:0] fun);
65          casez(opc)
66          6'b001???, 6'b100011, 6'b000011: IsWritten = 1'b1;
67          6'b000000: IsWritten = fun != 6'b001000;
68          default: IsWritten = 1'b0;
69          endcase
70          endfunction
71
72          function [1:0] WhereFrom(input [2:0]ttype, input [5:0] opc, input
               [5:0]fun);
73          case(ttype)
74          2'b10: casez(fun)
75          6'b001001: WhereFrom = 2'b11; // PC
76          6'b000010, 6'b000011, 6'b000000: WhereFrom = 2'b10; //Shift
77          default: WhereFrom = 2'b00; // ALU
78          endcase
79          2'b00: if(opc == 6'b100011) WhereFrom = 2'b01; //Memory
80          else casez (opc)
81          6'b001???: WhereFrom = 2'b00; //ALU
82          default: WhereFrom = 2'b11; //PC
83          endcase
84          default: WhereFrom = 2'b11;
85          endcase
86          endfunction
87
88          function [1:0] PCSelect(input [5:0] opc, input [5:0] fun);
89          casez(opc)
90          6'b000000: PCSelect =
91          (fun == 6'b001000 || fun == 6'b001001) ?
92          2'b00 : 2'b11; //Jr/Jalr
93          6'b0001??, 6'b000001: PCSelect = 2'b01; //Branch
94          6'b00001?: PCSelect = 2'b10; //J/Jal
95          default: PCSelect = 2'b11;
96          endcase
97          endfunction
98
99          endmodule
```

Code 16: i_decoder module

8.1 Next PC environment

This module and the module above are two fundamental parts of the decoding stage. We then combine these two into a single master module, which encapsulates the entire decoding stage.

```
1
2         module next_pc_calc(
3     input wire [31:0] pc,
4     input wire [31:0] a_gpr, b_gpr,
5     input wire [31:0] i_fetch,
6     input wire [39:0] decode_data_packed,
7     output wire [31:0] next_pc
8 );
9
10 // Splitter logic
11 wire [15:0] imm;
12 wire [25:0] iindex;
13 splitter splt(.i_input(i_fetch), .imm(imm), .iindex(iindex));
14
15 // Unpack only the necessary wires from decode_data_packed
16 wire [3:0] bf;
17 wire [1:0] pc_mux_select;
18 decoder_deconcat deconcat_inst (
19     .packed_in(decode_data_packed),
20     .bf(bf),
21     .pc_mux_select(pc_mux_select)
22 );
23
24 // Branch condition evaluation unit
25 wire bcres;
26 bce bce(a_gpr, b_gpr, bf, bcres);
27
28 // Next PC calculation logic
29 wire [31:0] incr = pc + 4;
30 wire [31:0] pc_imm_sum = $unsigned($signed(pc+4) + $signed(imm << 2));
31 wire [31:0] jjal = {incr[31:28], iindex, 2'b00};
32 wire [31:0] mux_bcres = bcres ? pc_imm_sum : incr;
33
34 assign next_pc =
35     (i_fetch == 32'b0)       ? pc         : // Halt signal
36     (pc_mux_select == 2'b11) ? incr       :
37     (pc_mux_select == 2'b01) ? mux_bcres  :
38     (pc_mux_select == 2'b10) ? jjal       :
39                                a_gpr;
40
41 endmodule
```

Code 17: next_pc_calc.v

Module Above captures essential idea of the following circuit:

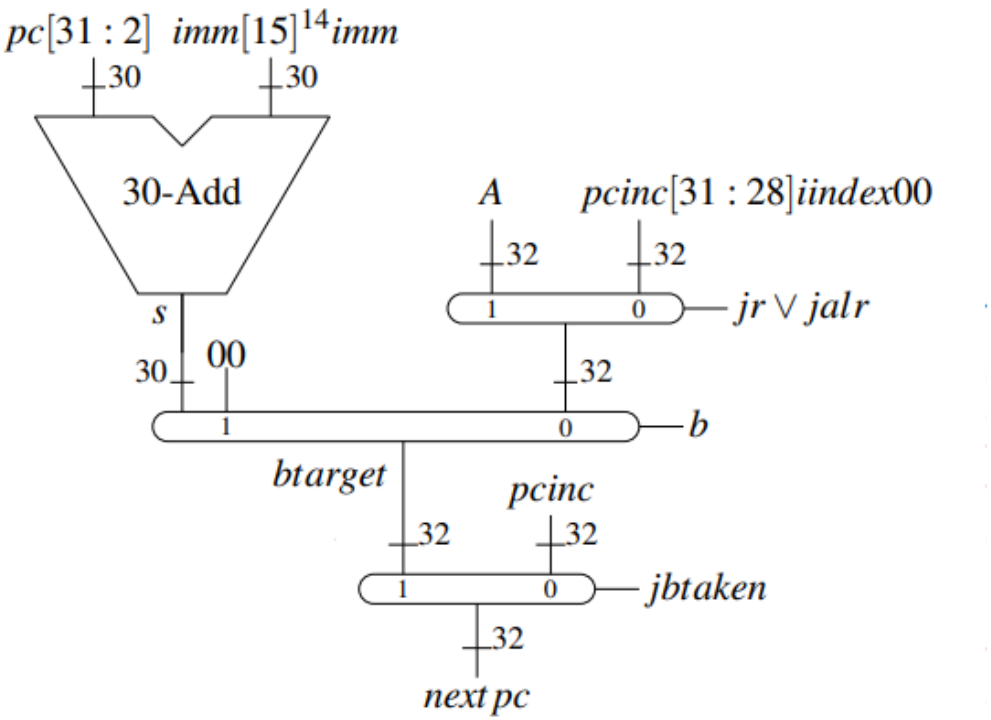


Fig. 64. Next PC computation

Figure 19: Next PC computation ([5], Fig. 64, p. 134)

As input, we have the current program counter, based on which we compute the next one. We also have two gpr operands and the current instruction, which is used to calculate the next program counter. Additionally, we have a 40-bit packed data signal of predicates, which is also used for the next PC computation. The only output of the module, declared on line 7, is the next program counter.

The `i_decoder` module, described above, provides a 40-bit output of packed predicates. However, this 40-bit signal does not include the immediate constant. Therefore, if we need the immediate value during the computation of the `nextPC` address, we also need the original 32-bit instruction. Based on that 32-bit instruction, we use the `splitter` module again on line 13. However, this time we only need the `immediate` and `iindex` fields from the split result. (We could have made `iindex` and `imm` part of the packed decoded result, but that would nearly double the size of that data.)

So instead, we have a separate input for the `next_pc_calc` module: the 32-bit instruction. That instruction is used to generate `decode_data_packed`, which is also an input to the `nextpc` module, and it also provides us with `iindex` and `immediate` constants. Therefore, the 40-bit packed predicates alone are not sufficient — we also need the `iindex` and `immediate` constants.

That is why the 32-bit instruction is also included as an input, and as an invariant, we assume that `decode_data_packed` is produced by the instruction `i_fetch`.

On line 18, we deconcatenate those 40 bits. From that, we extract `bf` (control for the `BCE` unit) and `pc_mux_select` (which determines the source of the next PC). We compute `bcres` using our `BCE` unit on line 26. Then, we compute possible next PC values on lines 29–32, and based on `pc_mux_select`, we assign the final result on line 34.

8.2 Master Module that handles Decoder Stage

This module combines two modules and handles the decoding stage.

```
1         module decode_master(
2         input [31:0] a_gpr, b_gpr,
3         input [31:0] i_fetch,
4         input [31:0] pc,
5         output [31:0] next_pc,
6         output [4:0] rs, rt,
7         output [39:0] decoder_packed
8         );
9
10        i_decoder idc(
11        .rs(rs),
12        .rt(rt),
13        .instruction(i_fetch),
14        .out_data_packed(decoder_packed)
15        );
16
17        next_pc_calc next_pc_calc_inst(
18        .pc(pc),
19        .a_gpr(a_gpr),
20        .b_gpr(b_gpr),
21        .i_fetch(i_fetch),
22        .decode_data_packed(decoder_packed),
23        .next_pc(next_pc)
24        );
25
26
27        endmodule
```

Code 18: decode_master.v

9 Execute Stage

This module covers the Execute stage in our sequential machine. It instantiates all the modules discussed in the previous chapter to handle all the requirements of the Execute stage.

```
1
2         module execute_master(
3         input [39:0] decoder_packed,
4         input [31:0] a_decoder, b_decoder,
5         input [31:0] i_decoder,
6         input [31:0] pc,
7         output [31:0] alu_res, shift_res,
```

```
8           output [31:0] dm_in,
9           output [31:0] ea,
10          output ovfalu
11          );
12
13          wire [5:0] opc, fun;
14          wire [4:0] rs, rt, rd, sa;
15          wire [15:0] imm;
16          wire [25:0] iindex;
17          splitter splt(i_decoder, opc, fun, rs, rt, rd, sa, imm, iindex);
18
19          wire [31:0] imm_extended = {{16{imm[15]}}, imm};
20          wire [31:0] alu_b;
21          wire [3:0] af;
22          wire i, alu_mux_sel;
23          wire [2:0] shift_type;
24
25          decoder_deconcat deconcat_inst (
26          .packed_in(decoder_packed),
27          .af(af),
28          .i(i),
29          .alu_mux_sel(alu_mux_sel),
30          .shift_type(shift_type)
31          );
32
33          assign alu_b = alu_mux_sel ? imm_extended : b_decoder;
34          assign dm_in = b_decoder;
35
36          alu_decoder_bridge aludec(
37          .opcode(opc),
38          .a(a_decoder),
39          .b(alu_b),
40          .af(af),
41          .i(i),
42          .result(alu_res),
43          .ovfalu(ovfalu)
44          );
45
46          shifter shft (
47          .funct(shift_type),
48          .a(b_decoder),
49          .shamt(sa),
50          .res(shift_res)
51          );
52
53          assign ea = a_decoder + imm_extended;
54
55          endmodule
```

Code 19: Execute Stage

10 Memory

Every important part of the CPU has been discussed, except for the crucial and complex memory component. Initially, for simulation purposes, we use what we call a "mock memory module," which is a `.v` file—a Verilog module where memory is simulated as an array. For hardware, we use SRAM and LPDDR2 memory. Thanks to Sandro Lortkipanidze's effort, the CPU successfully

incorporates both types of memory.

Below is the `memory_master.v` module, which is declared in `master.v`. This is the memory using both LPDDR2 and SRAM memories. Right after this, we provide the register memory implementation, which we would advise be used while starting out with the design.

LPDDR2 and SRAM:

```
module memory_master(
input wire clk,
input wire mem_clk,
input wire rst,
input wire [29:0] addr_in,
input wire [31:0] data_in,
input wire mem_wren,
input wire mem_rren,
input wire gp_we,
output wire [31:0] out,
output reg E = 0,
// LPDDR2 Memory
output wire [26:0] address,
output wire [31:0] write_data,
input wire [31:0] read_data,
output wire read_req,
output wire write_req
);

wire [31:0] sram_q;

wire is_lpddr2_used = addr_in > {11{1'b1}} /* synthesis keep */;

wire [31:0] sram_q_grounded = (~E || E && mem_rren) ? sram_q : 0;
assign out = is_lpddr2_used ? read_data : sram_q_grounded;

wire sram_wren = E && mem_wren && !is_lpddr2_used;
always @(posedge clk or posedge rst) begin
if(rst)
E <= 0;
else
E <= ~E;
end

sram memory_sram (
.address(addr_in[11:0]),
.clock(mem_clk),
.data(data_in),
.wren(sram_wren),
.q(sram_q)
);

// LPDDR2
assign address = addr_in - {11{1'b1}};
assign write_req = E && mem_wren && is_lpddr2_used;
assign read_req = is_lpddr2_used && ~write_req && (E && mem_rren
   || ~E);
assign write_data = data_in;


endmodule
```

Code 20: Memory Module in CPU.v

10.1 Register Memory

This type of memory is now known as “Mock memory” in hardware. It’s as simple as it can get. We define an array of 32-bit registers and address them from the module input - much like how the GPR defined before works.

```
module memory_master_mock(
    input wire clk,
    input wire mem_clk,
    input wire rst,
    input wire [29:0] addr_in,
    input wire [31:0] data_in,
    input wire mem_wren,
    input wire mem_rren,
    output wire [31:0] out,
    input wire E
);
reg [31:0] memory [0:4096];
initial begin
    for (integer i = 0; i < 4096; i = i + 1)
        memory[i] = 32'h00000000;
    memory[0] = 31'h....; // assembly code here
    memory[...];
end
reg [31:0] out_reg = 0;
always @(posedge mem_clk) begin
    if (E && mem_wren)
        memory[addr_in] = data_in;
    if(~E || E && mem_rren)
        out_reg = memory[addr_in];
end
assign out = out_reg;
endmodule
```

Code 21: Memory Module in CPU.v

As you can see, we define an array of `reg [31:0] memory [0:4096];` and then in the `initial begin` block we specify the assembly code we want the CPU to execute.

In the `always` block, meanwhile:

- data input is assigned to memory at address `addr_in`
- output is latched onto an output register from address `addr_in`

Which of these happens depends on the cycle and `mem_wren`/`mem_rren`.

Reset is ignored. Memory isn’t meant to be reset entirely.

As for why we use 2 clocks - that is to keep consistency with other types of RAM. We will get to the reasoning behind this later.

11 CPU (Master Module to rule them all)

In this final module, we instantiate all the modules described above to construct the final CPU. Instantiation follows principle of 5 stage sequential CPU. Decode, execute and writeback stages are handled by corresponding master modules, 1 important detail here is "ifdef" part during memory instantiation, which based on testing environment instantiates different modules, if we are using modelsim simulation for testing it compiles mock memory module, however if we are testing our CPU on hardware, then it instantiates memory_master which is hardware memory described in corresponding chapters for LPDDR2 and SRAM.

```
module master (
input external_clk,
input ena,
input rst,
output [31:0] debug_hex_display,
output[7:0] clock_led,
// LPDDR2
output wire [26:0] lpddr2_address,
output wire [31:0] lpddr2_write_data,
input wire [31:0] lpddr2_read_data,
output wire lpddr2_rreq,
output wire lpddr2_wreq
);

// Clock definitions
wire clk, clk_wen, mem_clk;
wire E;  // E signal from memory stage
assign mem_clk = external_clk;
clock_div divider(mem_clk, clk_wen);

`ifdef HARDWARE
clock_control contr(
.inclk(clk_wen),        //  altclkctrl_input.inclk
.ena(ena),              //                  .ena
.outclk(clk)            // altclkctrl_output.outclk
);
`else
assign clk = ena ? clk_wen : 0;
`endif


assign clock_led[0] = clk;
assign clock_led[1] = external_clk;
assign clock_led[2] = ena;

// Control and data signals
wire [4:0] ra1, ra2;
wire [31:0] a_gpr, b_gpr;
wire gp_we;
wire [4:0] cad;
wire [31:0] alu_res, shift_res;
wire [31:0] mem_out;
wire mem_rren;
wire mem_wren;
wire [1:0] gp_mux_sel;
wire [31:0] gpr_data_in;
wire [39:0] decoder_packed;
```

```
48
49          // Program counter
50          reg [31:0] pc = 0;
51          wire [31:0] next_pc;
52
53          // Instruction register
54          wire [31:0] instruction;
55
56          always @(posedge clk or posedge rst) begin
57          if (rst)
58          pc <= 32'h0;
59          else if (E)
60          pc <= next_pc;
61          end
62
63          // Instruction register
64          reg [31:0] instruction_reg = 0;
65          always @(posedge mem_clk) begin
66          if (~E)
67          instruction_reg <= mem_out;
68          end
69
70          assign instruction = instruction_reg;
71
72          `ifdef HARDWARE
73          // Memory stage with E signal
74          memory_master memory(
75          .clk(clk),
76          .mem_clk(mem_clk),
77          .rst(rst),
78          .addr_in(~E ? pc[31:2] : alu_res[31:2]),
79          .data_in(b_gpr),
80          .mem_rren(mem_rren),
81          .mem_wren(mem_wren),
82          .gp_we(gp_we),
83          .out(mem_out),
84          .E(E),
85          // LPDDR2 Memory
86          .address(lpddr2_address),
87          .write_data(lpddr2_write_data),
88          .read_data(lpddr2_read_data),
89          .read_req(lpddr2_rreq),
90          .write_req(lpddr2_wreq)
91          );
92          `else
93          memory_master_mock memory(
94          .clk(clk),
95          .mem_clk(mem_clk),
96          .rst(rst),
97          .addr_in(~E ? pc[31:2] : alu_res[31:2]),
98          .data_in(b_gpr),
99          .mem_rren(mem_rren),
100         .mem_wren(mem_wren),
101         .gp_we(gp_we),
102         .out(mem_out),
103         .E(E)
104         );
105         `endif
106
107         // Decode stage
108         decode_master decode(
109         .a_gpr(a_gpr),
110         .b_gpr(b_gpr),
111         .pc(pc),
```

```
112         .i_fetch(instruction),
113         .next_pc(next_pc),
114         .rs(ra1),
115         .rt(ra2),
116         .decoder_packed(decoder_packed)
117         );
118
119         decoder_deconcat defcon(
120         .packed_in(decoder_packed),
121         .gp_we(gp_we),
122         .mem_wren(mem_wren),
123         .mem_rren(mem_rren),
124         .gp_mux_sel(gp_mux_sel),
125         .cad(cad)
126         );
127
128         // Execute stage
129         execute_master execute(
130         .decoder_packed(decoder_packed),
131         .a_decoder(a_gpr),
132         .b_decoder(b_gpr),
133         .i_decoder(instruction),
134         .pc(pc),
135         .alu_res(alu_res),
136         .shift_res(shift_res)
137         );
138
139         // Writeback stage
140         writeback_master writeback(
141         .memory_out(mem_out),
142         .alu_res(alu_res),
143         .shift_res(shift_res),
144         .gp_mux_sel(gp_mux_sel),
145         .gpr_data_in(gpr_data_in),
146         .link_addr(pc+32'd4)
147         );
148
149         // General purpose register file
150         gpr genreg(
151         .clk(clk),
152         .rst(rst),
153         .we(gp_we & E),  // Only write during execute phase
154         .ra1(ra1),
155         .ra2(ra2),
156         .wa(cad),
157         .rd1(a_gpr),
158         .rd2(b_gpr),
159         .wd(gpr_data_in),
160         .register_out(debug_hex_display)
161         );
162
163         endmodule
```

Code 22: CPU

Above, we see the master module in action. It successfully executes the instructions present in the mock memory module, which we use for simulation purposes. The instructions that were loaded into the mock memory module for this test are given below. The testbench is provided in the appendix (Chapter 19), where we simply instantiate the appropriate modules. However, the instruction content that the CPU executes resides in the mock memory module.

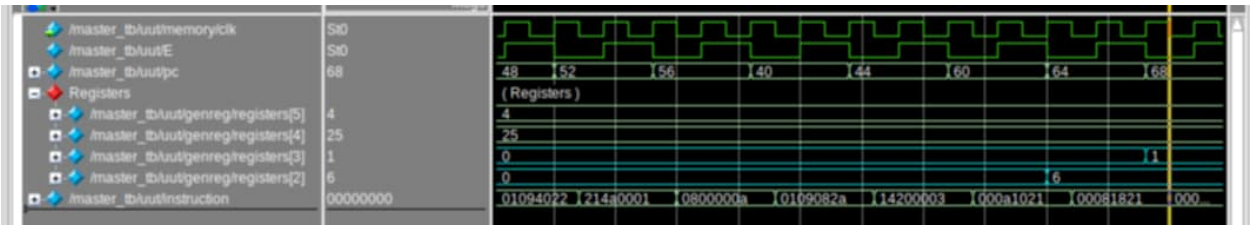


Figure 20: waveforms for master module, check chapter 19 for testbench

```
memory[0] = 32'h24040019;   // addiu $4, $0, 25
memory[1] = 32'h24050004;   // addiu $5, $0, 4
memory[2] = 32'h0c000005;   // jal 5
memory[3] = 32'h2402000a;   // addiu $2, $0, 10
memory[4] = 32'h00000000;   // syscall
memory[5] = 32'h23bdfffc;   // addiu $sp, $sp, -4
memory[6] = 32'hafbf0000;   // sw $ra, 0($sp)
memory[7] = 32'h00044021;   // addu $8, $0, $4
memory[8] = 32'h00054821;   // addu $9, $0, $5
memory[9] = 32'h240a0000;   // addiu $10, $0, 0
memory[10] = 32'h0109082a; // slt $1, $8, $9
memory[11] = 32'h14200003; // bne $1, $0, 3
memory[12] = 32'h01094022; // sub $8, $8, $9
memory[13] = 32'h214a0001; // addiu $10, $10, 1
memory[14] = 32'h0800000a; // j 10
memory[15] = 32'h000a1021; // addu $2, $0, $10
memory[16] = 32'h00081821; // addu $3, $0, $8
```

12 SRAM

12.1 Introduction

12.1.0 What is it?

SRAM (Static random-access memory) is a fast, yet small memory present on many FPGAs, including the one we're dealing with in the lab.

This document guides you through the process of creating an SRAM instance using the built-in IP, and basic details of how to use it, as well as implement it in the MIPS CPU design discussed in the Lab.

There are 2 types of SRAM present on the board - the PDF mainly focuses on the **on chip SRAM**, but the last section of the PDF deals with **off chip SRAM** as well.

12.1.0 Why use it?

SRAM offers a much bigger memory than you can have with registers, and it allows you to edit and modify memory content **from the computer** at runtime. This allows you to rapidly prototype and insert data on the device. Unlike LPDDR2 memory, it offers much greater speed as well, at the cost of size.

12.2 Setup

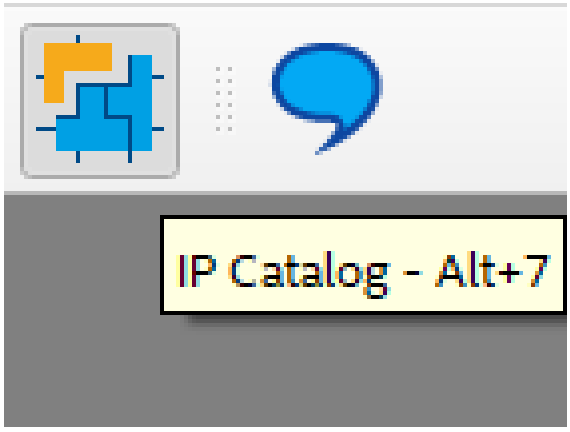
12.2.1 IP Catalog

1. Open Quartus Prime of your choice.
2. Open or create your project.
3. Open the IP Catalog:
    - Press `Alt+7`, or
    - Click the IP Catalog icon on the GUI (shown in **Figure 21b**)
4. Navigate through the catalog:

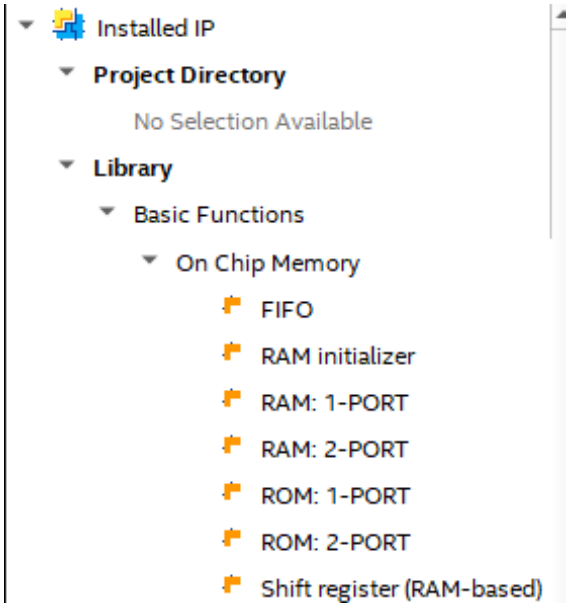
- Expand the **Basic Functions** dropdown
- Select **On Chip Memory**

5. Choose the desired type of memory for your project.

In the guide we'll be using **RAM - 1 PORT**, however other types can be implemented mostly identically.



(a) IP Catalog option in the UI, typically on top panel of Quartus.



(b) IP Catalog. Steps 4/5

Figure 21: IP Catalog and its dropdown menu

12.2.2 IP creation wizard

Now that we have chosen the type of the memory, we can proceed to filling out the information required. On the first panel, **name your instance.** By default, the input field has the location of the IP instance created. Come up with a module name and write it next to the existing text in the input field.

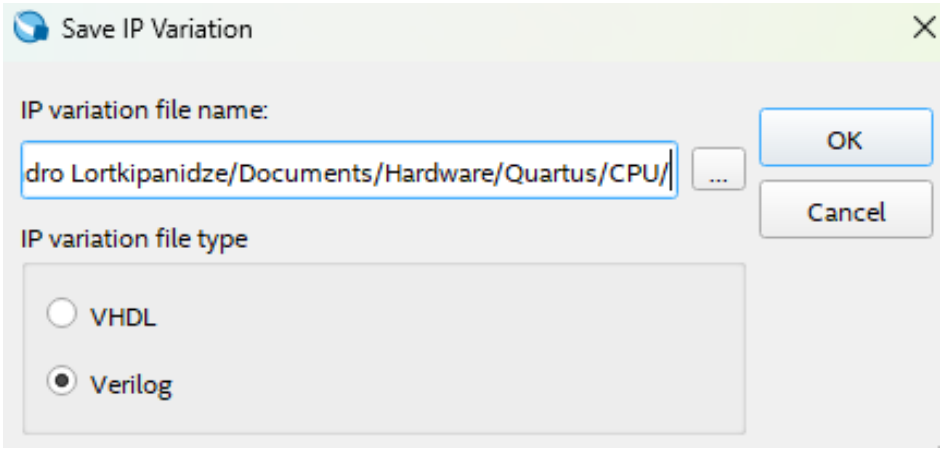


Figure 22: IP instance creation.
Name would be written next to "CPU/" in the example above.

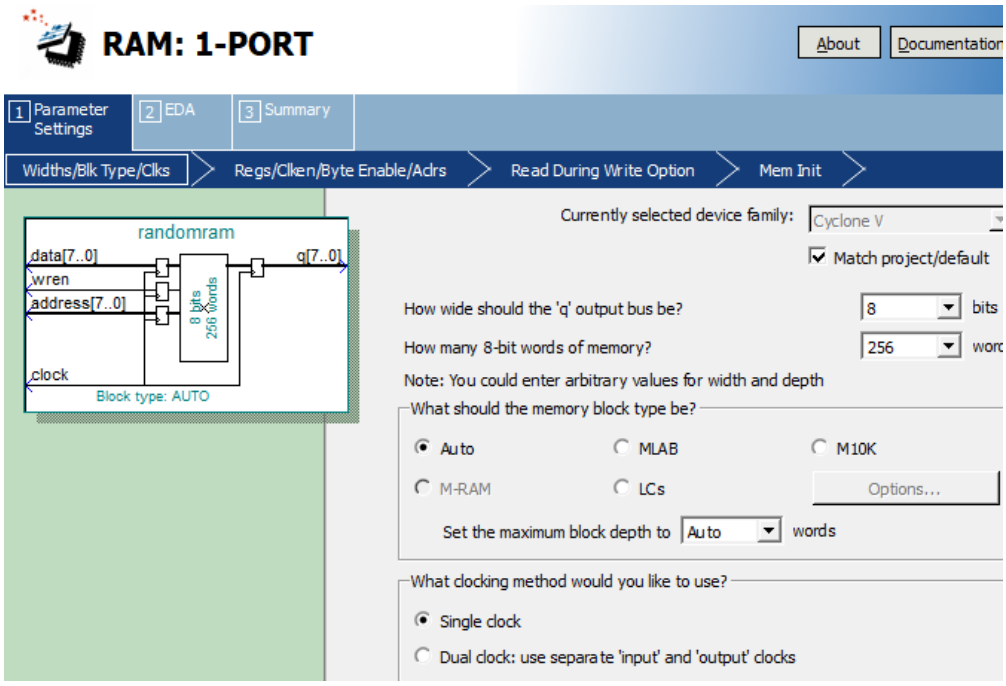


Figure 23: First page of the wizard

Here, specify the word size in the first dropdown, and the RAM size in the second. Word size is typically the same as the CPU bits, in our case, since the CPU is 32 bits in size, I'll specify 32 bits. Second field, enter how many words you need - don't get too greedy, this RAM typically can't go over a few thousand words.

Rest of the fields are typically left default, here and in other tabs as well. One thing you may want to add, however, is memory initialization. It'll allow you to load a default memory in the SRAM on FPGA startup. See more information in the **Memory Initialization** section.

Go through all the tabs and click finish. You'll have an SRAM in the design now.

12.3 Usage

Quartus handled rest of the setup of the SRAM for us.

Now we have a module we can directly use in our design.

Here's the module used in the MIPS CPU for the Lab:

```
        wire [11:0] sram_address;
        wire [31:0] sram_q;
        wire sram_wren;

        SRAM memory_sram (
        .address(sram_address),
        .clock(memory_clk),
        .data(data_in),
        .wren(sram_wren),
        .q(sram_q)
        );
```

I named my static memory module (RAM - 1 PORT) **"SRAM"**.

Inputs and Outputs:

- address - address of the word written/read from.
- clock - the clock the SRAM module uses.
- data - data input, from our end, for writing.
- wren - write enabled. If 0, reads from address, otherwise writes.
- q - data output, from the SRAM, for reading.

That's it. You have got an SRAM up and running. Use the pins as you desire.

12.4 Memory Initialization

12.4.1 Wizard configuration

In Quartus, when creating a new instance of an SRAM, you might've spotted a section in the wizard named "Mem Init". This section is dedicated for inserting initial values in the memory when the board is started. A very useful feature that has been used in our MIPS CPU to insert programs on the board the moment its powered up.

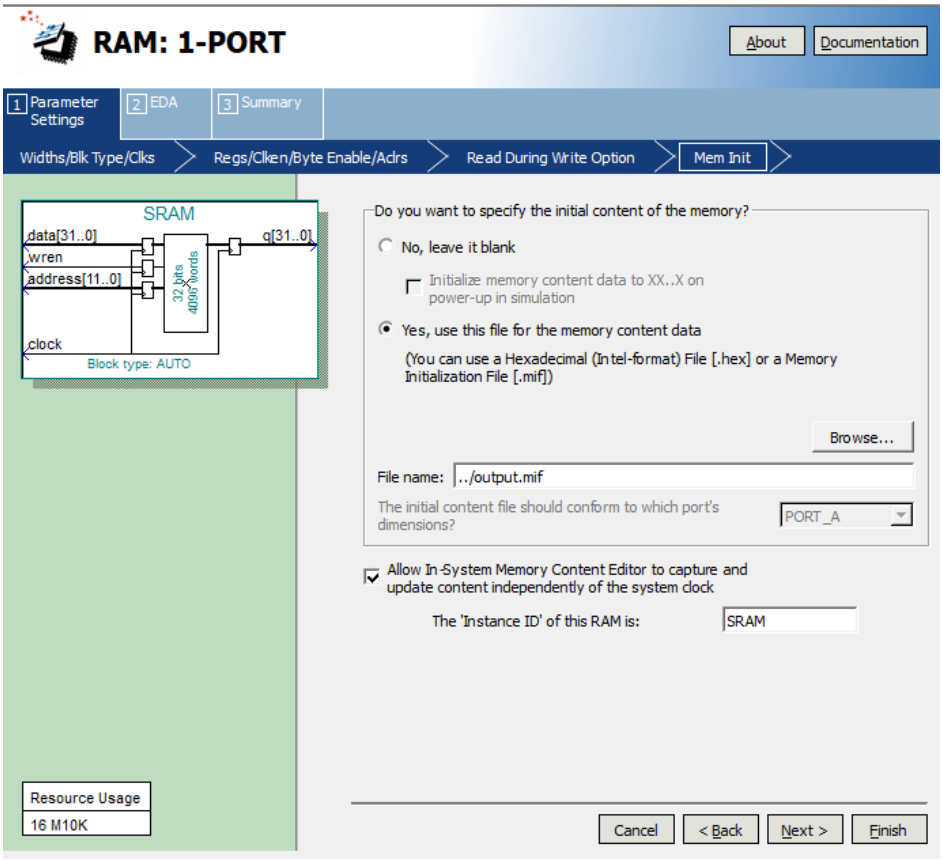


Figure 24: Memory init tab in the wizard, already configured to load initial values.

As seen in Figure 24, to initialize the memory, select "Yes, use this file for the memory content data", then click "Browse" and find your memory initialization file (see subsection 12.3 and 12.4 on how to create these files). For now create an empty .mif file and select that.
You may disregard the 'Instance ID' input field for now. See section about Memory Content Editor for more details.

12.4.2 .Mif or .Hex?

Both .mif and .hex files offer a way to "describe" the initial values in the memory. I have found .mif to be more flexible and easier to read/write than .hex files. However, if you'll be using custom software for reading/writing memory files, then .hex may offer a better alternative. Both types are explained below.

12.4.3 Hex files

This file type is from 1973, created to be loaded from a paper tape to a computer.

In .hex files, each line describes a word in memory. The following is the syntax of each of those lines:

:CCAAAATTXXXXX……XXSS

- : - Every line starts with a colon. In a typical implementation, anything can be written before the colon and it'll be ignored. Can be used for comments, though not all software supports this.
- CC - How many bytes of data there are, in hexadecimal. It'd be 20 for a 32 bit word (20 in hexadecimal means 32).
- AAAA - Address where the memory is stored, in hexadecimal. You'd write 0000 if you wanted to insert word on address 0, 0001 if you wanted on address 1, 000F if you wanted on address 15 and so on.
- XXXXX...XX - The data itself, also in hexadecimal
- SS - Checksum byte. It's calculated by summing all the bytes in the line, and picking the least significant byte. Check the Wikipedia page for Intel HEX for further details.

12.4.4 Mif files

Memory Initialization Files are longer, but easier to read and modify. A .mif file can take values and addresses in binary, decimal, unsigned decimal or hexadecimal.

Prefix of the file:

```
WIDTH=32;
DEPTH=4096;

ADDRESS_RADIX=HEX;
DATA_RADIX=HEX;
```

- **WIDTH** describes how many bits per word.
- **DEPTH** describes how many words are there.
- **ADDRESS_RADIX** describes what format we'll be using for addresses. I'm using hexadecimal, but you can write any other format.
- **DATA_RADIX** describes what format we'll be using for the data.

The Radix that can be specified are **HEX**, **DEC** (decimal), **BIN** (binary) and **UNS** (unsigned decimal)

Data initialization:

```
CONTENT BEGIN
ADDR : DATA;
ADDR : DATA;
...
[ADDR_START..ADDR_END] : DATA;
END;
```

- ADDR - data addess, specified in **ADDRESS_RADIX** format.
  (e.g. 10 for address 10 in decimal, or 0A in hexadecimal)
- DATA - data address, specified in **DATA_RADIX** format.
- ADDR_START...ADDR_END - .mif file format allows you to initialize spans of data with a value.

Example file:

```
WIDTH=32;
DEPTH=4096;

ADDRESS_RADIX=HEX;
DATA_RADIX=HEX;

CONTENT BEGIN
000 : 8C01000A;
001 : 8C02000B;
002 : 20030000;
003 : 10610004;
004 : 00442021;
005 : 20630001;
006 : 1000FFFD;
007 : 00042821;
008 : AC050020;
009 : 00000000;
00A : 00000003;
00B : 0000000B;
[00C..FFF] : 00000000;
END;
```

The span of memory from **00C** to **FFF** is initialized to be 0.

Spans can be specified in the middle as well. For example, you could initialize span **009-00C** to be 0, and then configure **00D** to be some other value.

12.5 Memory Content Editor

Memory Content Editor allows you to read and edit data on the board at runtime.

In the RAM creation wizard, in the last section (See Figure 24), fill the 'Instance ID' field with an unique identifier for your memory, this can be anything short. You can have multiple SRAM instances in one project, hence this ID is used to differentiate between them.

Once you do that, program the FPGA, and open **Tools** → **In-System Content Editor**

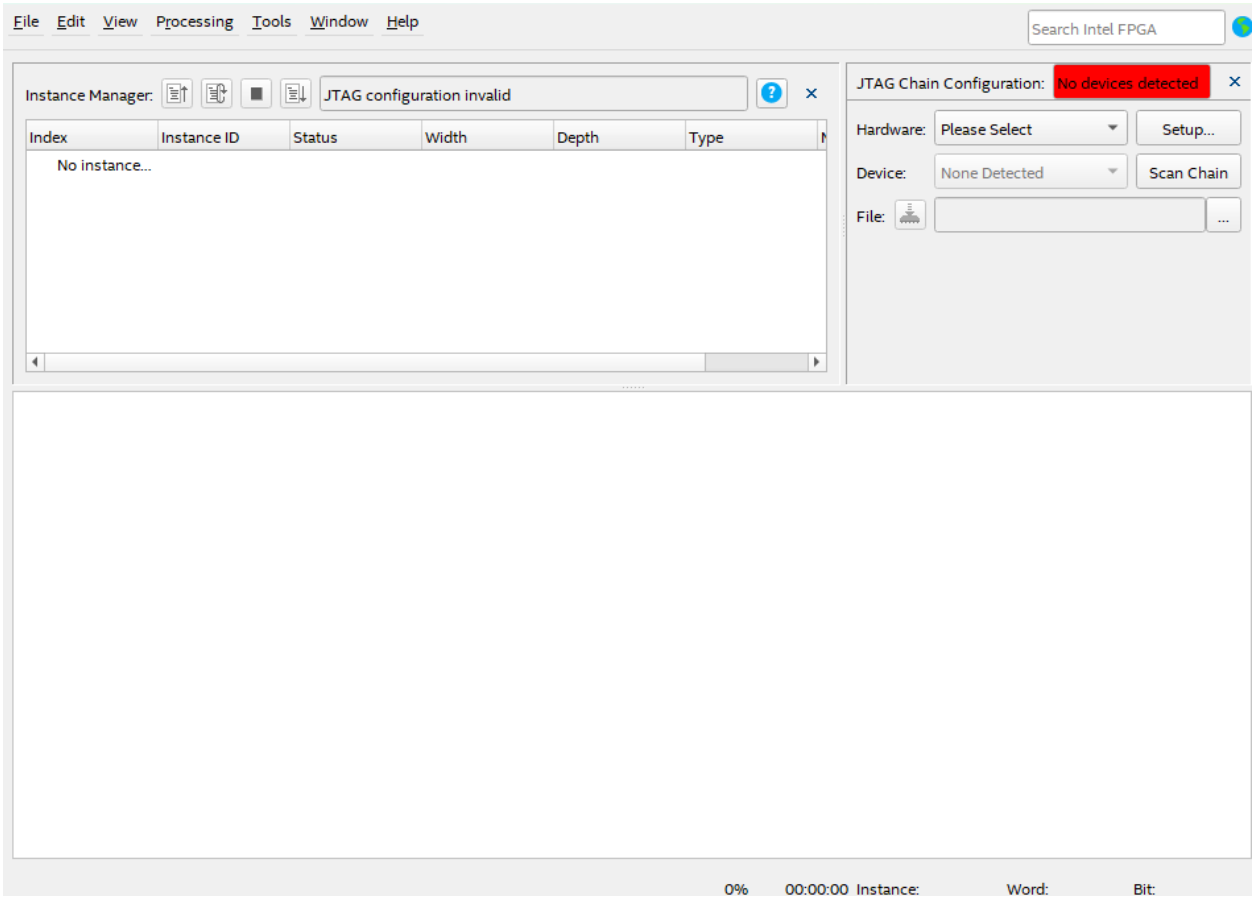


Figure 25: Freshly opened Memory Content Editor

Select your hardware in the dropdown (USB-Blaster), and if device doesn't appear by itself, click "Scan Chain"

Once that's done, an instance with the name you specified will appear on the left. Select it.

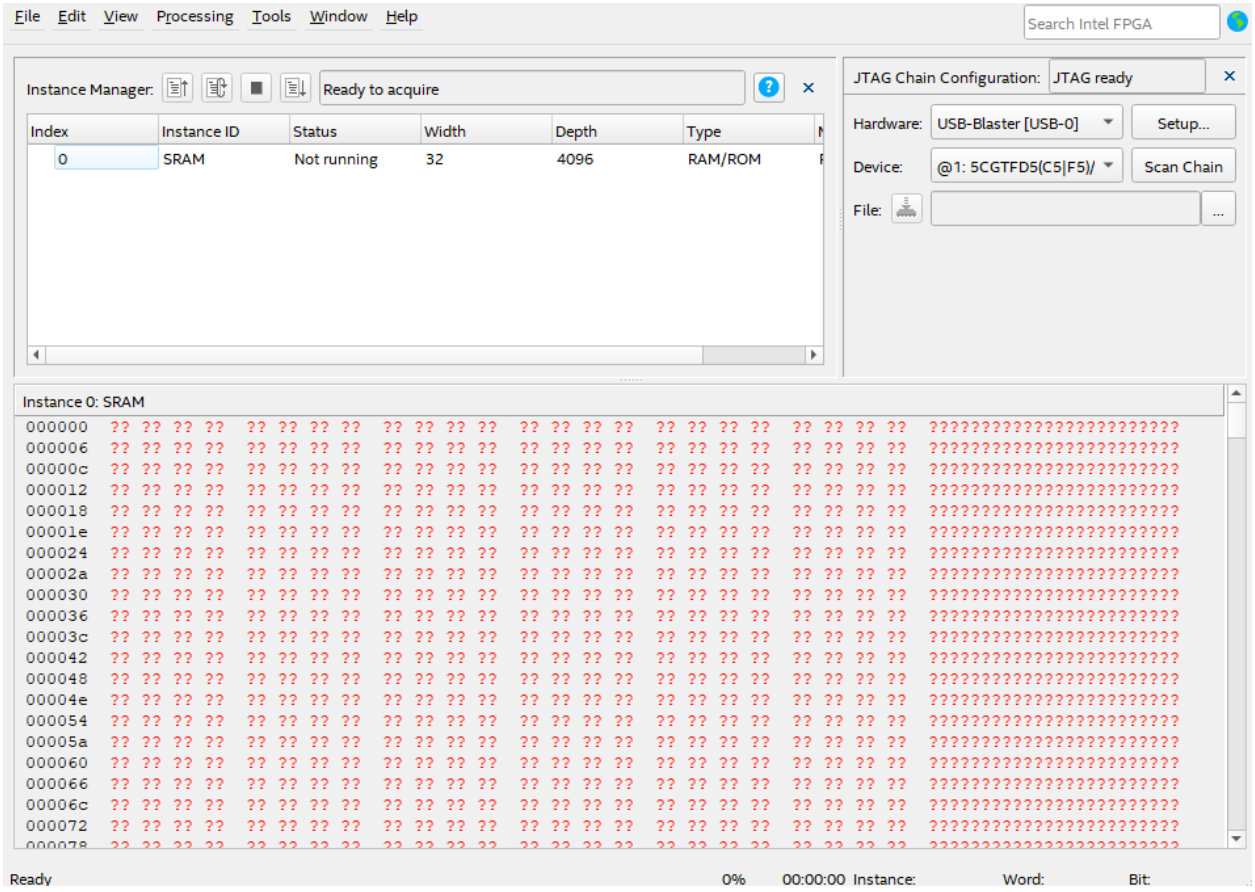


Figure 26: Memory Content Editor, connected but with no data loaded

You'll see a few buttons on the top left.

- Use  to load data from the existing memory.
- Use the  buttons to continously load and stop continous load. This'll refresh the data displayed constantly, but it'll stop you from writing new data.
- Use  to save the content you edited in the editor to the board.

After loading the memory with the first button, you can edit data in the display. Save the modified data with the last button.

You can also directly import .mif/.hex files and put them in memory. To do this, select your instance, **Edit→Import Data from file.**

Make sure the instance is selected for all the things above, otherwise buttons will be grayed out.

12.6 MIPS CPU specific

The MIPS CPU in our lab worked great with the basic register based memory, but it failed when moving to SRAM. The diagnosis of the issue revealed that, unlike registers, Quartus doesn't take in account the delay of the SRAM. This causes old value to be read from the memory a few nanoseconds before the actual value is loaded (e.g. pc is incremented and loads next instruction, execute cycle. Address is changed, and the CPU expects the instruction to change too - but it fails to due to the delay, in time, hence old instruction is executed). The delay is 7-20ns.

To circumvent the issue, the CPU was clocked at half the speed of the memory (Memory is 125MHz, using **CLOCK_125_P**, while CPU was clocked at 67.5MHz). This allowed the SRAM to fetch data for the next cycle ahead of time. Potentially, a delay can be introduced to clock the Memory and CPU at the same speed, however this is no easy task with our current level of expertise.

```
1         reg [2:0] clk_div;
2         wire slow_clk;
3
4         always @(posedge CLOCK_125_p) clk_div <= clk_div + 1'b1;
5
6         assign slow_clk = clk_div[1]; // 67.5MHz clock (125MHz / 2)
7
8         CPU cpu(
9         .clk(slow_clk),
10        .mem_clk(CLOCK_125_p),
11        // Other assignments here...
12        );
```

The 2 bit **clk_div** register is incremented every 125MHz clock cycle, and whenever **clk_div[1]** becomes 1 (Which happens twice as slow as the **clk_div[0]** becoming 1), it causes the **slow_clk** to be 1 and et cetera. This causes the **slow_clk** to be clocked two times slower than the original.

Here's how we implemented the SRAM in our memory module:

```
module memory_full (
input wire clk,
input wire memory_clk,
input wire rst,
input wire S,
input wire [31:0] next_pc,
input wire [29:0] addr_in,
input wire [31:0] data_in,
output reg [31:0] I,
output reg [31:0] PC,
output wire [31:0] data_out,
output reg E
);

wire [11:0] sram_address;
wire [31:0] sram_q;
wire sram_wren;

assign sram_address = ~E ? addr_in[11:0] : PC[13:2];
assign sram_wren = S && ~E;
assign data_out = sram_q;

SRAM memory_sram (
.address(sram_address),
.clock(memory_clk),
.data(data_in),
.wren(sram_wren),
.q(sram_q)
);

always @(posedge clk or posedge rst) begin
// Reset/I-register/PC logic here...
end
```

13 Off-Chip SRAM

Warning: We don't use this module (and the chapter itself) in the hardware, as the SRAM on board is 16 bits, and provided an awkward interface to our memory with little to no advantage. Nevertheless, we included the documentation as it acts as a logical bridge in difficulty between the simple on-chip SRAM and off-chip LPDDR2.

13.0.0 What is it?

It is a slower, yet more spacious ($\sim$ 2MB) SRAM present on the board. You can imagine it as something in-between On-chip SRAM and LPDDR2 memory.

It inherits the flaws of LPDDR2 memory - memory operations take more than a single clock cycle, hence you have to explicitly wait for operations to complete. Still, much faster than LPDDR2 memory.

13.1 Setup

Catalog

1. Open the IP Catalog:
    - Press `Alt+7`, or
    - Click the IP Catalog icon on the GUI (**Figure 21b**)
2. Navigate through the catalog:
    - Expand the **University Program** dropdown
    - Select **Memory**
3. Choose the memory type you need.

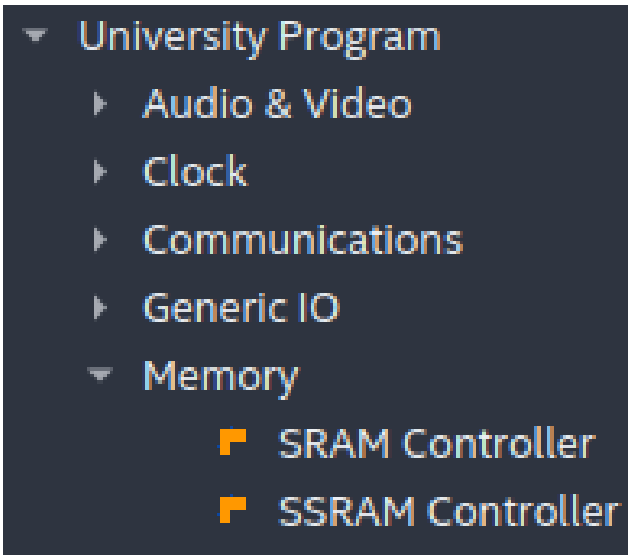


Figure 27: The IP Catalog entries for off-board SRAM

Wizard

1. Double click the SRAM module to open up the wizard.

2. Name your module:

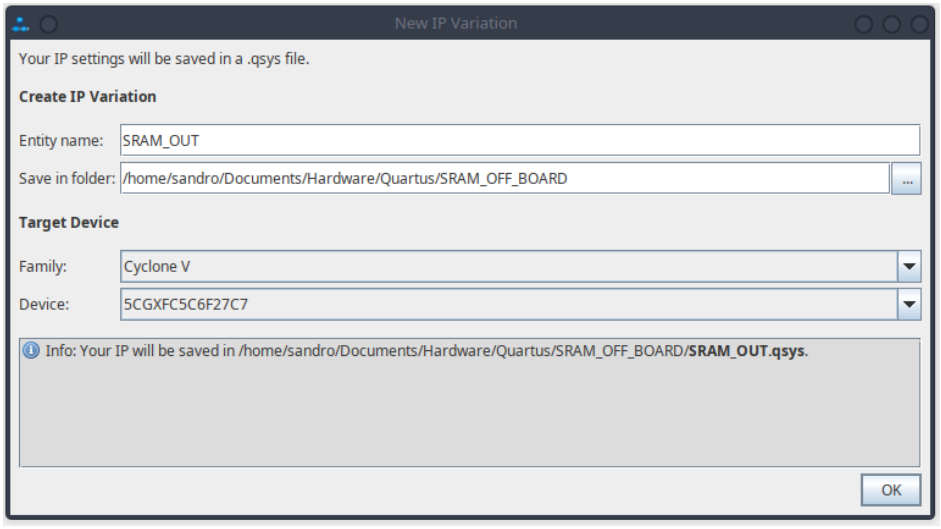


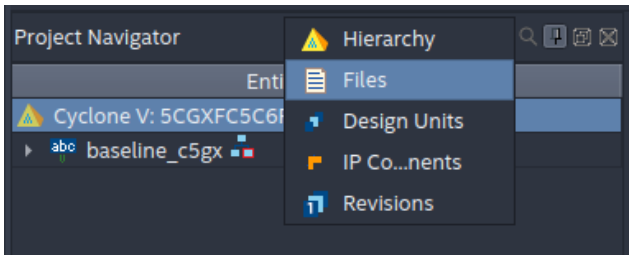
Figure 28: Name the module

3. Click "OK" and "Generate HDL"

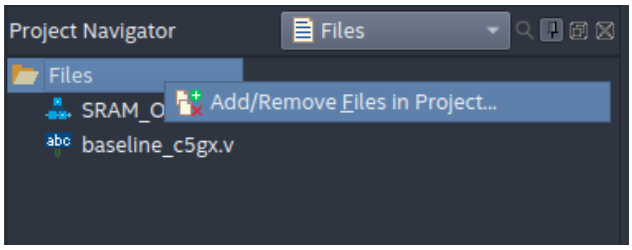
Wait for both generation operations to complete and hit finish.

We're not done though - we have to include the newly generated verilog files in our project.

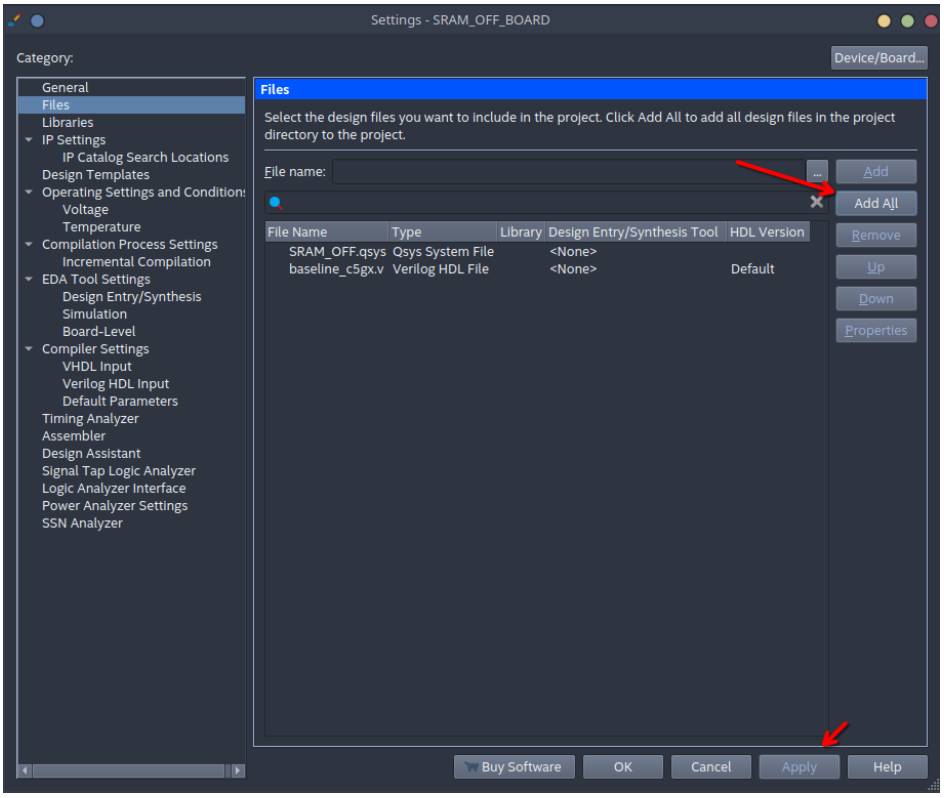
1. Open the Hierarchy dropdown, select Files.



2. Right-Click the "Files" at the top of the hierarchy and select "Add/Remove Files"



3. Hit "Add all" (or manually select files) and "Apply".



That’s all. You’re now finally ready to use the SRAM.

13.1.1 Module Specification and Example

In Quartus, by default, external wires (busses) in the project template provided are pre-mapped to the baseline_c5gx.v (different name for other boards from cyclone v gx) file. By default, this file is provided with a module which has all inputs and outputs on the board - for example, it contains LEDG wires, which drive green LEDs on the board.

Similar to that, we have SRAM pins provided as inputs/outputs to the baseline. We can simply use those to make the SRAM IP work, by adding the code below to the baseline_c5gx.v.

```
    SRAM_OUT sram0 (
    .clk            (CLOCK_50_B5B),          // clk.clk
    .reset          (SW[0]),                 // reset.reset
    .SRAM_DQ        (SRAM_D),                // external_interface.DQ
    .SRAM_ADDR      (SRAM_A),                // .ADDR
    .SRAM_LB_N      (SRAM_LB_n),             // .LB_N
    .SRAM_UB_N      (SRAM_UB_n),             // .UB_N
    .SRAM_CE_N      (SRAM_CE_n),             // .CE_N
    .SRAM_OE_N      (SRAM_OE_n),             // .OE_N
    .SRAM_WE_N      (SRAM_WE_n),             // .WE_N
    .address        (address),               // avalon_sram_slave.address
    .byteenable     (byteenable),            // .byteenable
    .read           (read),                  // .read
    .write          (write),                 // .write
    .writedata      (writedata),             // .writedata
    .readdata       (readdata),              // .readdata
    .readdatavalid (readdatavalid)           // .readdatavalid
    );
```

Notable wires:

SRAM_... - These are pre-mapped wires which are provided in the template. You can directly pass them as arguments.

byteenable - for 16 bit SRAM, its 2 bit flag that tells the board whether or not to write in the first/second byte. Useful, for example, when doing Load Upper Immediate instruction, the flag would be 10, if you wanted to save the whole word, however - it'd be 11.

13.1.1 Wire Specification Table

name	size	type
address	20	input
byteenable	2	input
read	1	input
write	1	input
writedata	16	input
readdata	16	output
readdatavalid	1	output
clk	1	input

13.2 What if we want to make it work for 32 bits?

(Note: Usually, on-board SRAM seems like a better option to pick, especially if you're doing 32 bits).

While the default SRAM implementation works with 16-bit words, you might need to work with 32-bit data in your design. The way Quartus wants us to do this is overcomplicated, hence I've written a custom module myself that handles it.

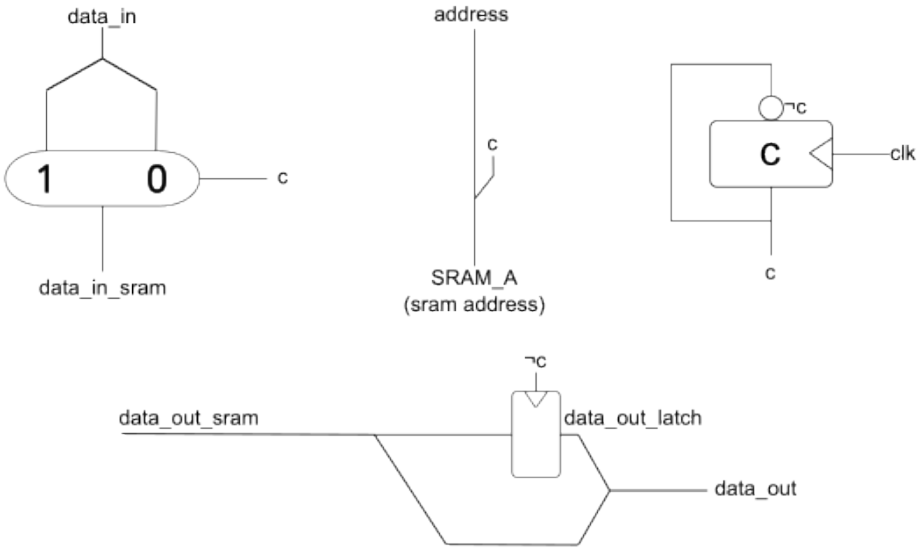
13.2.1 Overview

The `sram_32` module provides a 32-bit interface to the 16-bit SRAM. The way it works is by splitting 32 bits of data into two 16 bit data blocks. Writes happen in 2 separate cycles, 1 cycle per 16 bits. Reading is done in the same way.

- Each read operation takes 2 cycles, with 2 16 bit operations - total of 4.
- Each write operation takes 1 cycle, with 2 16 bit operations - total of 2.

13.3 How we do it

We add a single state bit **c**, which acts as a state indicator - whether we're in higher or lower bits. It cycles between **HIGH** and **LOW** (1 and 0) each clock cycle, and this is done by a simple register feeding inverse back into itself.



As for the data input, the input is 32 bits of data. We simply split it into two 16 bit parts, and feed the SRAM each of those parts per cycle. Note that this requires input signal to be stable.

Address to SRAM is address input, concatenated with the **c** bit. This way, when c is low, lower data is written at address **...0**, and then when c is high, high data is written at address **...1**.

And finally, data read is done by latching on the lower half of the bits, and feeding high end directly. This way, when the hardware is ready (at c = 1), output is stable.

13.4 Module source code

```
module sram_controller(
    input  clk, rst,
    input  [16:0] address,
    input  read, write,
    output [31:0] readdata,
    input  [31:0] writedata,
    output ready,
    input  SRAM_CE_n, SRAM_LB_n, SRAM_UB_n, SRAM_WE_n,
    input  [17:0] SRAM_A,
    input  [15:0] SRAM_D
);
reg  c;
reg  [15:0] data_out_latch;
reg  [16:0] addr_latch;

wire [15:0] readdata_sram, writedata_sram;
wire [17:0] address_sram;
```

```
18 wire readdatavalid;
19
20 always @(posedge clk)
21     c <= ~c;
22
23 always @(negedge c) begin
24     data_out_latch <= readdata_sram;
25     addr_latch <= address;
26 end
27
28 assign readdata = {readdata_sram, data_out_latch};
29 assign writedata = {address, c};
30 assign ready = addr_latch == address && c;
31
32 sram_u u0 (
33     .clk        (clk),
34     .reset      (rst),
35     .SRAM_DQ    (SRAM_D),
36     .SRAM_ADDR  (SRAM_A),
37     .SRAM_LB_N  (SRAM_LB_n),
38     .SRAM_UB_N  (SRAM_UB_n),
39     .SRAM_CE_N  (SRAM_CE_n),
40     .SRAM_OE_N  (SRAM_OE_n),
41     .SRAM_WE_N  (SRAM_WE_n),
42     .address    (address_sram),
43     .byteenable(2'b11),
44     .read       (read),
45     .write      (write),
46     .writedata  (readdata_sram),
47     .readdata   (writedata_sram),
48     .readdatavalid (readdatavalid)
49 );
50
51 endmodule
```

If you look closely at the source code, you'll see a few interesting details - namely, there's an additional latch added to the address. In general this latch is used to make sure that address used on c=0 matches the address used on c=1 for data fetch, in the edge case the data read started when c=1. Might not be needed in our CPU design, but in case it is used elsewhere, this would help.

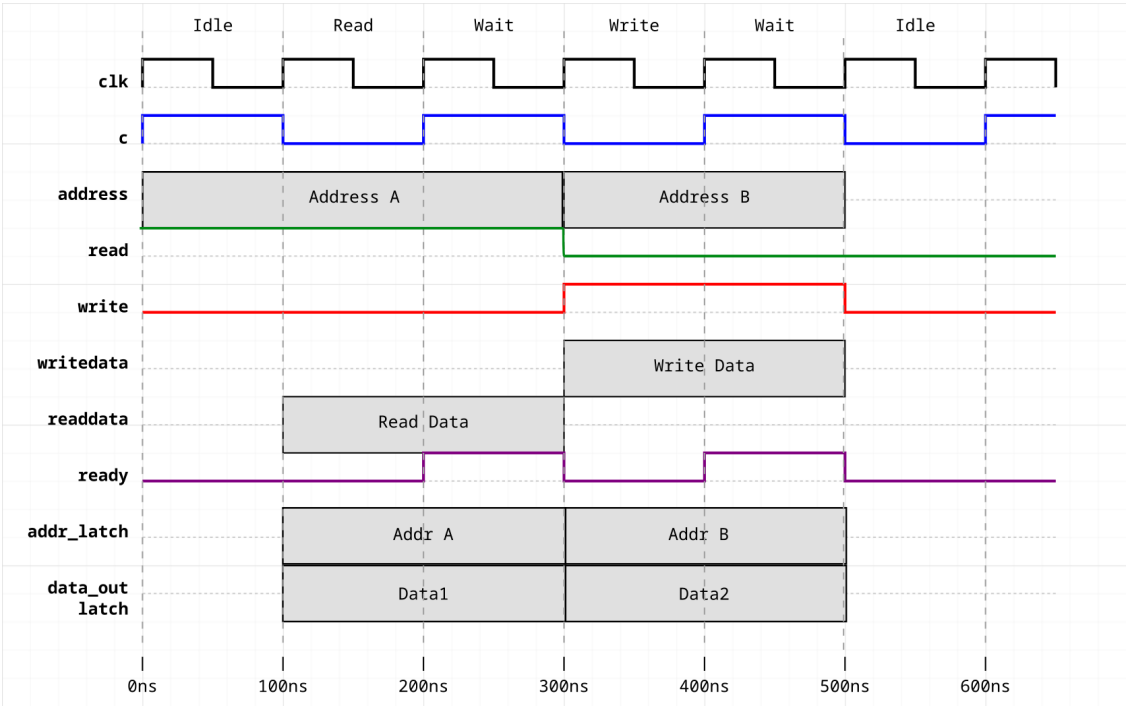


Figure 29: Timing diagram showing typical usage and how internal wires behave.

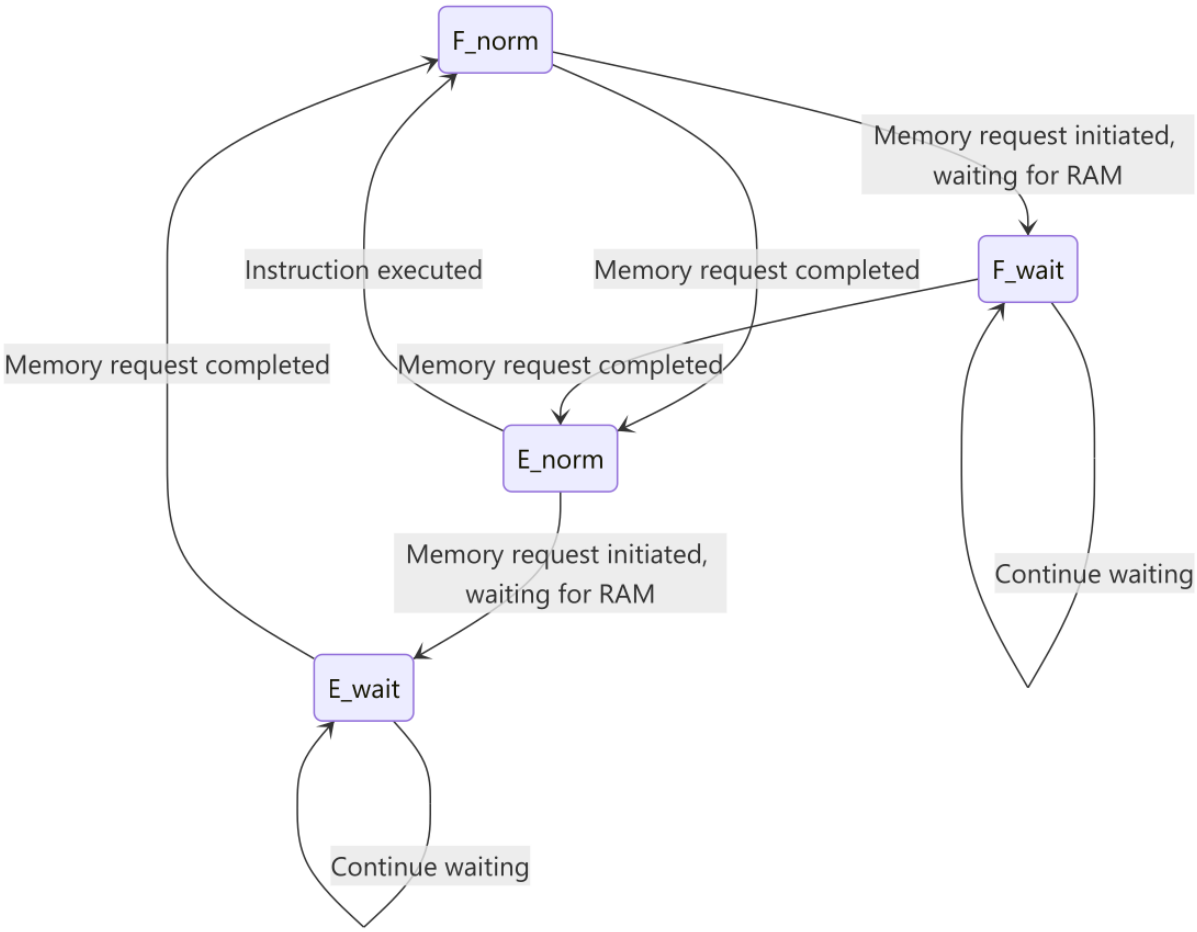
14 LPDDR2 RAM

The design up to now has used 8kB of SRAM as its memory. While very fast, it doesn't provide the memory we'd typically want from a computer, or other frequent applications of an FPGA. For this purpose, Altera has provided LPDDR2 Memory on its Cyclone V GX boards. This type of RAM sacrifices a bit of performance for lower power consumption compared to the DDR RAM, as indicated by the name (Low-Power Double Data Rate). Despite its impressive capabilities, we immediately face an issue - it's slow. A lot slower than typical SRAM. Hence, we can't expect a memory request to complete until the next cycle (When E=0, we have no guarantee that instruction requested from memory will be available at E=1).

To address this issue, we create a state machine for the Execute bit, which holds the E register until the memory request completes before allowing it to transition to 1.

Instead of directly changing from 0 to 1 and vice versa, we define 4 states:

- E_norm - normal execution
- F_norm - normal fetch
- E_wait - we are waiting for the RAM in Execute.
- F_wait - we are waiting for the RAM in Fetch.



From	To	Condition
F_norm	E_norm	Memory request completed
E_norm	F_norm	Instruction executed
F_norm	F_wait	Memory request initiated, waiting for RAM
F_wait	E_norm	Memory request completed
E_norm	E_wait	Memory request initiated, waiting for RAM
E_wait	F_norm	Memory request completed
F_wait	F_wait	Continue waiting
E_wait	E_wait	Continue waiting

Figure 30: State machine of the E bit

This construction allows us to wait for the RAM without disturbing the CPU Execution.

```
// Note - ena is low for memory operation. we execute normally at ena
    =1
```

```
reg E = 0;
reg waiting = 0;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        E <= 0;
        waiting <= 0;
    end else begin
        if (waiting) begin
            if (ena) begin
                waiting <= 0;
                E <= ~E;
            end
        end else begin
            if (!ena) begin
                waiting <= 1;
            end else begin
                E <= ~E;
            end
        end
    end
end
```

The `waiting` register tracks whether the system is in a normal or waiting state.

When in normal operation, the system toggles between E and F states on each clock cycle.

When a memory request occurs (`ena` goes low), the system enters a waiting state and pauses the state transition.

The system remains in its waiting state until the memory request completes.

Once memory access completes, the system resumes normal operation by transitioning to the next state.

14.1 Logic behind the IP

In this project we use the IP abstraction to simplify our work. For a detailed, bit-by-bit description of how the abstraction works, see the Intel External Memory Interface Handbook [2].

14.2 Logic behind our Implementation

Before we jump head into the details, let's look at the general layout of the project, to get the idea of what is happening.

Keep in mind - the LPDDR2 RAM is a physical memory on the board. It has wires connected to the FPGA which we have to use to communicate - much like off-chip SRAM.

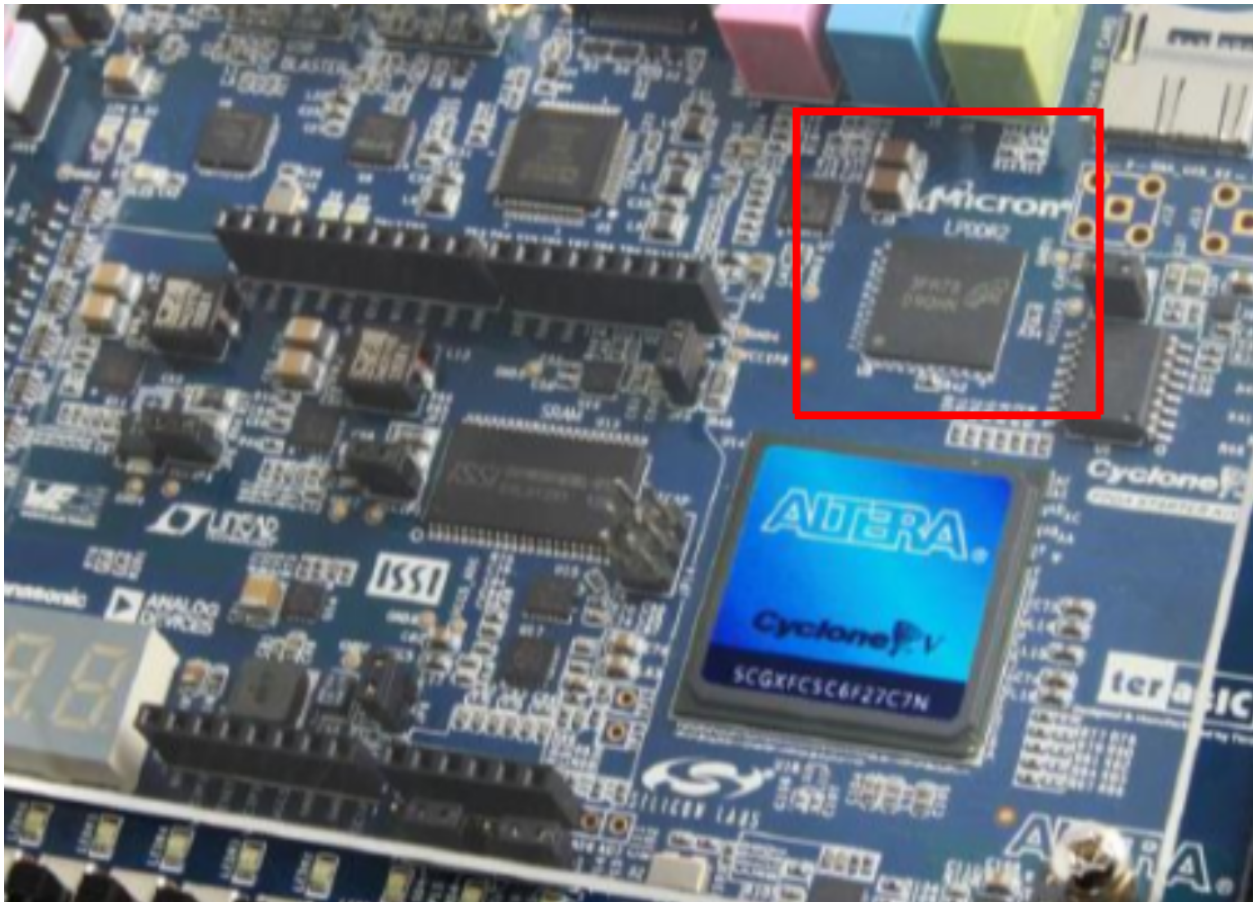


Figure 31: LPDDR2 RAM on the chip

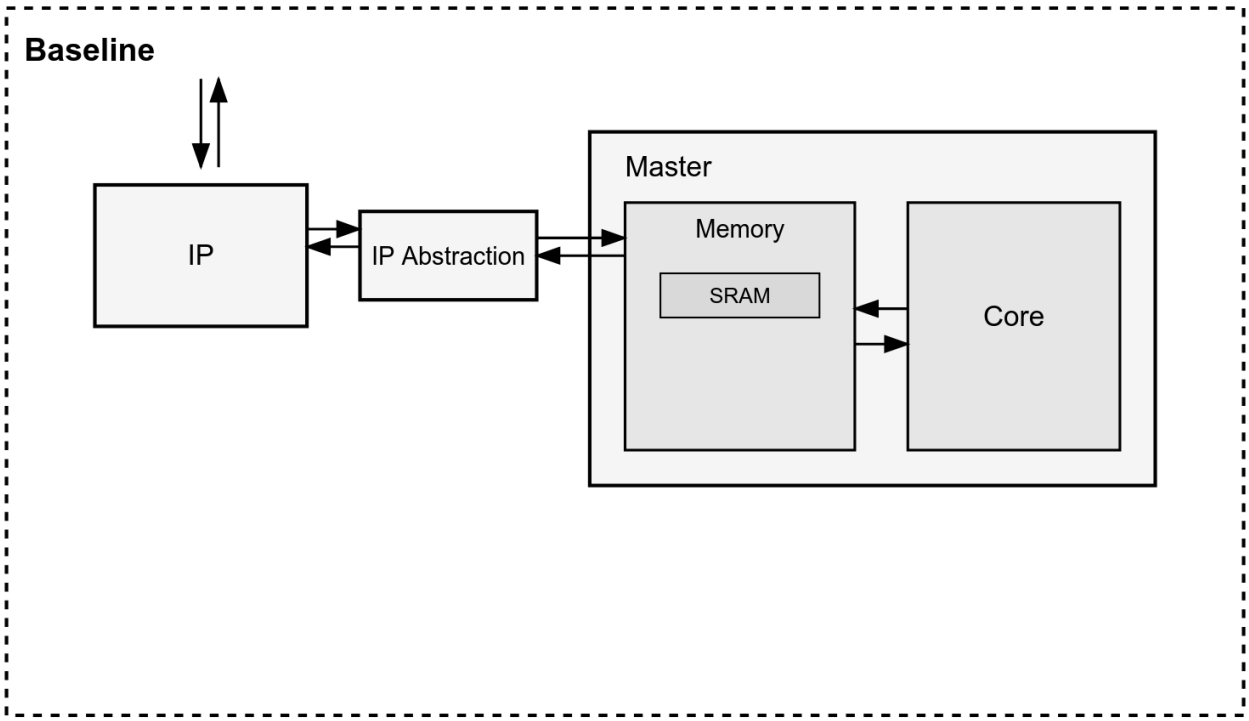


Figure 32: General layout of the hardware, how we implement the memory

There are several layers of abstraction going on here.

1. Abstract by IP Provided by intel. Hence, we don't have to manually implement the protocols required (check "Logic behind the IP" for more details). Hardware wires from LPDDR2 chip are provided directly from baseline.

2. We further abstract the interface using our own **IP Abstraction** module, which implements a state machine handling memory interaction. This way, Memory will only have to manage 5 wires instead of a few dozen.

3. The abstracted connection is fed to the Memory module.

4. Memory module implements a switch which changes between the SRAM and the LPDDR2 RAM.

Let's start from the LPDDR2 hardware side, and work our way up to the CPU core.

14.3 The IP provided

The IP that we have has plenty of wires coming in and out. Most of these can be grounded or directly fed from baseline (e.g. we don't need a byte mask or all of the reset signals).

```
assign test_software_reset_n = KEY[0];

lpddr2 fpga_lpddr2_inst(
    .pll_ref_clk(CLOCK_50_B5B),
    .global_reset_n(1'b1), // Tie high (inactive)
    .soft_reset_n(test_software_reset_n),
    .afi_clk(afi_clk),
    .afi_half_clk(afi_half_clk),
    .mem_ca(DDR2LP_CA),
    .mem_ck(DDR2LP_CK_p),
    .mem_ck_n(DDR2LP_CK_n),
    .mem_cke(DDR2LP_CKE[0]),
    .mem_cs_n(DDR2LP_CS_n[0]),
    .mem_dm(DDR2LP_DM),
    .mem_dq(DDR2LP_DQ),
    .mem_dqs(DDR2LP_DQS_p),
    .mem_dqs_n(DDR2LP_DQS_n),
    .avl_ready_0(fpga_lpddr2_avl_ready),
    .avl_burstbegin_0(fpga_lpddr2_avl_burstbegin),
    .avl_addr_0(fpga_lpddr2_avl_addr),
    .avl_rdata_valid_0(fpga_lpddr2_avl_rdata_valid),
    .avl_rdata_0(fpga_lpddr2_avl_rdata),
    .avl_wdata_0(fpga_lpddr2_avl_wdata),
    .avl_be_0(4'hF), // byte enable
    .avl_read_req_0(fpga_lpddr2_avl_read_req),
    .avl_write_req_0(fpga_lpddr2_avl_write_req),
    .avl_size_0(fpga_lpddr2_avl_size),
    .mp_cmd_clk_0_clk(afi_half_clk),
    .mp_cmd_reset_n_0_reset_n(test_software_reset_n),
    .mp_rfifo_clk_0_clk(afi_half_clk),
    .mp_rfifo_reset_n_0_reset_n(test_software_reset_n),
    .mp_wfifo_clk_0_clk(afi_half_clk),
    .mp_wfifo_reset_n_0_reset_n(test_software_reset_n),
    .local_init_done(fpga_lpddr2_local_init_done),
```

```
35      .oct_rzqin(DDR2LP_OCT_RZQ)
36  );
```

Most wires are already provided as baseline inputs, we don't have to worry about them and pass them directly (e.g. DDR2LP_* signals).

The signals we have to define ourselves are the following:

```
1  .local_init_done(fpga_lpddr2_local_init_done),   // LPDDR2 RAM is
     ready
2  .avl_waitrequest_n(fpga_lpddr2_avl_ready),       // LPDDR2 is done
3  .avl_address(fpga_lpddr2_avl_addr),              // Address input
4  .avl_readdatavalid(fpga_lpddr2_avl_rdata_valid),// Data on the bus is
      valid
5  .avl_readdata(fpga_lpddr2_avl_rdata),            // Read Data (32 bits
     )
6  .avl_writedata(fpga_lpddr2_avl_wdata),           // Write Data (32
     bits)
7  .avl_read(fpga_lpddr2_avl_read_req),             // Read Request
8  .avl_write(fpga_lpddr2_avl_write_req),           // Write request
9  .avl_burstbegin(fpga_lpddr2_avl_burstbegin),     // Burst begin
```

The above 9 wires are the ones we further simplify using a state machine we wrote.

This is the Avalon Memory interface. It is (roughly) defined by the following timing diagram:

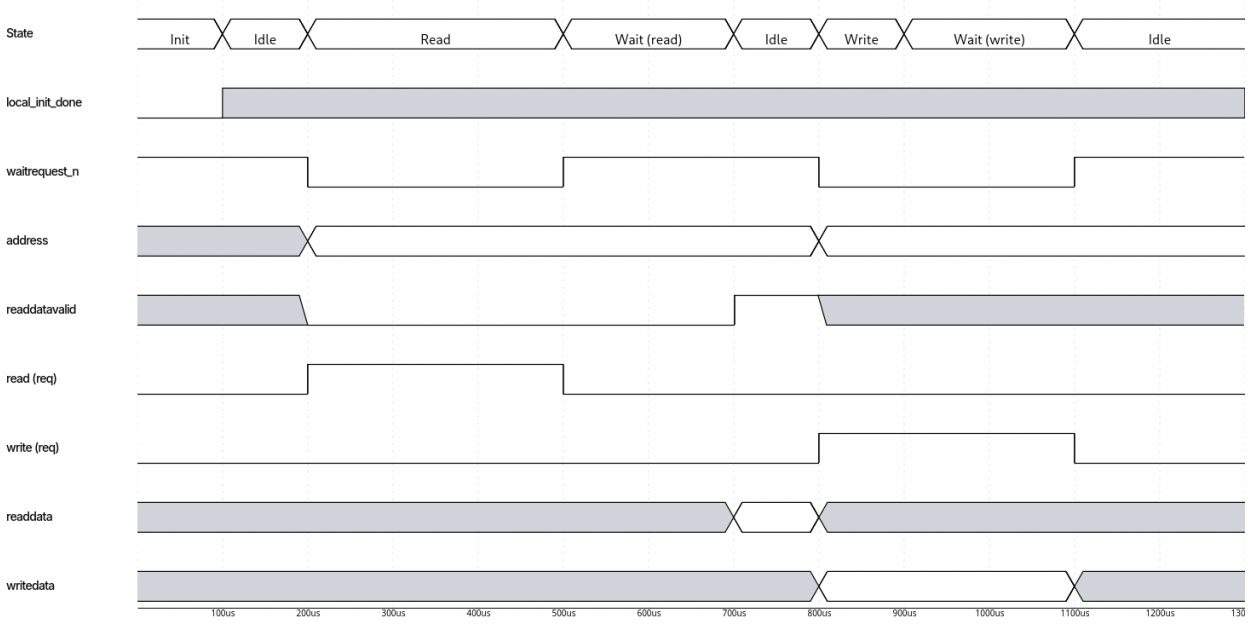


Figure 33: Avalon Memory interface. Times shown are not accurate, they're just for visual interpretation.

Let's start with write, the second half of the diagram, as it is easier. We set the `write_req` flag, telling the memory we want to write data. We also set the address and the data we wish to write.

After a while, the hardware sets `waitrequest_n` to 1, indicating that it is done with the memory operation.

Note that write can be simplified to 1 state, as it only has to wait for 1 signal, however we split it into two - 1 for setting signals, another for waiting.

Reading is a bit more complex, and requires an additional state compared to write. This is due to it waiting for 2 signals - `waitrequest_n` and `readdatavalid`.

We set the `read_req` flag, telling the memory we want to read data. We also set the address.

`waitrequest_n` is soon after set to 1. We proceed to next state, and wait for `readdatavalid`.

After that is done too, we have valid data on the bus.

The reason as to why we wait for `readdatavalid` is because even after memory is done reading data, we need to retrieve it. Unlike write, which requires no response from the memory other than `waitrequest_n`, read requires a data response. This data retrieval itself, as well as putting it on the bus takes time, which we have to wait for.

14.4 Our abstraction

We've created an abstraction for the states defined above, to make it easier for memory to read and write data. To achieve this, we've created a state machine.

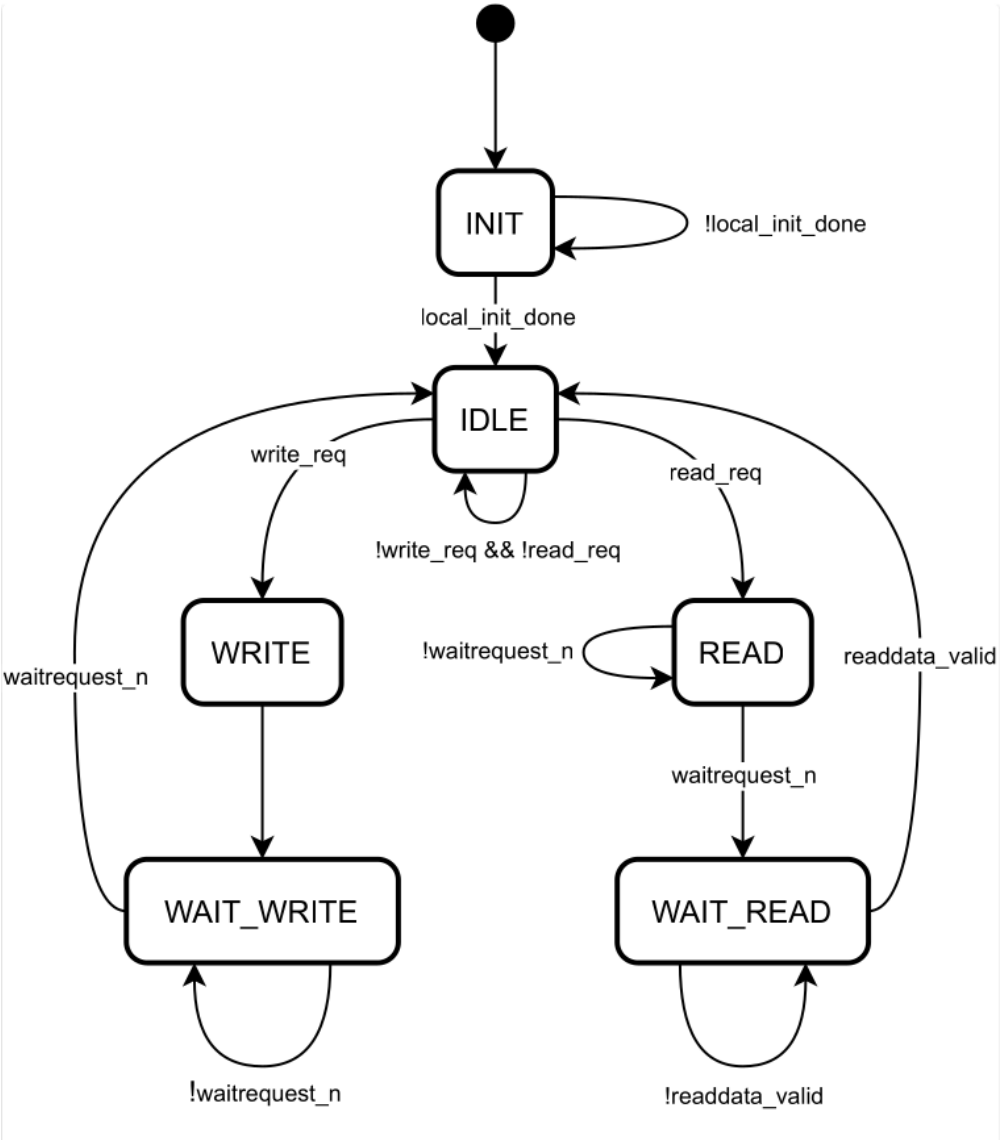


Figure 34: The State machine

The write request and read request come from the core. We send a different pair of requests to the memory which we control within the state machine. Namely, `avl_write` and `avl_read`.

- Init - we start here. We wait for initialization to be done, until then, we must hold the processor from executing. After `local_init_done` is asserted, we proceed to the `Idle` state.

```
1   INIT: begin
2     ...
3     if (local_init_done)
4       c_state <= IDLE;
5   end
```

- Idle - sort of a "crossroad" for the rest of the states. This is where read/write states are dispatched from. Until any of those are requested, we loop in the Idle state.

```
IDLE: begin
  if (read_req)
    c_state <= READ;
  else if (write_req)
    c_state <= WRITE;
end
```

- Write - When there is a write request, we set relevant data for the memory (`data`, `address`, and `avl_write`) and move on to the next state.

```
WRITE: begin
  avl_writedata <= inData;
  avl_address <= addr;
  avl_write <= 1'b1;
  c_state <= WAIT_WRITE;
end
```

- Wait Write - this state loops into itself, until `waitrequest_n` is satisfied, after which `avl_write` is deasserted, and we return to Idle state.

```
WAIT_WRITE: begin
  if (avl_waitrequest_n) begin
    c_state <= IDLE;
    avl_write <= 1'b0;
  end
end
```

- Read - When there is a read request, we set relevant data for the memory (`address`, `avl_read`). The state loops until `avl_waitrequest_n` is asserted by the memory hardware. We deassert write request as well.

```
READ: begin
  avl_address <= addr;
  avl_read <= 1'b1;
  if (avl_waitrequest_n)
```

```
        c_state <= WAIT_READ;
    end
```

- Wait Read - After the `avl_waitrequest_n` is asserted, we can now wait for the retrieval on the bus. We wait, and after `avl_readdatavalid` is asserted, we latch onto the data and go back to the Idle state. We deassert read request as well.

```
    WAIT_READ: begin
      avl_read <= 1'b0;
      if (avl_readdatavalid) begin
        data_reg <= avl_readdata;
        c_state <= IDLE;
      end
    end
```

In case of a software reset, we fall back to Idle state.

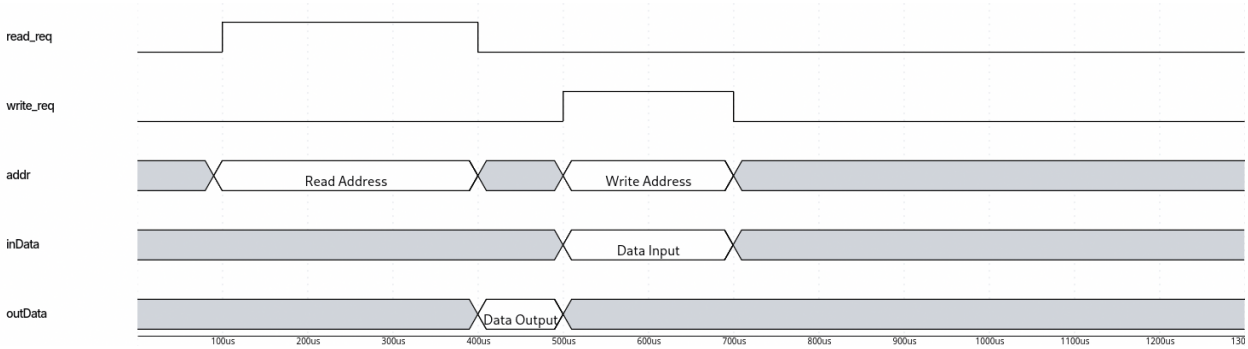


Figure 35: The final interface we create. Simpler than the original.

Let's implement the above logic in the hardware.

Parameters are compile time values that allow us to have variable runtime constants.

```
module lpddr2_memory #(
    parameter ADDR_W = 27,
    parameter DATA_W = 32
) (
    input                       iCLK,
    input                       iRST_n,
    input                       read_req,
    input                       write_req,
```

```
9       input   [ADDR_W-1:0]        addr,
10      input   [DATA_W-1:0]        inData,
11      output  [DATA_W-1:0]        outData,
12      input                       local_init_done,
13      input                       avl_waitrequest_n,
14      output reg [ADDR_W-1:0]     avl_address,
15      input                       avl_readdatavalid,
16      input   [DATA_W-1:0]        avl_readdata,
17      output reg [DATA_W-1:0]     avl_writedata,
18      output reg                  avl_read,
19      output reg                  avl_write,
20      output                      avl_burstbegin,
21      output reg [3:0]            c_state
22  );
23      reg [DATA_W-1:0] data_reg;
24
25    // Used to request a burst. We just have bursts of 1, but
26    // Potentially could be improved, used.
27      assign avl_burstbegin = avl_write || avl_read;
28      assign outData = data_reg;
29
30      // State definitions, equivalent to Enum in verilog
31      localparam INIT       = 3'd0,
32                 IDLE       = 3'd1,
33                 WRITE      = 3'd2,
34                 WAIT_WRITE = 3'd3,
35                 READ       = 3'd4,
36                 WAIT_READ  = 3'd5;
37
38      // State machine
39      always @(posedge iCLK) begin
40          if (!iRST_n) begin
41              c_state <= INIT;
42              avl_write <= 1'b0;
```

```
43              avl_read <= 1'b0;
44          end
45          else begin
46              case (c_state)
47                  INIT: begin
48                      avl_address <= addr;
49                      if (local_init_done)
50                          c_state <= IDLE;
51                  end
52
53                  IDLE: begin
54                      if (read_req)
55                          c_state <= READ;
56                      else if (write_req)
57                          c_state <= WRITE;
58                  end
59
60                  WRITE: begin
61                      avl_writedata <= inData;
62                      avl_address <= addr;
63                      avl_write <= 1'b1;
64                      c_state <= WAIT_WRITE;
65                  end
66
67                  WAIT_WRITE: begin
68                      if (avl_waitrequest_n) begin
69                          c_state <= IDLE;
70                          avl_write <= 1'b0;
71                      end
72                  end
73
74                  READ: begin
75                      avl_address <= addr;
76                      avl_read <= 1'b1;
```

```
77                      if (avl_waitrequest_n)
78                          c_state <= WAIT_READ;
79                  end
80
81                  WAIT_READ: begin
82                      avl_read <= 1'b0;
83                      if (avl_readdatavalid) begin
84                          data_reg <= avl_readdata;
85                          c_state <= IDLE;
86                      end
87                  end
88
89                  default: c_state <= INIT;
90              endcase
91          end
92      end
93  endmodule
```

14.5 How we use it (Memory master module)

Before we get into too much detail, you need to know - Memory on our board is partitioned into 2, SRAM and LPDDR2.

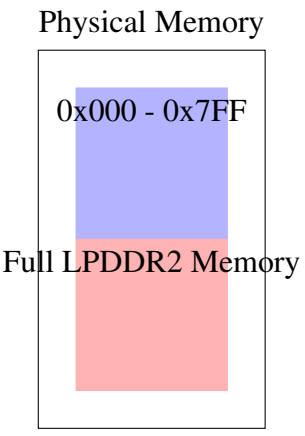


Figure 36: Memory partitioning

In the figure above, you can see how the memory is partitioned. At the beginning, we have SRAM memory, and after some address is reached, the hardware starts using LPDDR2 memory.

This was done so that we could use the SRAM for initialization. SRAM can be preloaded with test code, or a bootloader in the future, which itself could load data from a disk. To not complicate the switching, the address is directly mapped in both places - this means that both SRAM and LPDDR2 use input address from the CPU without any changes. LPDDR2 is used only when 0x7FF address is exceeded. This has an unintended side effect of LPDDR2 only being available from address 0x800, wasting first few kilobytes. Not that it matters much - LPDDR2 is 512MB (4Gbit).

14.5.1 LPDDR2 specific

Let's take a look at inputs and outputs:

```
module memory_master(
  input wire mem_clk,
  input wire mem_wren,
  input wire mem_rren,
  input wire [31:0] data_in,
  output wire [31:0] out,
  input wire E,
  // LPDDR2 Memory
  output wire [26:0] address,
  output wire [31:0] write_data,
  input wire [31:0] read_data,
  output wire read_req,
  output wire write_req
);
```

At the top, we have incoming signals from the CPU. Things such as memory clock, data in, read/write requests, data output and the E register.

At the bottom we have outgoing signals to LPDDR2. address, write/read data, and read/write requests.

The two sets of wires seem pretty similar, and that is because in most cases they're directly mapped to each other. However, having the SRAM at the bottom forces us to add some additional logic in-between.

Switching:

```
wire is_lpddr2_used = addr_in > {11{1'b1}};
```

This wire, `is_lpddr2_used` acts as a switch between the 2 types of memories. As soon as we exceed 111'b1 (max 11 bit number), we switch to lpddr2.

Other logic:

- address is directly mapped
- write_data is directly mapped
- `write_req = E && mem_wren && is_lpddr2_used` - write is only requested when executing, memory write is enabled (we have a sw instruction) and address space is within LPDDR2.
- `read_req = is_lpddr2_used && ~write_req && (E && mem_rren | E)` - read is only requested when address space is within LPDDR2, we have no write request, and
  - We are not in execute stage OR
  - We are in execute stage, yet have a read request (lw instruction)

That's it for LPDDR2 specific logic here. Surprisingly simple.

Now let's get to the SRAM implementation.

14.5.2 SRAM specific and common

SRAM is implemented as we have already seen before, with a simple interface provided by the IP.

```
sram memory_sram (
  .address(addr_in[11:0]),
  .clock(mem_clk),
  .data(data_in),
  .wren(sram_wren),
  .q(sram_q)
);
```

Only notable difference is that we use only first 11 bits of the address.

As for the `sram_wren`, it is set according to the following logic:

```
sram_wren = E && mem_wren && !is_lpddr2_used
```

SRAM is only written to when executing, write is enabled (sw instruction), and LPDDR2 isn't used.

That's all for the SRAM related logic. As for the common logic - we have a simple MUX that switched between SRAM output and LPDDR2 output.

```
assign out = is_lpddr2_used ? read_data : sram_q;
```

That's the entire memory module.

14.5.3 Code

```
module memory_master(
  input wire mem_clk,
  input wire [29:0] addr_in,
  input wire [31:0] data_in,
  input wire mem_wren,
  input wire mem_rren,
  output wire [31:0] out,
  input wire E,
  // LPDDR2 Memory
  output wire [26:0] address,
  output wire [31:0] write_data,
  input wire [31:0] read_data,
  output wire read_req,
  output wire write_req
);

wire [31:0] sram_q;

wire is_lpddr2_used = addr_in > {11{1'b1}};
wire sram_wren = E && mem_wren && !is_lpddr2_used;

assign out = is_lpddr2_used ? read_data : sram_q;

sram memory_sram (
  .address(addr_in[11:0]),
```

```
26    .clock(mem_clk),
27    .data(data_in),
28    .wren(sram_wren),
29    .q(sram_q)
30 );
31
32 // LPDDR2
33 assign address = addr_in;
34 assign write_req = E && mem_wren && is_lpddr2_used;
35 assign read_req = is_lpddr2_used && ~write_req && (E && mem_rren || ~
       E);
36 assign write_data = data_in;
37
38 endmodule
```

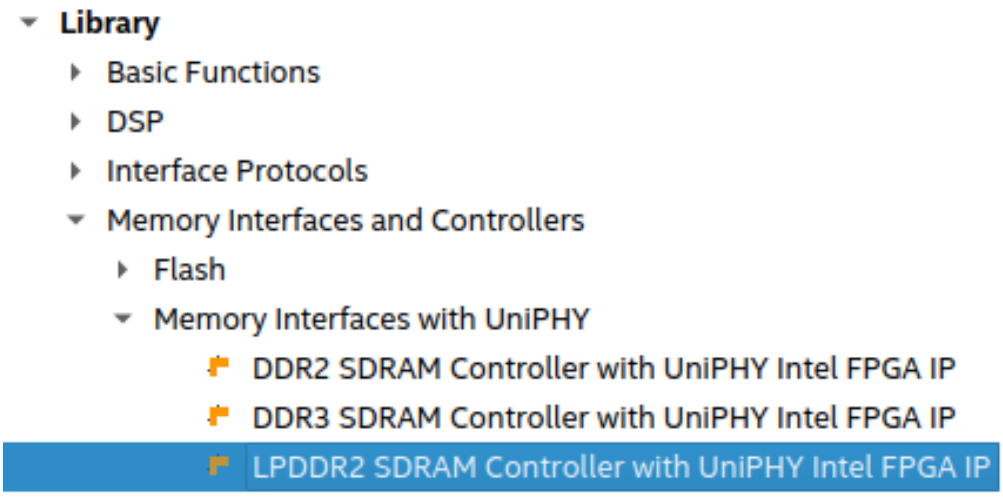
14.6 User Guide for LPDDR2 Memory

This section provides a step-by-step guide on how to use the LPDDR2 memory module in your FPGA project.

14.6.1 Setting Up the IP Core

Before using the LPDDR2 memory controller, you need to set up the LPDDR2 IP core in your Quartus project:

1. Open Quartus Prime and create a new project or open your existing project
2. Go to Tools → IP Catalog
3. Navigate Here:



4. Set up your module with settings similar to the CPU module on the git repository [3]; the matching `.qprs` configuration file is included there.

That should be enough to get the IP up and running.

14.6.2 Integrating the Memory Controller

To integrate the LPDDR2 memory controller into your design, follow these steps:

Step 1: Instantiate the Memory Controller Module Add the `lpddr2_memory` module to your project and instantiate it in your top-level design:

```
lpddr2_memory #(
    .ADDR_W(27),  // Adjust based on your memory size
    .DATA_W(32)   // 32-bit data width
) lpddr2_controller (
    .iCLK(afi_half_clk),              // Clock from the LPDDR2 IP core
    .iRST_n(reset_n),                 // Active-low reset
    .read_req(read_req),              // Read request signal
    .write_req(write_req),            // Write request signal
    .addr(address),                   // Memory address
    .inData(write_data),              // Data to write
    .outData(read_data),              // Data read from memory
    .local_init_done(local_init_done), // Signal from IP core
    .avl_waitrequest_n(avl_ready),    // Ready signal from IP core
    .avl_address(avl_addr),           // Address to IP core
```

```
    .avl_readdatavalid(avl_rdata_valid), // Data valid signal
    .avl_readdata(avl_rdata),         // Data from IP core
    .avl_writedata(avl_wdata),        // Data to IP core
    .avl_read(avl_read_req),          // Read signal to IP core
    .avl_write(avl_write_req),        // Write signal to IP core
    .avl_burstbegin(avl_burstbegin), // Burst begin signal
    .c_state(c_state)                 // State machine state (for
        debugging)
);
```

Step 2: Connect to the LPDDR2 IP Core Connect the memory controller to the LPDDR2 IP core generated in the setup step:

```
lpddr2 lpddr2_ip_instance(
    .pll_ref_clk(CLOCK_50_B5B),        // System clock
    .global_reset_n(global_reset_n),  // Global reset
    .soft_reset_n(soft_reset_n),      // Soft reset
    .afi_clk(afi_clk),                // Clock output
    .afi_half_clk(afi_half_clk),      // Half-rate clock
    .mem_ca(DDR2LP_CA),               // Command/Address to memory
    .mem_ck(DDR2LP_CK_p),             // Clock positive
    .mem_ck_n(DDR2LP_CK_n),           // Clock negative
    .mem_cke(DDR2LP_CKE[0]),          // Clock enable
    .mem_cs_n(DDR2LP_CS_n[0]),        // Chip select
    .mem_dm(DDR2LP_DM),               // Data mask
    .mem_dq(DDR2LP_DQ),               // Data
    .mem_dqs(DDR2LP_DQS_p),           // Data strobe positive
    .mem_dqs_n(DDR2LP_DQS_n),         // Data strobe negative

    // Avalon interface (connect to memory controller)
    .avl_ready_0(avl_ready),
    .avl_burstbegin_0(avl_burstbegin),
    .avl_addr_0(avl_addr),
    .avl_rdata_valid_0(avl_rdata_valid),
    .avl_rdata_0(avl_rdata),
```

```
23      .avl_wdata_0(avl_wdata),
24      .avl_be_0(4'hF),                    // Byte enable (all active)
25      .avl_read_req_0(avl_read_req),
26      .avl_write_req_0(avl_write_req),
27      .avl_size_0(3'b001),                // Burst size of 2
28
29      // Memory interface clocks
30      .mp_cmd_clk_0_clk(afi_half_clk),
31      .mp_cmd_reset_n_0_reset_n(soft_reset_n),
32      .mp_rfifo_clk_0_clk(afi_half_clk),
33      .mp_rfifo_reset_n_0_reset_n(soft_reset_n),
34      .mp_wfifo_clk_0_clk(afi_half_clk),
35      .mp_wfifo_reset_n_0_reset_n(soft_reset_n),
36
37      // Status signals
38      .local_init_done(local_init_done),
39      .local_cal_success(),
40      .local_cal_fail(),
41
42      // OCT (On-Chip Termination)
43      .oct_rzqin(DDR2LP_OCT_RZQ)
44  );
```

This IP abstracts away many of the things from us. The wires on hardware are CLOCK_50_B5B and any wire that starts with DDR2LP. Soft reset is user-defined. Rest of the wires are inputs/outputs from/to others modules.

Step 3: Memory Access Patterns When accessing the LPDDR2 memory from your design, follow these guidelines:

1. Assert the appropriate request signal (`read_req` or `write_req`)
2. For writes, provide the address and data
3. For reads, provide the address and wait for the data to become available
4. Monitor the controller state through the `c_state` output if needed for debugging

Example read operation (pseudocode):

```
// Start a read operation
read_req <= 1'b1;
address <= memory_address;

// Wait for one cycle and then...
read_req <= 1'b0;

// Wait for the read to complete (c_state returns to IDLE)
wait(c_state == IDLE);

// Data is now available in read_data
process_data(read_data);
```

Example write operation (pseudocode):

```
// Start a write operation
write_req <= 1'b1;
address <= memory_address;
write_data <= data_to_write;

// Wait for one cycle
write_req <= 1'b0;

// Wait for the write to complete (c_state returns to IDLE)
wait(c_state == IDLE);
```

14.6.3 Compiler Directive for Memory Selection

A feature of our implementation is to switch between the mock memory (Register memory) and the real hardware memory (which includes both SRAM on lower addresses and LPDDR2 RAM on higher addresses) using compiler directives. This is achieved through the `‘ifdef HARDWARE` directive.

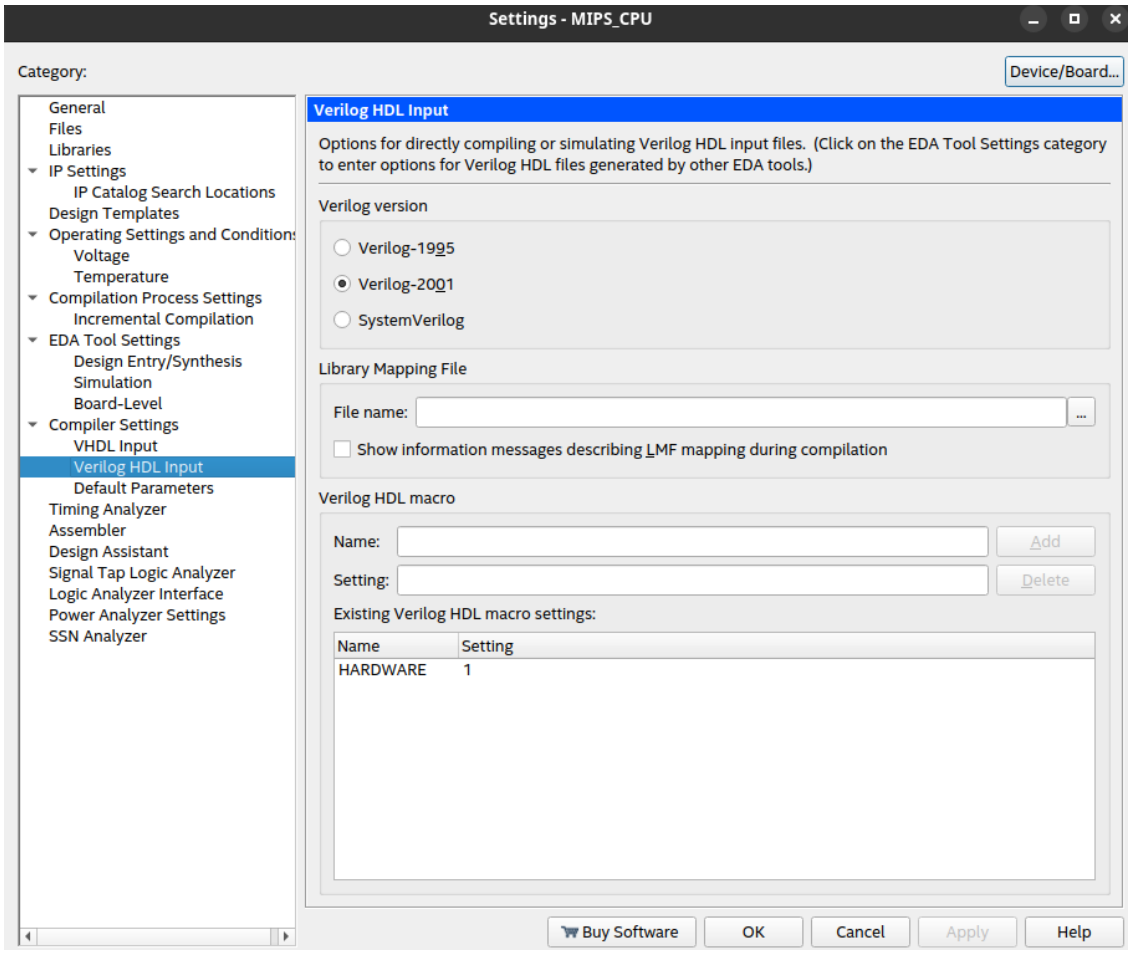
Implementation The compiler directive is used to conditionally select which memory module is instantiated:

```
`ifdef HARDWARE
// Memory stage with E signal
memory_master memory(
    .mem_clk(mem_clk),
    .rst(rst),
    .addr_in(~E ? pc[31:2] : alu_res),
    .data_in(b_gpr),
    .mem_rren(mem_rren),
    .mem_wren(mem_wren),
    .out(mem_out),
    .E(E),
    // LPDDR2 Memory
    .address(lpddr2_address),
    .write_data(lpddr2_write_data),
    .read_data(lpddr2_read_data),
    .read_req(lpddr2_rreq),
    .write_req(lpddr2_wreq)
);
`else
memory_master_mock memory(
    .mem_clk(mem_clk),
    .rst(rst),
    .addr_in(~E ? pc[31:2] : alu_res),
    .data_in(b_gpr),
    .mem_rren(mem_rren),
    .mem_wren(mem_wren),
    .gp_we(gp_we),
    .out(mem_out),
    .E(E)
);
`endif
```

When the `HARDWARE` macro is defined, the real memory master module is instantiated with connections to the LPDDR2 hardware. When the macro is not defined, the simplified `memory_master_mock` module is used instead.

Configuration in Quartus The compiler directive is typically configured in the Quartus project settings:

1. In Quartus Prime, navigate to Assignments → Settings
2. Select Compiler Settings in the left panel
3. Click on the Verilog HDL Input tab
4. In the Verilog HDL Macro section, add the macro definition:
    - To enable hardware memory: Add HARDWARE=1
    - To use mock memory: Leave the macro undefined or remove it



Alternatively, the macro can be defined in a source file that is included in the project:

```
`define HARDWARE 1
```

15 Interrupts

This design is based on Chapter 14 of [5]. Note that we are not addressing external devices. The *sysc* interrupt will be detected but it should be handled by the operating system, not the hardware. Additionally, to handle "abort" instructions for the module tests, the $*stop* command is used. This halts the simulation but would not stop the actual hardware on an FPGA. For the moment being, $pfff$ and $pfls$ interrupts are also handled in the same manner, even though they are of "repeat" type, as implementing memory translation and paging is out of the scope of this work.

For actual stopping, we will add additional signal, which will only be turned off if an interrupt occurs. It is important to recall that *reset* was addressed in the original code. The table below shows all the interrupts described in [5]:

index j	synonym for $ev[j]$	name	maskable	resume
0	*reset*	reset	no	abort
$j \in [1:15]$	$eev[j]$	external event signal from a device	yes	repeat
16	*malf*	misaligned fetch	no	abort
17	*pff*	page fault during fetch	no	repeat
18	*ill*	illegal instruction	no	abort
19	*malls*	misaligned load/store	no	abort
20	*pfls*	page fault during load/store	no	repeat
21	*sysc*	system call	yes	continue
22	*ovf*	arithmetic overflow	yes	continue

Figure 37: Interrupts handled by our design

15.1 Changes to the Original Code

In the ‘i_decoder.v‘, *is_illegal* output wire was added. It’s value is computed by:

```
assign is_illegal = Type(opc, fun, rt, rs) == 2'b11;
```

Correspondingly, this same output parameter was added in ‘decode_master.v‘ as the output parameter. Note that the instantiation of ‘decoder_master.v‘ in the ‘master.sv‘ should also be changed accordingly. Major change is in *Type* method in ‘i_decoder.v‘ (check 8th chapter of [5] for previous decoder computation):

```
function [1:0] Type(input [5:0] opc, input [5:0] fun, input [4:0] rt,
    input [4:0] rs);
    begin
        casez (opc)
            6'b10?011, 6'b001???, 6'b00010?:
                Type = 2'b00; // I-Type
```

```
6               6'b000001:
7                   if (rt[4:1] == 4'b0000)
8                       Type = 2'b00; // I-Type
9               6'b00011?:
10                  if (rt == 5'b00000)
11                      Type = 2'b00; // I-Type
12              6'b000000:
13                  if (fun[5:3] == 3'b100 || fun == 6'b000010 || fun[5:1] ==
                        5'b10101 || fun == 6'b001000 || fun == 6'b001010 || fun
                        == 6'b001100)
14                      Type = 2'b10; // R-Type
15              6'b010000:
16                  if ((fun == 6'b011000 && rs == 5'b10000) || rs == 5'b00100
                        || rs == 5'b00000)
17                      Type = 2'b10; // R-Type
18              6'b00001?:
19                  Type = 2'b01; // J-Type
20              default: begin
21                  Type = 2'b11; // None of the above, undefined(c) ->
                        illegal instruction
22                  $display("ill: Undefined Instruction");
23                  // $stop; // According to Table 13 on page 377, ill should
                        be aborted
24              end
25          endcase
26      end
27  endfunction
```

The Verilog function above had to be changed as the predicates for determining the instruction type:

$$rtype(c) \equiv opc(c) = 0 * 0^4$$

$$jtype(c) \equiv opc(c) = 0^4 1 *$$

$$itype(c) \equiv \overline{rtype(c) \vee jtype(c)}$$

There are 16 instructions of $itype(c)$ (see table in chapter 8.1.1 of [5]), but its predicates covered more 32-bit strings than there were instructions. For undefined instructions, an *ill* interrupt should be raised, so stricter conditions were required.

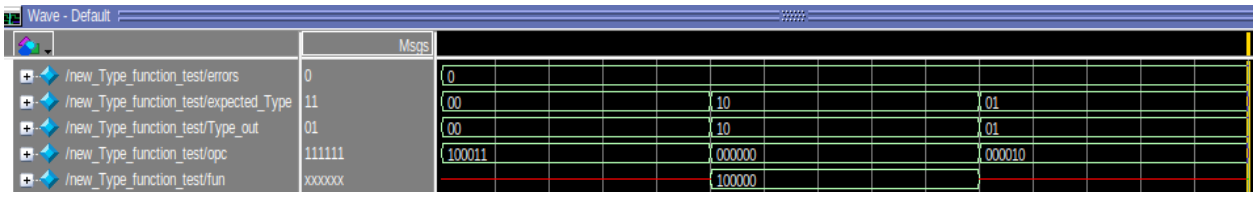


Figure 38: Testbench for updated 'Type' function

Code below shows testbench for new *Type* function. We have intentionally shown inputs and outputs after 3rd test, because 4th instruction is illegal and $*stop* actually stops the whole simulation, so no new output for ModelSim.

```
// Test 1: I-Type (6'b100011)
opc = 6'b100011; fun = 6'bxxxxxx; rt = 5'bxxxxx; rs = 5'bxxxxx;
expected_Type = 2'b00;
#10;

// Test 2: R-Type (6'b000000, fun = 6'b100000)
opc = 6'b000000; fun = 6'b100000; rt = 5'bxxxxx; rs = 5'bxxxxx;
expected_Type = 2'b10;
#10;

// Test 3: J-Type (6'b000010)
opc = 6'b000010; fun = 6'bxxxxxx; rt = 5'bxxxxx; rs = 5'bxxxxx;
expected_Type = 2'b01;
#10;

// Test 4: Illegal Instruction (default case)
opc = 6'b111111; fun = 6'bxxxxxx; rt = 5'bxxxxx; rs = 5'bxxxxx;
expected_Type = 2'b11;
#10;
```

We also have to modify *WhereFrom* function to include detection of *movs2g* instruction. We increase width of the output from 3 bits to 4 and update accordingly:

```
function [2:0] WhereFrom(input [1:0]ttype, input [5:0] opc, input [5:0]
    fun, input [4:0] rs);
    case(ttype)
        2'b10: begin
            if (opc == 6'b010000 && rs == 5'b00000)
                WhereFrom = 3'b100; // movs2g
            casez(fun)
                6'b001001: WhereFrom = 3'b011; // PC
                6'b000010, 6'b000011: WhereFrom = 3'b010; //Shift
                default: WhereFrom = 3'b000; // ALU
            endcase
        end
        2'b00: if(opc == 6'b100011) WhereFrom = 3'b001; //Memory
            else casez (opc)
                6'b001???: WhereFrom = 3'b000; //ALU
                default: WhereFrom = 3'b011; //PC
            endcase
        default: WhereFrom = 3'b11;
    endcase
endfunction
```

We add wire for special purpose register mux select:

```
wire spr_mux_sel = rs != 5'b00000;
```

This also need to be reconsidered in ‘decoder_concat.sv‘:

```
module decoder_concat (
    input [3:0] af,
    input i,
    input alu_mux_sel,
    input [2:0] shift_type,
    input [4:0] cad,
    input gp_we,
    input [2:0] gp_mux_sel,
    input [3:0] bf,
    input [1:0] pc_mux_select,
    input spr_mux_sel,
    input mem_wren,
    input mem_rren,
    input [4:0] rs, rt, rd,
```

```
    output [41:0] packed_out
);

    assign packed_out= {af, i, alu_mux_sel, shift_type, cad, gp_we,
        gp_mux_sel, bf, pc_mux_select, spr_mux_sel, mem_wren, mem_rren, rs
            , rt, rd};

endmoduleassign
```

Notice that we have expanded the width of *gp_mux_sel* by one bit (because the width of *WhereFrom* was increased). Accordingly, the width of *packed_out* must also be increased. Additionally, the *spr_mux_sel* wire was added, and it must be included as an output in `decoder_deconcat.sv`.

The widths of *gp_mux_sel* and *packed_out*, along with the addition of *spr_mux_sel*, need to be updated in every instance where they appear. Here is the list of files and their corresponding name alternatives for *packed_out*:

- `decoder_concat.sv` and `decoder_deconcat.sv` → *packed_out*
- `master.sv`, `decode_master.v` and `execute_master.v` → *decoder_packed*
- `next_pc_calc.v` → *decode_data_packed*
- `i_decoder.v` → *out_data_packed*

We also need to update ‘writeback_master.sv‘ and its instances to include $[31:0]spr_in$ and increase the width of *gp_mux_sel*:

```
module writeback_master(
    input [31:0] memory_out,
    input [31:0] alu_res,
    input [31:0] shift_res,
    input [31:0] link_addr,
    input [31:0] spr_in,
    input [2:0] gp_mux_sel,
    output [31:0] gpr_data_in
);

assign gpr_data_in =
    (gp_mux_sel == 3'b000) ? alu_res :
    (gp_mux_sel == 3'b001) ? memory_out :
    (gp_mux_sel == 3'b010) ? shift_res :
    (gp_mux_sel == 3'b011) ? link_addr :
    (gp_mux_sel == 3'b100) ? spr_in :
    32'b0;

endmodule
```

Finally, for ‘au.v‘ and ‘alu.v‘, since *ovf* was continue interrupt, we simply added this lines in ‘au.v‘ and ‘alu.v‘ to display simple message with both operands and bit indicating whether addition or substitution was performed:

```
if (ovf) begin
    $display("Overflow detected with a = %b, b = %b, sub = %b", a, b, sub)
        ;
end
```

15.2 Find First One Circuit (FF1)

The "find first one" circuit determines the position of the first occurrence of a bit with value 1 in a binary string, where the search is performed starting from the least significant bit. Although in the context of this work the FF1 circuit is specifically applied to identify the interrupt level, its underlying principles and design make it a compelling subject for further exploration.

15.2.1 Parallel Prefix Circuit

According to [5]: "An n-parallel prefix circuit for an associative function $\circ : M \times M \to M$ is a circuit with inputs $x[n-1:0] \in M^n$ and outputs $y[n-1:0] \in M^n$ satisfying $y_0 = x_0$ and $y_{i+1} = x_{i+1} \circ y_i$". They have a depth of $d(n) = 2 \cdot \log n - 1 = O(\log n)$ and a cost of $c(n) \leq 2 \cdot n = O(n)$.

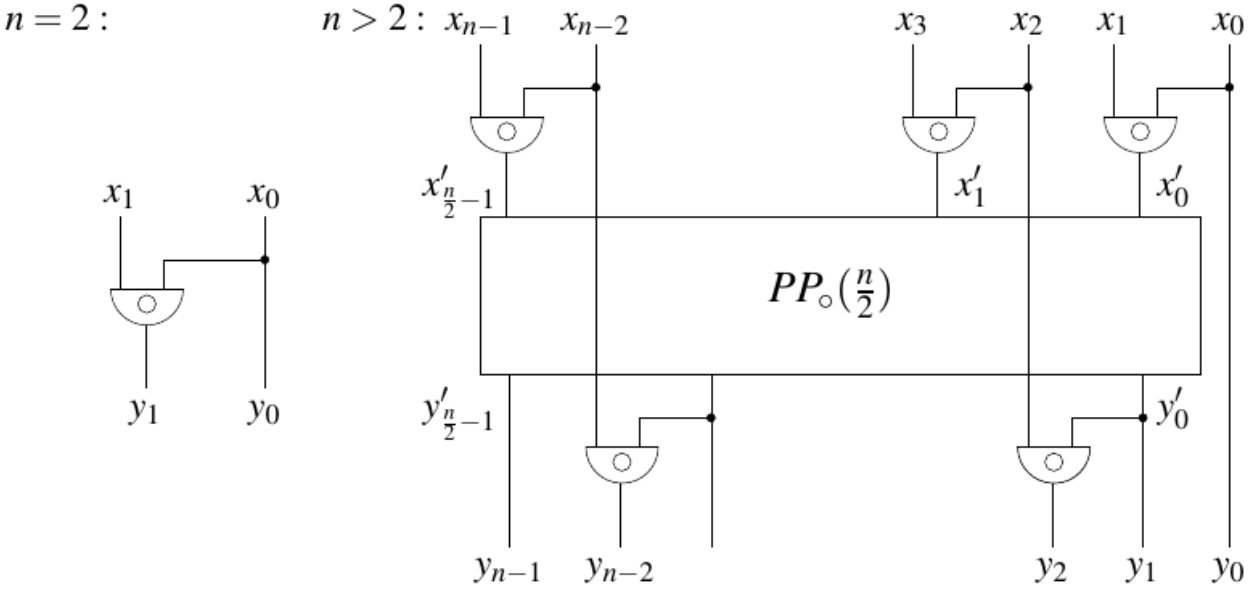


Figure 39: Recursive construction of an n-bit parallel prefix circuit of the function $\circ$ for an even n ([5], Fig. 35, p. 86)

We will use logical OR as an associative operation with bits as the input-output domain, i.e., $\vee : \mathbb{B}^n \to \mathbb{B}^n$. This circuit will efficiently calculate the "tail" of consecutive 1s. For an input $x \in \mathbb{B}^n$, the output is given by $y = 1^{n-ff1(x)}0^{ff1(x)}$ where $ff1(x) = \min\{i \geq 0 \mid x[i] = 1\}$.

SystemVerilog (with the .sv extension) allows recursive generation of circuits. Although this feature is not directly supported in Verilog (with the .v extension), ModelSim appears to use a

SystemVerilog compiler whenever it encounters recursive constructions. Below is the Verilog (SystemVerilog) implementation of the circuit described above:

```
module pp_or #(parameter N = 32)(
    input  wire [N-1:0] x,
    output wire [N-1:0] y
);
    generate
        if (N == 2) begin: base_case
            assign y[0] = x[0];
            assign y[1] = x[1] | x[0];
        end else begin: recursive_case
            wire [N/2-1:0] x_prime;
            genvar i;
            for (i = 0; i < N/2; i = i + 1) begin: combine_pairs
                assign x_prime[i] = x[2*i] | x[2*i + 1];
            end

            wire [N/2-1:0] y_prime;
            pp_or #(.N(N/2)) pp_rec (.x(x_prime), .y(y_prime));

            assign y[0] = x[0];
            for (i = 1; i < N/2; i = i + 1) begin: even_indices
                assign y[2*i] = x[2*i] | y_prime[i-1];
            end
            for (i = 0; i < N/2; i = i + 1) begin: odd_indices
                assign y[2*i + 1] = y_prime[i];
            end
        end
    endgenerate
endmodule
```

The tests on the left with the correct outputs on the right:

```
  0 x = 00000000000000000000000000000000, y = 00000000000000000000000000000000
 10 x = 10000000000000000000000000000000, y = 10000000000000000000000000000000
 20 x = 11000000000000000000000000000000, y = 11000000000000000000000000000000
 30 x = 11100000000000000000000000000000, y = 11100000000000000000000000000000
 40 x = 11111111000000000000000000000000, y = 11111111000000000000000000000000
 50 x = 10101010101010101010101010101010, y = 11111111111111111111111111111110
 60 x = 01010101010101010101010101010101, y = 11111111111111111111111111111111
 70 x = 00000000000001000000000000000000, y = 11111111111111000000000000000000
 80 x = 00000001000000000000000000000000, y = 11111111100000000000000000000000
 90 x = 00000000000000000010000000000000, y = 11111111111111111110000000000000
100 x = 00000000000000000010000000000001, y = 11111111111111111111111111111111
```

Figure 40: Output in ModelSim's Transcript window

15.2.2 $n-1$ NOT gates

The final step is to remove the extra $n - ff1(x) - 1$ consecutive 1s from the most significant bits, leaving only the $ff1(x)$ bit on, and then convert the "one-hot encoded" value to its binary representation $ff1_x$ such that $\langle ff1(x) \rangle = ff1_x$. In the following Verilog design, the output is 5 bits wide, as this is the minimum number of bits required to address all possible values of the interrupt level:

```
module ff1 #(parameter N = 32)(
    input  wire [N-1:0] x,
    output wire [N-1:0] y
);
    wire [N-1:0] temp_y;

    pp_or #(.N(N)) parallel_prefix_or (.x(x), .y(temp_y));

    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin: out_invert
            if (i == 0) begin
                assign y[i] = temp_y[i];
            end else begin
                assign y[i] = temp_y[i] & ~temp_y[i-1];
            end
        end
    endgenerate
endmodule
```

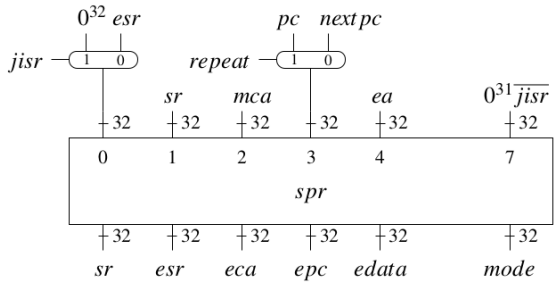
The tests on the left with the correct outputs on the right:

```
  0 x = 00000000000000000000000000000000, y = 00000000000000000000000000000000
 10 x = 10000000000000000000000000000000, y = 10000000000000000000000000000000
 20 x = 11000000000000000000000000000000, y = 01000000000000000000000000000000
 30 x = 11100000000000000000000000000000, y = 00100000000000000000000000000000
 40 x = 11111111000000000000000000000000, y = 00000001000000000000000000000000
 50 x = 10101010101010101010101010101010, y = 00000000000000000000000000000010
 60 x = 01010101010101010101010101010101, y = 00000000000000000000000000000001
 70 x = 00000000000000100000000000000000, y = 00000000000000100000000000000000
 80 x = 00000001000000000000000000000000, y = 00000001000000000000000000000000
 90 x = 00000000000000000010000000000000, y = 00000000000000000010000000000000
100 x = 00000000000000000010000000000001, y = 00000000000000000000000000000001
```

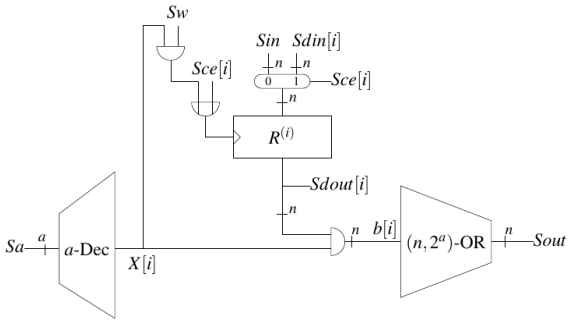
Figure 41: Output in ModelSim's Transcript window

15.3 Special Purpose Register

Design for Special Purpose Register file(SPR for short) from [5] looks as follows:



(a) symbol ([5], Fig. 209, p. 382, missing *pto* and *ptl*)



(b) implementation

Figure 42: Special Purpose Register file

However, we are missing 2 special purpose registers that were listed in the table at the beginning of the 14th chapter of [5].

0	*sr*	status register
1	*esr*	exception status register
2	*eca*	exception cause
3	*epc*	exception *pc*
4	*edata*	exception data
5	*pto*	page table origin
6	*ptl*	page table length
7	*mode*	mode register

Figure 43: Special Purpose Registers

Missing registers are page table origin(pto) and page table length(ptl). Both of these registers only make sense after defining address translation and paging, which is done later in the same chapter of [5], but will not be treated here. However, Verilog implementation should include both registers as outputs.

Note the change in computation of *eca_out* when $jisr(d,eev)$. We are concatenating $mca(d,eev)$ with 9 zeros instead of 10, even though in [5] we have $d'.eca = 0^{10} \circ mca(d,eev)$, but this is a mistake since $mca(d,eev)$ is 23 bits wide, not 22(interrupts 0 to 22 inclusive), so it should actually be $d'.eca = 0^{9} \circ mca(d,eev)$.

```
1 module spr (
2     input wire clk,
3     input wire jisr,
4     input wire eret,
```

```
5       input wire [22:0] mca,
6       input wire rpt,  // 1 bit 'repeat' signal
7       input wire [31:0] pc,
8       input wire [31:0] next_pc,
9       input wire [31:0] ea,
10      input wire [31:0] data_in,
11      input wire [2:0] reg_sel,
12      input wire sprw,
13      output wire [31:0] spr_out,
14      output wire [31:0] sr,
15      output wire [31:0] epc,
16      output wire [31:0] mode
17  );
18      reg [31:0] spr [7:0];
19
20      initial begin : init_spr
21          integer i;
22          for (i = 0; i < 8; i = i + 1)
23              spr[i] = 32'b0;
24      end
25
26      always @(posedge clk) begin
27          if (jisr) begin
28              spr[1] = spr[0];  // moved up as we first store value of d.sr
                    in d'.esr
29              spr[0] = 32'b0;
30              spr[2] <= {9'b0, mca};
31              spr[3] <= ((rpt) ? pc : next_pc);
32              spr[4] <= ea;
33              spr[5] <= spr[5];  // paging is NOT implemented
34              spr[6] <= spr[6];  // paging is NOT implemented
35              spr[7] <= 32'b0;  // user mode
36          end else if (eret) begin
37              spr[0] <= spr[1];  // d'.sr = d.esr
38              spr[7] <= {31'b0, 1'b1};  // d'.mode = 0^{31} 1
39              spr[1] <= spr[1];
40              spr[2] <= spr[2];
41              spr[3] <= spr[3];
42              spr[4] <= spr[4];
43              spr[5] <= spr[5];
44              spr[6] <= spr[6];
45          end else begin
46              spr[0] <= (reg_sel==3'b000 && sprw) ? data_in : spr[0];
47              spr[1] <= (reg_sel==3'b001 && sprw) ? data_in : spr[1];
48              spr[2] <= (reg_sel==3'b010 && sprw) ? data_in : spr[2];
49              spr[3] <= (reg_sel==3'b011 && sprw) ? data_in : spr[3];
50              spr[4] <= (reg_sel==3'b100 && sprw) ? data_in : spr[4];
51              spr[5] <= (reg_sel==3'b101 && sprw) ? data_in : spr[5];
52              spr[6] <= (reg_sel==3'b110 && sprw) ? data_in : spr[6];
53              spr[7] <= (reg_sel==3'b111 && sprw) ? data_in : spr[7];
54          end
55      end
56
57      assign spr_out = spr[reg_sel];
58      assign sr = spr[3'b000];
59      assign epc = spr[3'b011];
60      assign mode = spr[3'b111];
61
62  endmodule
```

The if statement inside the always block, which depends on the *jisr* signal, functions as described in [5] (except that the page table registers always have the same value, as they are not

utilized in this context):

$$d'.pc = 0_{32}$$
$$d'.sr = 0_{32}$$
$$d'.esr = d.sr$$
$$d'.eca = 0^9 \circ mca(d,eev)$$
$$d'.m = d.m$$
$$d'.mode = 0^{32}$$
$$d'.edata = ea(d)$$
$$repeat(d,eev) \equiv il(d,eev) \in [1:15] \cup \{17,20\}$$
$$d'.gpr = \begin{cases} \delta^{\text{old}}(d,0).gpr, & il(d,eev) = 22, \\ d.gpr, & \text{otherwise} \end{cases}$$
$$d'.epc = \begin{cases} d.pc, & repeat(d,eev), \\ \delta^{\text{old}}(d,0).pc, & \text{otherwise} \end{cases}$$

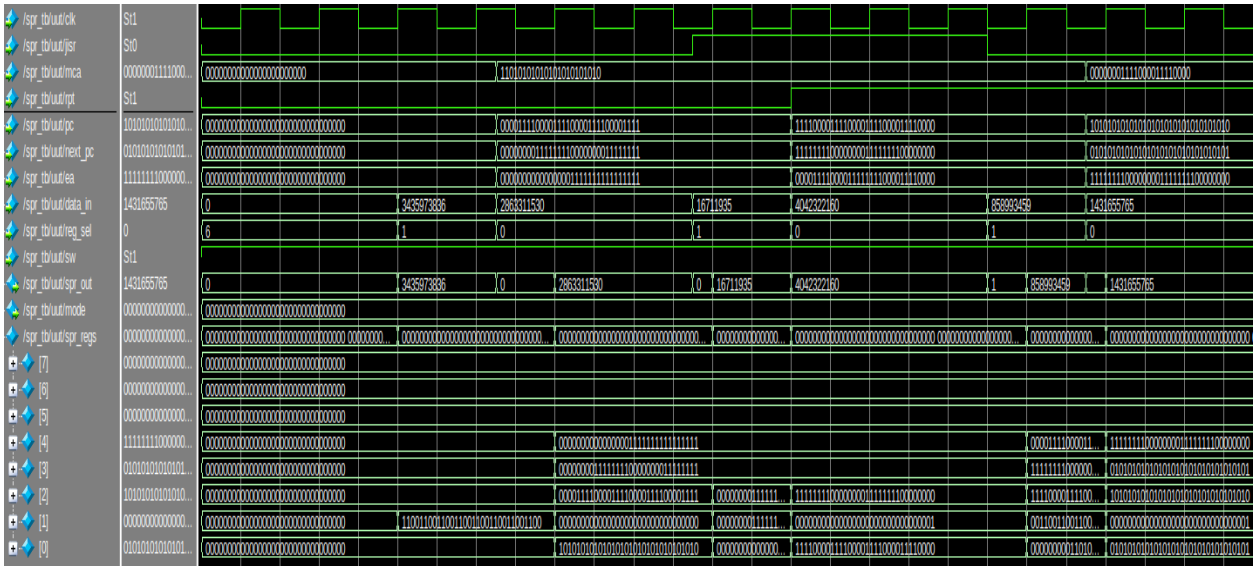


Figure 44: SPR Testbench Waveform

As for *eret*, from [5] we have:

$$d'.pc = d.epc$$
$$d'.sr = d.esr$$
$$d'.mode = 0^{31}1$$

To reproduce the results above, see testbench below:

```
initial begin
    clk = 0;
end

```

```
always
#2 clk = ~clk;

initial begin
    // Initialize inputs
    jisr = 1'b0;
    eret = 1'b0;
    mca = 23'b0;
    rpt = 1'b0;
    pc = 32'b0;
    next_pc = 32'b0;
    ea = 32'b0;
    data_in = 32'b0;
    reg_sel = 3'b110;
    sprw = 1'b1;

    // Example 1: No interrupt, rpt = 0
    #10;
    jisr = 1'b0;
    reg_sel = 3'b001;
    data_in = 32'b11001100110011001100110011001100;
    #5;
    reg_sel = 3'b0;
    data_in = 32'b10101010101010101010101010101010;
    mca = 23'b11010101010101010101010;
    rpt = 1'b0;
    pc = 32'b00001111000011110000111100001111;
    next_pc = 32'b00000000111111110000000011111111;
    ea = 32'b00000000000000001111111111111111;
    #5;

    // Example 2: Interrupt occurs, rpt = 1
    #5;
    jisr = 1'b1;
    reg_sel = 3'b001;
    data_in = 32'b00000000111111110000000011111111;
    #5;
    reg_sel = 3'b0;
    data_in = 32'b11110000111100001111000011110000;
    mca = 23'b11010101010101010101010;
    rpt = 1'b1;
    pc = 32'b11110000111100001111000011110000;
    next_pc = 32'b11111111000000001111111100000000;
    ea = 32'b00001111000011111111000011110000;
    #5;

    // Example 3: No interrupt, rpt = 1
    #5;
    jisr = 1'b0;
    reg_sel = 3'b001;
    data_in = 32'b00110011001100110011001100110011;
    #5;
    reg_sel = 3'b0;
    data_in = 32'b01010101010101010101010101010101;
    mca = 23'b00000111100001110000;
    rpt = 1'b1;
    pc = 32'b10101010101010101010101010101010;
    next_pc = 32'b01010101010101010101010101010101;
    ea = 32'b11111111000000001111111100000000;
    #5;

    // End simulation
    #10;
end
```

15.4 Interrupt Controller

Since transition function of MIPS ISA for new MIPS configuration has been updated from $\delta(d, reset)$ to $\delta(d, eev)$, we follow section 14.1.3(Pages 378 & 379) to compute *jisr* signal and interrupt level, but with a small change: the definition of $mca(d, eev)$ below masks interrupts $i \in [1:15] \cup \{20, 21\}$. Problem is, 20th interrupt is not maskable according to the Figure **??**, but 22nd is:

$$ca(d,eev)[i] = \begin{cases} eev[i], & i \leq 15, \\ iev(d)[i], & i \geq 16. \end{cases}$$

For maskable interrupts with index i we use bit i of the status register sr as the mask bit for this interrupt. Thus we define the vector $mca(d,eev)[22:0]$ of masked cause signals as

$$mca(d,eev)[i] = \begin{cases} ca(d,eev)[i] \wedge d.sr[i], & i \in [1:15] \cup \{20,21\}, \\ ca(d,eev)[i], & \text{otherwise.} \end{cases}$$

We jump to the interrupt service routine if any of the masked cause bits is on:

$$jisr(d,eev) = \bigvee_i mca(d,eev)[i].$$

In case this signal is active we define the interrupt level $il(d,eev)$ as the smallest index of an active masked cause bit:

$$il(d,eev) = \min\{i \mid mca(d,eev)[i]\}.$$

We handle interrupts with small indices with higher priority than interrupts with high indices.

Figure 45: *jisr* and interrupt level computation

```
module interrupt_controller (
    input wire [22:0] ca,  // Cause Signals
    input wire [31:0] sr,   // Status register
    output wire [22:0] mca,
    output wire jisr,
    output wire [31:0] il    // Interrupt level
);
    // Code segment below generates masked cause signals
    genvar i;
    generate
        for (i = 0; i < 23; i = i + 1) begin : gen_mca
            if (i >= 1 && i <= 15 || i == 21 || i == 22) // Unmaskable
                interrupts are reset, illegal instructions, misalignment
                and page faults
                 assign mca[i] = ca[i] & sr[i];
            else // Whose indices are 0, 16, 17, 18, 19, 20
                 assign mca[i] = ca[i];
        end
    endgenerate

    // Evaluating jisr predicate by ORing every bit
    assign jisr = |mca;

    // Use the find first one circuit to determine interrupt level
    ff1 #(32) ff1_inst (.x({9'b0, mca}), .y(il));
endmodule
```

Testcases with testbench:

```
// 0-indexed

// Test Case 1: No interrupts
ca = 23'b00000000000000000000000;
sr = 32'b00000000000000000000000000000000;
#10;

// Test Case 2: Masked interrupt at position 7
ca = 23'b00000000000000010000000;
sr = 32'b00000000000000000000000000000000;
#10;

// Test Case 3: Unmasked interrupt at position 21 and masked interrupt at
    position 1
ca = 23'b01000000000000000000010;
sr = 32'b00000000001000000000000000000000;
#10;

// Test Case 4: Multiple interrupts, highest priority at position 0
ca = 23'b00000000000000000000011;
sr = 32'b11111111111111111111111111111111;
#10;

// Test Case 5: Interrupts at positions 16 and 17, both should trigger
    unconditionally
ca = 23'b00000110000000000000000;
sr = 32'b00000000000000000000000000000000;
#10;
```

In case $il = 23$, then we have no interrupt, but we need not treat this special case:

$$jisr(d, eev) \Leftrightarrow \exists i < 23 : mca(d, eev)[i]$$

$$\neg jisr(d, eev) \Leftrightarrow \neg mca(d, eev)[i], \forall i \leq 22$$

. If $i = 23$, then $\neg jisr(d, eev)$ and we are not concerned about il. Thus, use of for loop for il is justified.

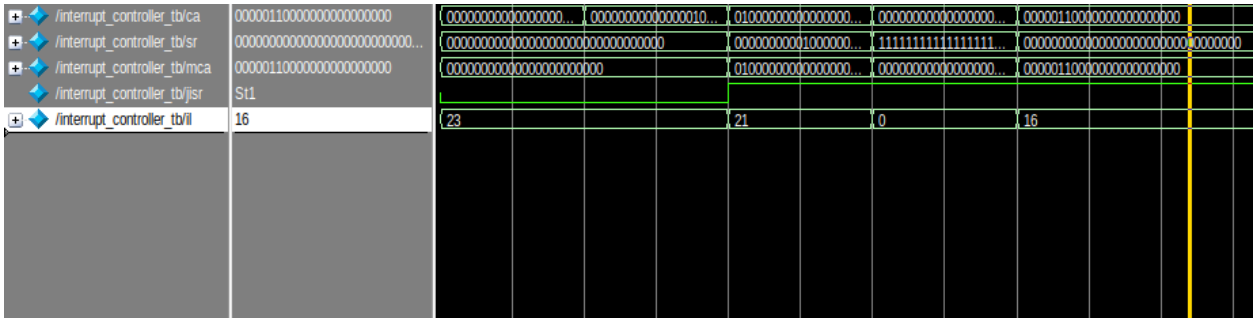


Figure 46: ‘interrupt_controller.v‘ testbench

On the waveform above, first 2 test cases should result in $il = 23$, as in the first test cases there are no interrupts and in the second test case we mask the interrupt. For the third case, 1st position(0-indexed) gets masked, but 21st bit does not get masked($\neg sr[21]$). For the fourth test

case, even though $sr[0] = 1$, interrupt level of 0 is not maskable, so it does not get masked and $il = 1$. For the final test case, 2 non-maskable interrupts($malf$ and pff) are both active, so one with the lowest priority($malf$ with $il = 16$) is the output.

15.5 ‘main_interrupt.v‘

This is the file which unites both: ‘spr.v‘ and ‘interrupt_controller.v‘ since it would be easier to implement a single file in already existing main file - ‘master.sv‘ and this is also where additional interrupts will be handled. This file is larger than previous 2, so we will split it into 3 parts for better clarity:

```
module main_interrupt (
    input wire [31:0] instruction,
    input wire [31:0] ea,
    input wire clk,
    input wire rst,  // 1 bit 'reset' signal
    input wire [15:0] ca_part_1,
    input wire is_illegal,
    input wire ovfalu,
    input wire [31:0] pc,
    input wire e,
    // input wire [31:0] mode_in, // Only for testing
    input wire [31:0] next_pc,
    input wire [31:0] data_in,
    input wire [2:0] reg_sel,
    output wire [31:0] spr_out,
    output wire [22:0] mca,
    output wire jisr,
    output reg abort = 1'b0,
    output wire [31:0] epc,
    output wire [31:0] mode
);

    // Internal wires for connecting modules
    wire [31:0] il;
    wire second_part_of_ill;
    wire misaf;
    wire misals;
    wire sysc;
    wire [31:0] sr;

    // Internal wires with assigned values
    wire ls = (instruction[31:26] == 6'b100011) || (instruction[31:26] ==
       6'b101011);
    wire eret = (instruction[31:26] == 6'b010000) && (instruction[5:0] ==
       6'b011000);
    wire [22:0] ca = {ca_part_1, misaf, 1'b0, (is_illegal |
       second_part_of_ill), misals, 1'b0, sysc, ovfalu};
    wire movg2s = (instruction[31:26] == 6'b010000) && (instruction[25:21]
        == 5'b00100);

    // Instantiate updated interrupt_controller module
    interrupt_controller ic_inst (
        .ca(ca),
        .sr(sr),
        .mca(mca),
        .jisr(jisr),
        .il(il)
```

```
44      );
```

First part is the I/O of the module, the declaration of internal wires and the instantiation of ‘interrupt_controller.v‘. Every wire/register name and declaration speaks for itself, except, possibly, ‘ca_part_1‘. The first 16 interrupts are reset and 15 external interrupts, and these 16 signals should be passed down from the topmost module, in our case, ‘master.sv‘. As such, we get these 16 bits as input and concatenate it with the remaining 7 bits we calculate here to get all signals of *ca*. One would notice we have hardcoded 2 zeros in 17th and 20th indices (0-indexed), as they correspond to pff and $pfls$, which are not implemented yet. Note also the addition of ‘abort‘ as an output parameter, which will be used in ‘master.sv‘ to "abort" the machine.

The "catching mechanism" for interrupts is described in [5] as follows:

$$ill(d) \equiv undefined(d) \vee mode(d) \wedge (move(d) \vee eret(d))$$

$$misaf(d) \equiv d.pc[1:0] \neq 00$$

$$misals(d) \equiv ls(d) \wedge ea(d)[1:0] \neq 00$$

$$sysc(d) \equiv opc(d) = 0^6 \wedge fun(d) = 001100$$

These interrupts are handled in the second part of ‘main_interrupt.v‘:

```
1  // Logic for illegal instruction in user mode
2  assign second_part_of_ill = mode == 32'b1 && (instruction[31:26] == 6'
       b010000);  // replace 'mode' with 'mode_in' for testing
3
4  // Misaligned fetch in execute phase
5  assign misaf = e == 1 && pc[1:0] != 2'b00;
6
7  // Misaligned load or store
8  assign misals = ls && ea[1:0] != 2'b00;
9
10 // Logic for system call
11 assign sysc = instruction[31:26] == 6'b000000 && instruction[5:0] == 6'
       b001100;
```

- First part of *ill*, the undefined instruction was handled by changing *Type* function in ‘i_decoder.v‘. But the second part restricts moving special purpose registers to general purpose registers and vice versa, if user mode is active, as well as exception return. $move(d) \vee eret(d)$ is the same as saying $opcode == 010000$.

- Misaligned fetch $misaf$ basically says that we can not execute instruction if value of program counter is not divisible by four, as it should be aligned to word boundaries.

- *misals* restricts last 2 bits to differ from 0 if we have load or store instruction. Wire is evaluated by previously defined *ls* wire and last 2 bits of effective address.
- *sysc* should be handled by operating system, so we simply output "System Call" in the ModelSim's Transcript.

The last part of this module:

```
always @(posedge clk) begin
    if (second_part_of_ill) begin
        $display("Trying to use unauthorized instructions");
        abort = 1'b1;
        // $stop;
    end

    if (misaf) begin
        $display("Misaligned fetch");
        abort = 1'b1;
        // $stop;
    end

    if (misals) begin
        $display("Misaligned load/store");
        abort = 1'b1;
        // $stop;
    end

    if (sysc) begin
        $display("System Call");
    end

    if (rst) begin  // 'rst' is on initially, so avoid raising other
       interrupts
        abort = 1'b0; // If reset is supposed to happen, it will
           happen thanks to the original code
    end

end

wire [4:0] rt = instruction[20:16];
wire [4:0] rd = instruction[15:11];
// By definition of repeat signal:
wire rpt = (il[15:1] != 0) || il[17] || il[20];

spr spr_inst (
    .clk(clk),
    .jisr(jisr),
    .eret(eret),
    .mca(mca),
    .rpt(rpt),
    .pc(pc),
    .next_pc(next_pc),
    .ea(ea),
    .data_in(data_in),
    .reg_sel(reg_sel),
    .sprw(movg2s),
    .spr_out(spr_out),
    .sr(sr),
    .epc(epc),
    .mode(mode)  // Switch to 'mode_in' for testing
);
```

```
endmodule
```

For testing only, I commented out every $abort = 1'b1$ and $\$stop$ for every "abort" interrupt, to get one nice message for this testbench, which will check if the interrupts that should get detected are indeed detected:

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    ea = 32'b0;
    pc = {30'b0, 2'b11};
    e = 1'b0;
    next_pc = 32'b0;
    data_in = 32'b0;
    reg_sel = 3'b0;

    // Test Case 1 (ill part 2):
    instruction = {6'b010000, 26'b0};
    mode_in = {31'b0, 1'b1};
    #10;

    // Test Case 2 (misaf):
    instruction = 32'b0; // Avoiding previous case
    e = 1'b1;
    pc[1:0] = 2'b01;
    #10

    // Test Case 3 (misals part 1):
    e = 1'b0; // Avoiding previous case
    instruction = {6'b100011, 24'b0, 2'b01};
    ea = {30'b0, 2'b11};
    #10

    // Test Case 4 (misals part 2):
    instruction = {6'b101011, 24'b0, 2'b01};
    ea = {30'b0, 2'b10};
    #10

    // Test Case 5 (sysc):
    instruction = {6'b000000, 20'b0, 6'b001100};
    #10
    $stop;
end
```

```
VSIM 10> run
# Trying to use unauthorized instructions
# Misaligned fetch
# Misaligned load/store
# Misaligned load/store
# System Call
```

Figure 47: Output of above testbench in ModelSim

15.6 Updated ‘master.sv‘ and ‘memory_master.v‘

In the ‘memory_master.v‘, input *jisr* was added, since updated computation of execute bit is clocked with *jisr* instead of only *reset*.

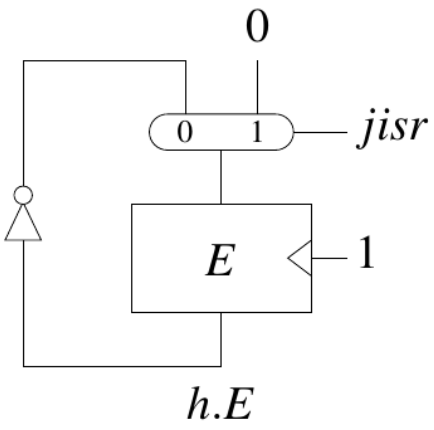


Figure 48: Computation of the execute bit with interrupts

```
always @(posedge clk or posedge jisr) begin
    if (jisr)  // 'rst' replaced with 'jisr'
        E <= 0;
    else
        E <= ~E;
end
```

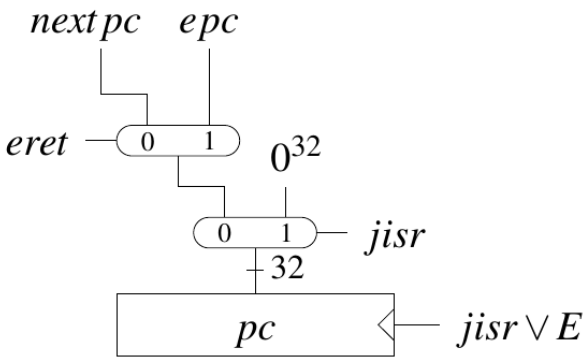


Figure 49: The new *pc* environment supporting interrupts and return from exception

Even though the actual hardware will use ‘memory_master.v‘, the simulation will use ‘memory_master_
and here we will write instructions by hand to test if the code works. Prior to that, we start with instantiating ‘main_interrupt.v‘ in ‘master.v‘:

```
wire abort;
```

```
wire is_illegal;
wire [31:0] ea;

// Interrupts
wire [2:0] reg_sel;
wire [31:0] spr_out;
wire jisr;
wire eret = instruction[31:26] == 6'b010000 && instruction[5:0] == 6'
    b011000;
wire [31:0] temp_pc = eret ? epc : next_pc;
wire [15:0] ca_part_1 = {rst, 15'b0};
wire mode;

main_interrupt interrupts(
    .instruction(instruction),
    .ea(ea),
    .clk(clk),
    .rst(rst),
    .ca_part_1(ca_part_1),
    .is_illegal(is_illegal),
    .ovfalu(ovfalu),
    .pc(pc),
    .e(E),
    .next_pc(next_pc),
    .data_in(b_gpr),
    .reg_sel(spr_mux_sel ? rt : rd),
    .spr_out(spr_out),
    .mca(mca),
    .jisr(jisr),
    .abort(abort),
    .epc(epc),
    .mode(mode)
);
```

Now, update *next_pc* computation:

```
always @(posedge E or posedge jisr) begin // JISR or E
    if (jisr) // reset
        pc <= 32'b0;
    else if (E) // next_pc vs epc
        pc <= temp_pc;
end
```

And we must not forget to update computation of clock controller enable *ena* signal, as well as update of clock in logical manner:

```
`ifdef HARDWARE
clock_control contr(.inclk(clk_wen), .ena(ena && !abort && !is_illegal),
    .outclk(clk));
`else
assign clk = (ena && !abort && !is_illegal) ? clk_wen : 0;
`endif
```

On *abort* we stop the CPU clock (not DRAM clock), so that nothing gets updated. Recall, tests below are written in the ‘memory_master_mock.v‘, as its first instruction(hence $memory[0] = \cdots$).

Let’s start with testing part 1 of illegal instruction, where instruction is simply not defined. 0^{32} is one of those possible bit strings.

```
memory[0] = 32'b0; // 'ill' part 1
```

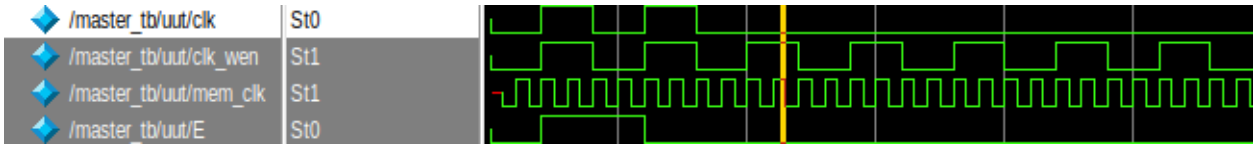


Figure 50: First part of *ill* waveform

As we can see from the waveform above, clock's signal was off after the instruction was executed. In the output below, since memory is initialized to all zeros by default, but the reset is on, nothing happens; After reset is turned off(so program should start executing) and illegal instruction gets detected, clock becomes inactive and our goals is reached.

```
VSIM 134> run
# ill: Undefined Instruction
# Time=0 rst=0 E=0 pc=00000000 debug_display=00000000
# Time=10 rst=1 E=0 pc=00000000 debug_display=00000000
# Time=20 rst=1 E=1 pc=00000000 debug_display=00000000
# Time=30 PC=00000000 Instruction=00000000
# Time=30 rst=0 E=1 pc=00000000 debug_display=00000000
# Time=40 PC=00000000 Instruction=00000000
# Time=50 PC=00000000 Instruction=00000000
# Time=60 PC=00000000 Instruction=00000000
# Time=60 rst=0 E=0 pc=zzzzzzzz debug_display=00000000
# ill: Undefined Instruction
```

Figure 51: First part of *ill* output

Since we are initially in user mode, if our first instruction is $move(d) \vee eret(d)$, abort should be raised. We will test with $eret(d)$:

```
memory[0] = {6'b010000, 5'b10000, 15'b0, 6'b01100}; // 'ill' part 2
```



Figure 52: Second part of *ill* waveform

Indeed that is what happens, and after display message that gets printed in the same cycle, the program stops.

```
# Trying to use unauthorized instructions
# Time=30 rst=0 E=0 pc=00000000 debug_display=00000000
run
VSIM 625> run -over
```

Figure 53: Second part of *ill* output

Next, we test *malf*. The only case, when $4 \nmid pc$ is when we use jump instruction and the jump distance is not divisible by 4. We will store integer $k \nmid 4$ in the first register and later we jump with that distance. Following MIPS code:

```
addi $1 $0 3
jr $1
```

can be written in ‘memory_master_mock.v‘ as:

```
memory[0] = {6'b001000, 5'b00000, 5'b00001, {14'b0, 2'b11}}; // 'malf' first instruction
memory[1] = {6'b000000, 5'b00001, 15'b0, 6'b001000};  // 'malf' second instruction
```

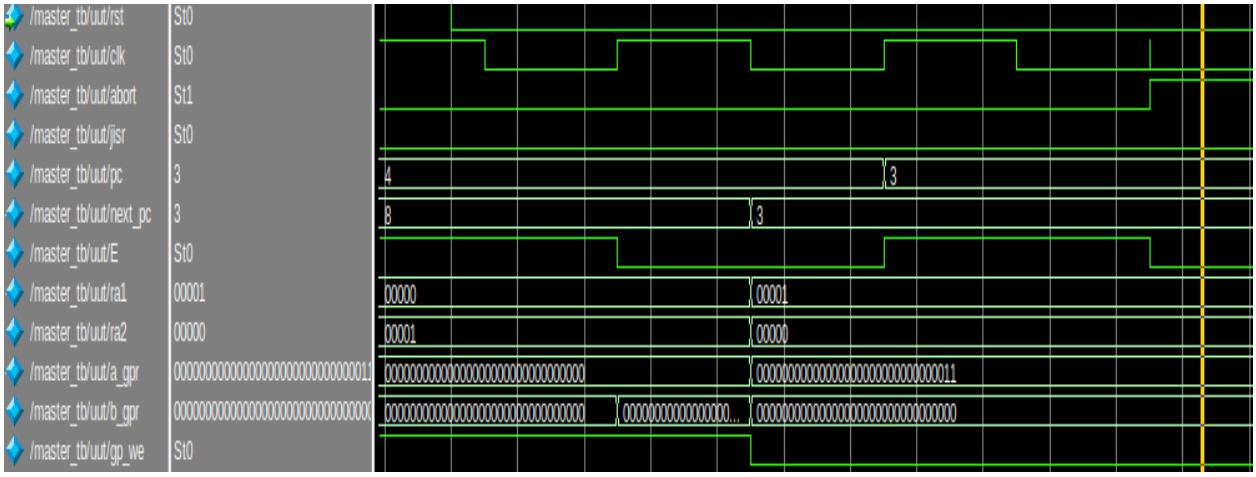


Figure 54: *malf* waveform

Since we have 2 instructions in this test case, we need to hit “Run” button twice in ModelSim to see the results.

On the waveform above, we can see that after *pc* becomes $3 \nmid 4$, *abort* is raised. We can also see the output displayed in ModelSim’s Transcript window:

```
# Time=0 rst=1 E=0 pc=00000000 debug_display=00000000
# Time=15 rst=1 E=1 pc=00000004 debug_display=00000000
# Time=25 PC=00000004 Instruction=20010003
# Time=30 rst=0 E=1 pc=00000004 debug_display=00000000
# Time=35 PC=00000004 Instruction=20010003
# Time=45 PC=00000004 Instruction=20010003
# Time=55 PC=00000004 Instruction=20010003
# Time=55 rst=0 E=0 pc=00000004 debug_display=00000000
# Time=95 rst=0 E=1 pc=00000003 debug_display=00000000
VSIM 154> run
# Time=105 PC=00000003 Instruction=00200008
# Time=115 PC=00000003 Instruction=00200008
# Time=125 PC=00000003 Instruction=00200008
# Time=135 PC=00000003 Instruction=00200008
# Misaligned fetch
# Time=135 rst=0 E=0 pc=00000003 debug_display=00000000
```

Figure 55: *malf* output

Moving on, we test *mals*, by simply loading from effective address of $0^{15}1$ and since last 2 bits of this address are not both zero, we should get abort signal raised:

```
memory[0] = {6'b101011, 5'b00000, 5'b00001, {15'b0, 1'b1}};
```



Figure 56: *misals* waveform

We can confirm our predictions from waveform above and display output below.

```
# Misaligned load/store
# Time=15 rst=1 E=1 pc=00000000 debug_display=00000000
# Time=25 PC=00000000 Instruction=ac010001
```

Figure 57: *misals* output

Finally, we test *syscall* by simply making first instruction "syscall":

```
# System Call
# Time=15 rst=1 E=1 pc=00000000 debug_display=00000000
# Time=25 PC=00000000 Instruction=0000000c
```

Figure 58: *syscall* output

15.6.0 Test for move instruction

At last, we test support for move instructions. This will be done by this 3 instructions (with $\neg mode$):

```
addiu $11 $0 7
movg2s $4 $11
movs2g $4 $11
```

The first instruction will save the number 7 into the 11th gpr, the second will move it to the 4th spr, and the third will bring it back.

In ‘memory_master_mock.sv‘:

```
memory[0] = 32'b00100100000010110000000000000111;
memory[1] = 32'b01000000100010110010000000000000;
memory[2] = 32'b01000000000010110010000000000000;
```

and test it in ModelSim to get:

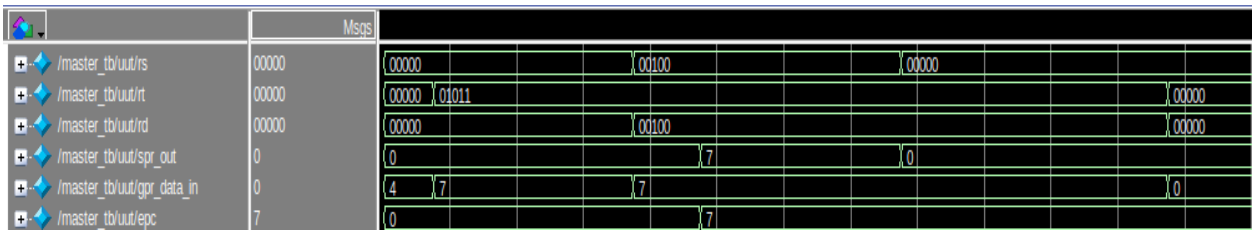


Figure 59: Test for move instructions

Above, we can see that we placed number 7 in the gpr (*gpr_data_in*). In the next instruction, it appears in 4th spr (*epc*). Afterwards, it is again input to gpr file (*gpr_data_in*).

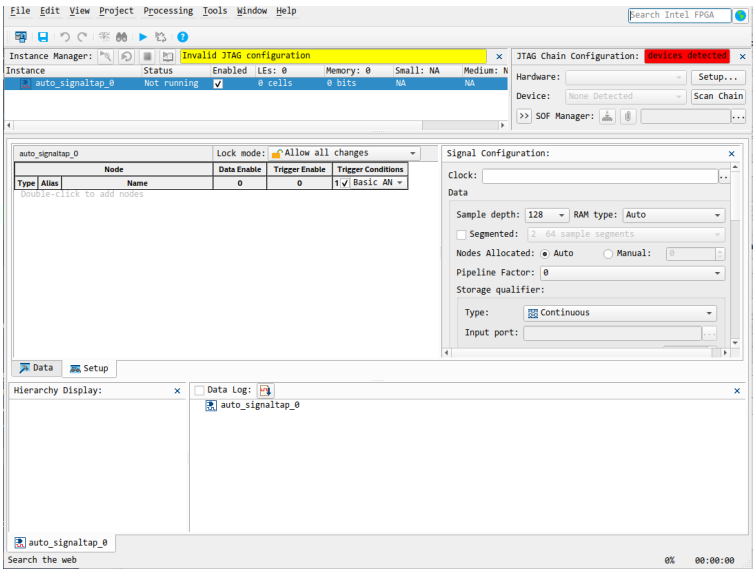
16 Further testing of the design

We further present the results of testing our implementation. We showcase the design tested on the multiplication algorithm from [5], as well as on the division algorithm from [5]. We provide both simulation results and results from the board, displayed on the Hex Display, as well as the results from the Signal Tap Analyzer.

16.1 A little introduction to Signal Tap Logic Analyzer

Signal Tap logic analyzer is a tool Quartus provides for debugging hardware. It allows us to obtain values of wires/registers on hardware at runtime - much like how we read the values of wires over time in ModelSim.

To do so, make sure the design is compiled. Then, go to Tools → Signal Tap Logic Analyzer.



You will see an (mostly) empty window. First, we need to give the Signal Tap a clock - click the ...next to Clock under "Signal Configuration". Hit "List" and find the wire you use as a clock. Some user-defined wires don't appear here, and for a good reason - after compilation, quartus removes many wires which it deems unnecessary through optimization. **To avoid quartus removing a wire used for debugging or as a signal tap clock, write a comment /* synthesis keep */ after the wire declaration.**

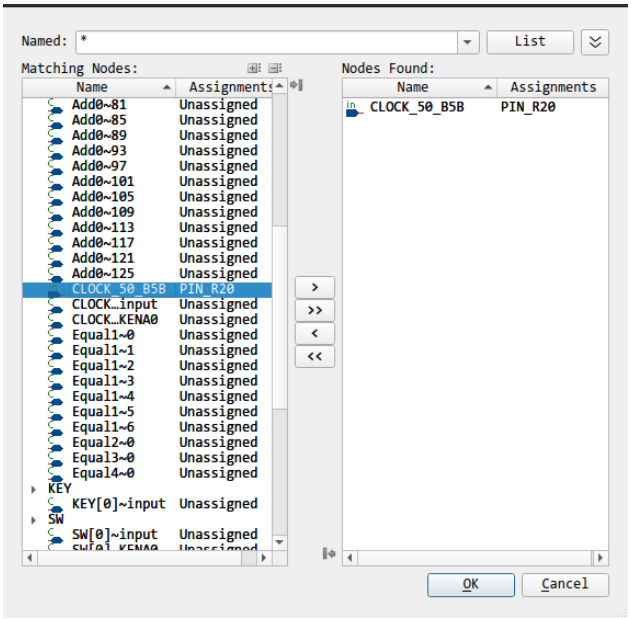


Figure 60: Clock selection UI. We have selected a 50MHz clock for the device. This affects the frequency of data fetch from the device.

As for the data, once you have selected the clock, you may double click where the UI says "Double click to add Nodes". This'll give you a node selection, same as the clock. I'd advise you click the arrow dropdown on the top right, and then select "Filter: Design Entry (all names)". This'll list all wires in the code in a much clearer manner but beware - if Quartus may remove wires in this list unless the comment mentioned above is present (if the wires are red after recompiliation, this is the case).

Node			Data Enable	Trigger Enable
Type	Alias	Name	1462	1462
		master:mstr|memory_master_mock:memory|mem_clk	☑	☑
		master:mstr|memory_master_mock:memory|mem_wren	☑	☑
		master:mstr|memory_master_mock:memory|rst	☑	☑
		⊞ master:mstr|pc[31..0]	☑	☑
		⊞ master:mstr|next_pc[31..0]	☑	☑
		⊞ master:mstr|instruction_reg[31..0]	☑	☑
		master:mstr|memory_master:memory|sram_wren	☑	☑
		master:mstr|memory_master:memory|write_req	☑	☑
		⊞ master:mstr|memory_master:memory|addr_in[29..0]	☑	☑
		⊞ master:mstr|memory_master:memory|address[26..0]	☑	☑
		⊞ master:mstr|memory_master:memory|data_in[31..0]	☑	☑
		⊞ master:mstr|memory_master:memory|out[31..0]	☑	☑

Figure 61: Signal tap nodes in use. As you can see, some wires are red - Quartus has optimized them, removed them. Adding /* synthesis keep */ would fix that.

16.2 Multiplication Test

Below is the multiplication code written in MARS assembler:

```
addi $6, $0, 5          # Set $6 to 5
addi $7, $0, 7          # Set $7 to 7
addi $24, $0, 1         # Set $24 to 1
addi $26, $7, 0         # Copy value of $7 to $26
addi $27, $0, 0         # Set $27 to 0
and $25, $6, $24        # Perform bitwise AND between $6 and $24, store in
    $25
beq $25, $0, 1          # If $25 equals 0, branch to next instruction (skip
    next line)
add $27, $27, $26       # Add $26 to $27 (this is executed only if the
    previous branch is not taken)
sll $24, $24, 1         # Shift $24 left by 1
sll $26, $26, 1         # Shift $26 left by 1
bne $24, $0, -6         # If $24 is not equal to 0, branch back 6
    instructions (loop)
addi $5, $27, 0         # Copy value of $27 to $5
```

Below is the .mif file for the above code. For an explanation of the .mif, refer to the chapter about SRAM.

```
DEPTH = 12;
WIDTH = 32;
ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;
CONTENT
BEGIN
    00000000 : 20060005;
    00000001 : 20070007;
    00000002 : 20180001;
    00000003 : 20fa0000;
    00000004 : 201b0000;
    00000005 : 00d8c824;
    00000006 : 13200001;
    00000007 : 037ad820;
    00000008 : 0018c040;
    00000009 : 001ad040;
    0000000a : 1700fffa;
    0000000b : 23650000;
END;
```

Below is the simulation test result, for this we loaded in mock memory instructions (based on .mif file):

```
    memory[0] = 32'h20060005; // addi $6, $0, 5
    memory[1] = 32'h20070007; // addi $7, $0, 7
    memory[2] = 32'h20180001; // addi $24, $0, 1
    memory[3] = 32'h20fa0000; // addi $26, $7, 0
    memory[4] = 32'h201b0000; // addi $27, $0, 0
    memory[5] = 32'h00d8c824; // and $25, $6, $24
    memory[6] = 32'h13200001; // beq $25, $0, 1
    memory[7] = 32'h037ad820; // add $27, $27, $26
    memory[8] = 32'h0018c040; // sll $24, $24, 1
    memory[9] = 32'h001ad040; // sll $26, $26, 1
    memory[10] = 32'h1700fffa; // bne $24, $0, -6
    memory[11] = 32'h23650000; // addi $5, $27, 0
```

Notice that in [5], we have add $24, $24, $24 (doubling the content of register 24). This relies

on the fact that in the MIPS semantics from [5], we use modulo computation. However, the MARS assembler raises an overflow error and does not continue the computation modulo (which is not the case for sll (shift left logical)). For this reason, instead of using add, we use sll, which is implemented in our Shifter.v file.

Below are the results of the simulation:

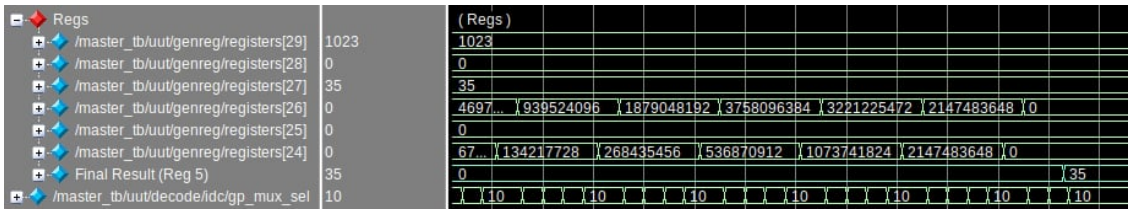


Figure 62: Result of the multiplication algorithm (multiplying 5 and 7)

Below is the result of multiplication displayed on Hex Display:

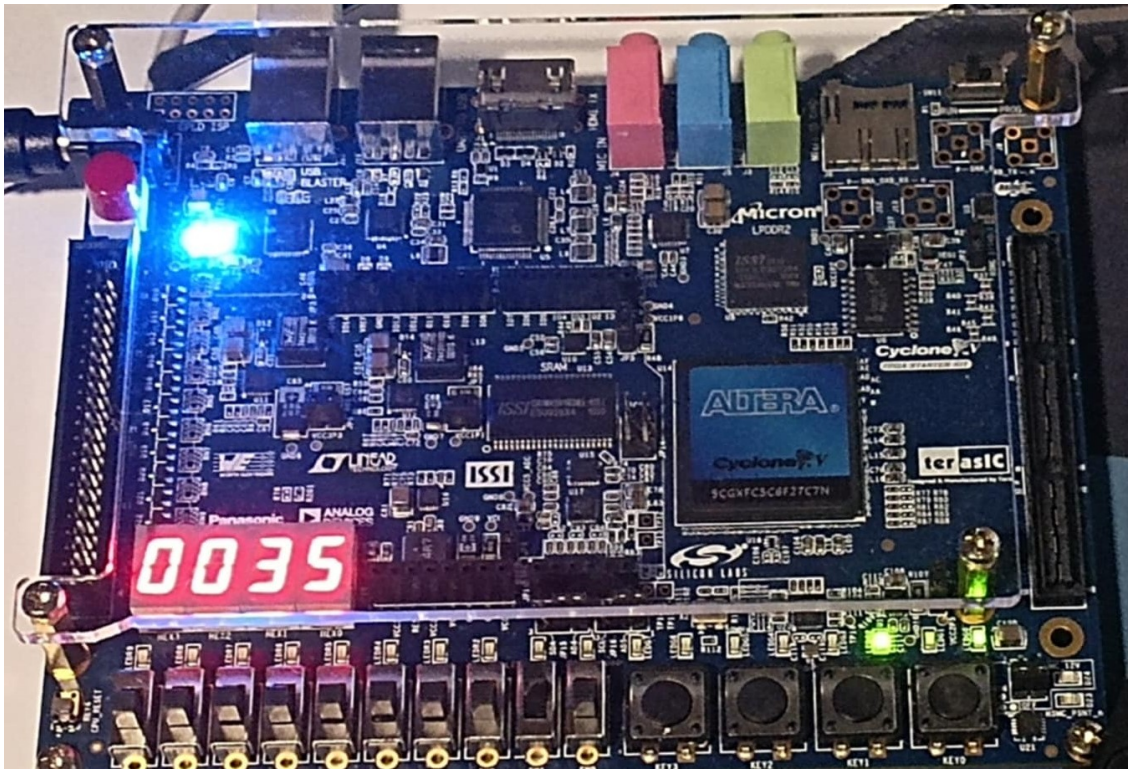
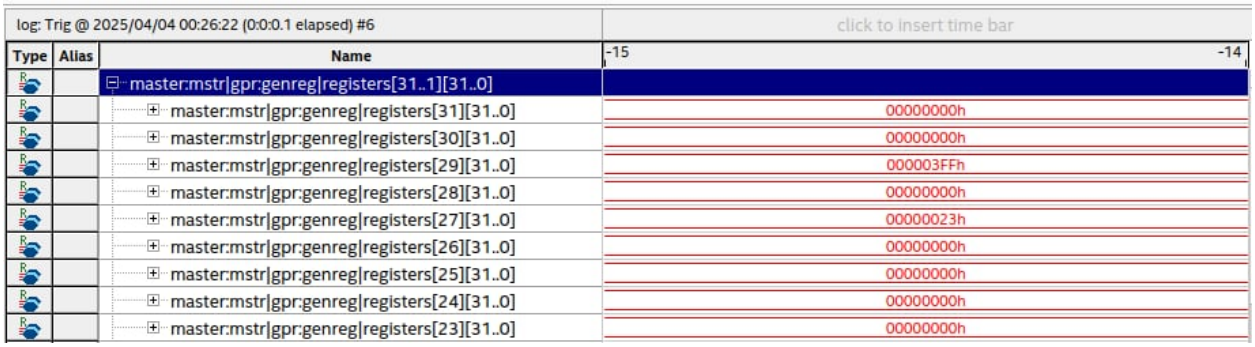


Figure 63: Result of the multiplication algorithm (multiplying 5 and 7)

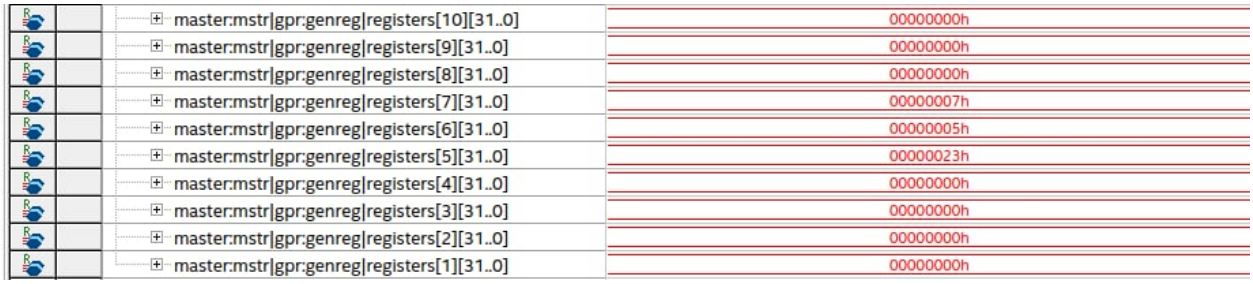
For more detailed monitoring of register values, we use Signal Tap, below is the register values at the end of the computation:



log: Trig @ 2025/04/04 00:26:22 (0:0:0.1 elapsed) #6 — click to insert time bar

Type	Alias	Name	-15 -14
		master:mstr|gpr:genreg|registers[31..1][31..0]	
		master:mstr|gpr:genreg|registers[31][31..0]	00000000h
		master:mstr|gpr:genreg|registers[30][31..0]	00000000h
		master:mstr|gpr:genreg|registers[29][31..0]	000003FFh
		master:mstr|gpr:genreg|registers[28][31..0]	00000000h
		master:mstr|gpr:genreg|registers[27][31..0]	00000023h
		master:mstr|gpr:genreg|registers[26][31..0]	00000000h
		master:mstr|gpr:genreg|registers[25][31..0]	00000000h
		master:mstr|gpr:genreg|registers[24][31..0]	00000000h
		master:mstr|gpr:genreg|registers[23][31..0]	00000000h

Figure 64: Final Content of Registers



		master:mstr|gpr:genreg|registers[10][31..0]	00000000h
		master:mstr|gpr:genreg|registers[9][31..0]	00000000h
		master:mstr|gpr:genreg|registers[8][31..0]	00000000h
		master:mstr|gpr:genreg|registers[7][31..0]	00000007h
		master:mstr|gpr:genreg|registers[6][31..0]	00000005h
		master:mstr|gpr:genreg|registers[5][31..0]	00000023h
		master:mstr|gpr:genreg|registers[4][31..0]	00000000h
		master:mstr|gpr:genreg|registers[3][31..0]	00000000h
		master:mstr|gpr:genreg|registers[2][31..0]	00000000h
		master:mstr|gpr:genreg|registers[1][31..0]	00000000h

Figure 65: Final Content of Registers

16.3 Division Test

Below, we present the code for the division algorithm:

```
.text
.macro sign_and_abs $J, $k, $i
    slti $J, $i, 0
    blez $J, escape_neg
    nor $k, $k, $k
    xor $k, $k, $i
    addiu $k, $k, 1
    blez $zero, escape
escape_neg:
    addiu $k, $i, 0
escape:
.end_macro
main:
    addi $8, $zero, 100
    addi $9, $zero, 5

    lui $19, 0x8000
    ori $19, $19, 0x0000
    bne $19, $8, skip_edge
    nor $20, $zero, $zero
    bne $20, $9, skip_edge
    add $5, $8, 0
    beq $zero, $zero, end2

skip_edge:
    sign_and_abs $21, $10, $8
    sign_and_abs $22, $11, $9
    xor $20, $21, $22
    add $5, $zero, $zero
```

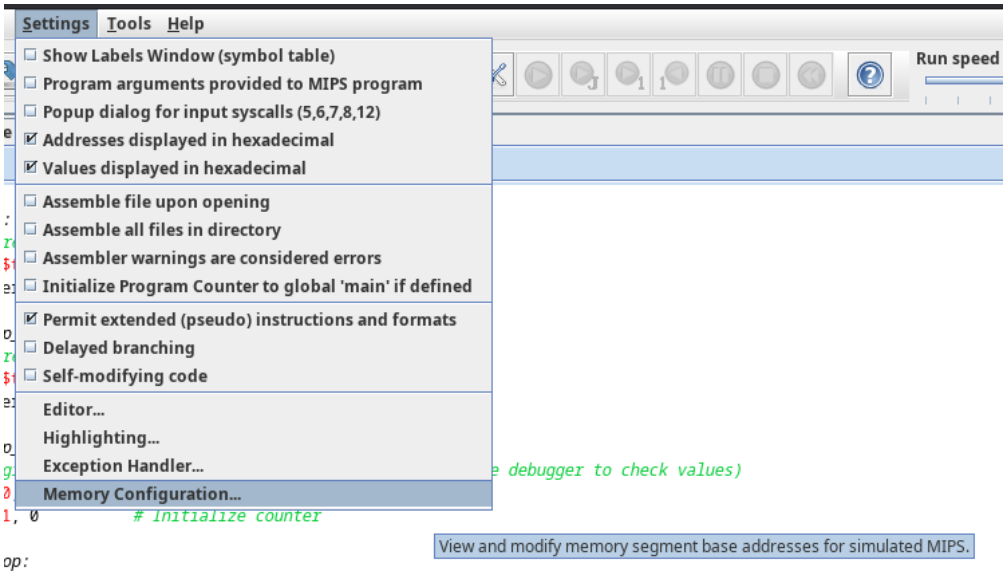
```
30      sltu $23, $10, $11
31      bgtz $23, end
32      add $23, $10, 0
33      add $24, $11, 0
34      addi $25, $zero, 1
35      add $26, $zero, $zero
36
37  Largest_B:
38      and $22, $24, $19
39      bnez $22, Skip_double
40      addu $25, $25, $25
41      addu $24, $24, $24
42      sltu $27, $23, $24
43      blez $27, Largest_B
44
45  Skip_double:
46      addiu $24, $24, -1
47      sltu $22, $24, $23
48      addiu $24, $24, 1
49      blez $22, Jump_To_Exit
50      or $26, $26, $25
51      subu $23, $23, $24
52
53  Jump_To_Exit:
54      sltu $27, $23, $24
55      blez $27, end
56
57  Step_down_B:
58      srl $25, $25, 1
59      srl $24, $24, 1
60      sltu $27, $23, $24
61      bgtz $27, Step_down_B
62      or $26, $26, $25
63      sub $23, $23, $24
64      sltu $27, $23, $24
65      bgtz $27, Step_down_B
66
67  end:
68      addi $5, $26, 0
69
70      blez $20, end2
71      nor $22, $zero, $zero
72      xor $5, $5, $22
73      addiu $5, $5, 1
74
75  end2:
```

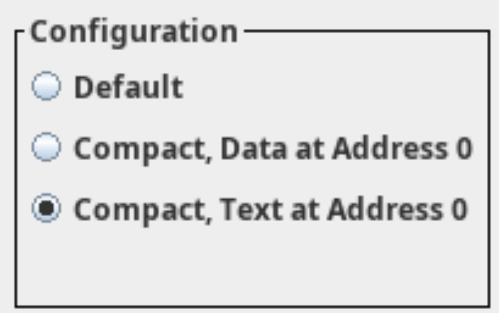
Below, is the .mif file for above code:

```
1  DEPTH = 25;
2  WIDTH = 32;
3  ADDRESS_RADIX = HEX;
4  DATA_RADIX = HEX;
5  CONTENT
6  BEGIN
7       00000000 : 24040018;
8       00000001 : 24050005;
9       00000002 : 0c000004;
10      00000003 : 08000019;
11      00000004 : 10a00011;
12      00000005 : 24020000;
13      00000006 : 00041821;
14      00000007 : 24080000;
15      00000008 : 04810002;
16      00000009 : 00042022;
```

```
    0000000a : 24080001;
    0000000b : 04a10002;
    0000000c : 00052822;
    0000000d : 39080001;
    0000000e : 0065082a;
    0000000f : 14200003;
    00000010 : 00651822;
    00000011 : 20420001;
    00000012 : 0800000e;
    00000013 : 11000001;
    00000014 : 00021022;
    00000015 : 03e00008;
    00000016 : 2402ffff;
    00000017 : 2403ffff;
    00000018 : 03e00008;
END;
```

There is a minor detail to be paid attention to - The code above was compiled using MARS assembler. MARS, by default configuration, doesn't start execution at the start (address 0). To match the configuration on our board (For jump instructions) we can manually change it in Settings → Memory Configuration. We need to set it to "Compact, Text at Address 0".





Below, we present waveforms for the division. Similarly, we initialize the mock memory with the above .mif file and simulate. The final output should be 20 (result) in register 5.

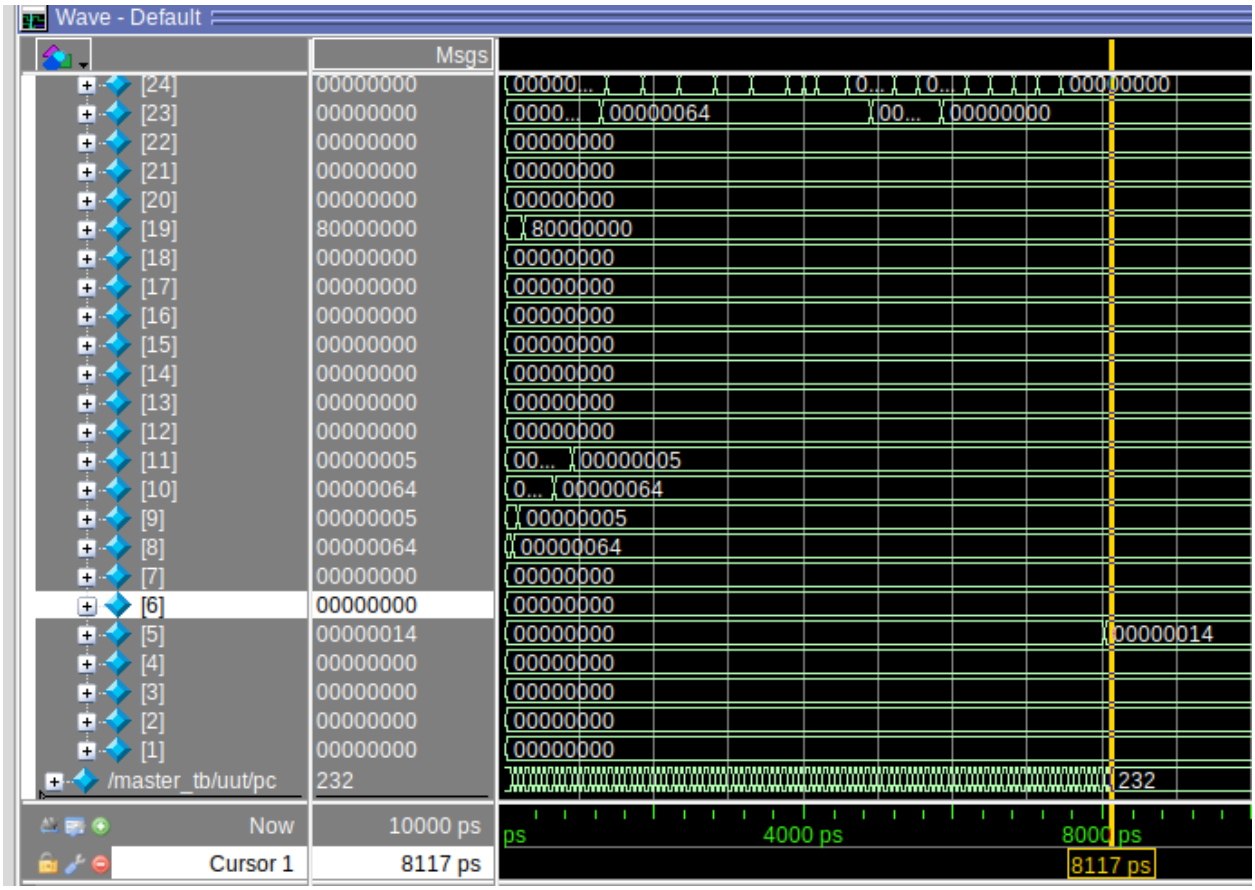


Figure 66: Computation of Division we divide 100 by 5

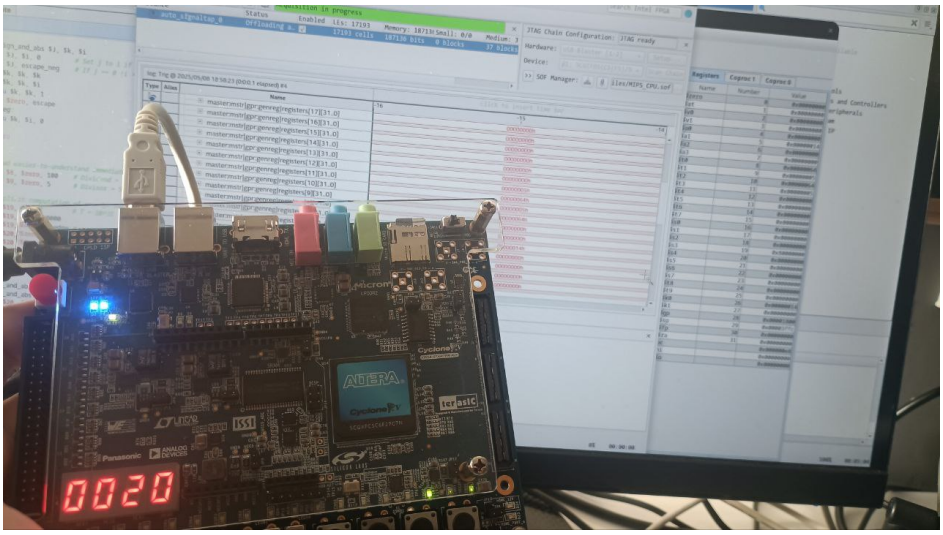


Figure 68: Result shown output on HEX display (Which has the register 5 mapped to it). Keep in mind that MARS and Signal Tap show results in Hexadecimal, hardware shows in decimal.

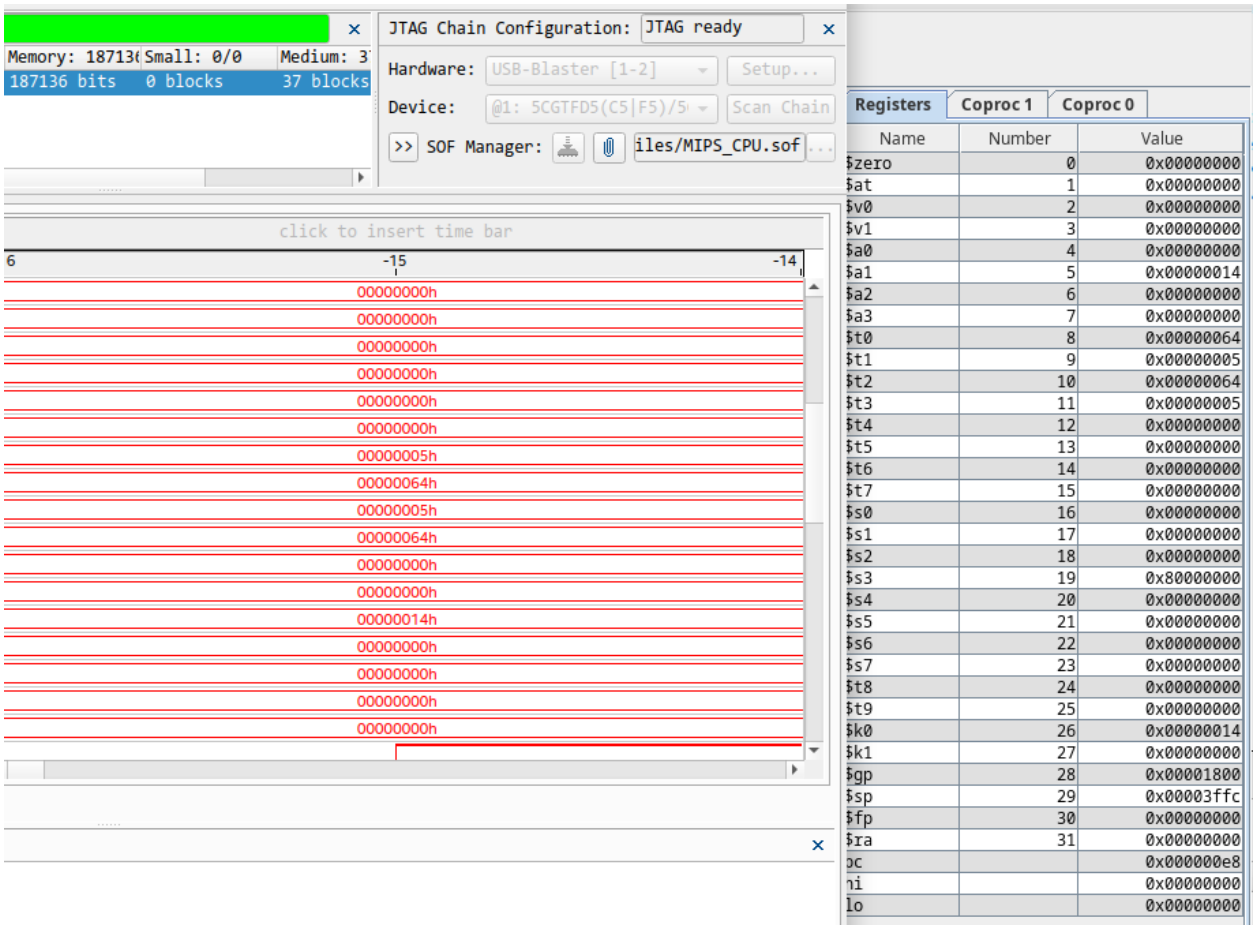


Figure 67: Computation of Division we divide 100 by 5, results in register 5. Left - Signal Tap (registers sorted descending, from 31 to 0), Right - MARS Simulator (registers ascending)

16.4 Bubble Sort Test

Given below is the code for the Bubble Sort in mars assembler:

```
# Bubble Sort Implementation in MIPS
# This program tests:
# - Register operations
# - Memory access
# - Branching and looping
# - Array manipulation
# - Conditional execution

.data
array:    .word 64, 34, 25, 12, 22, 11, 90, 45, 33, 21, 65, 89, 44, 1, 99,
    13
array_end:
length:   .word 16

.text
.globl main

main:
    la $t0, array
    lw $t1, length
    li $t2, 0

outer_loop:
    addi $t3, $t1, -1
    slt $t4, $t2, $t3
    beq $t4, $zero, outer_loop_end

    li $t3, 0

inner_loop:
    sub $t4, $t1, $t2
    addi $t4, $t4, -1
    slt $t5, $t3, $t4
    beq $t5, $zero, inner_loop_end

    sll $t5, $t3, 2
    add $t5, $t5, $t0

    lw $t6, 0($t5)
    lw $t7, 4($t5)

    slt $t8, $t7, $t6
    beq $t8, $zero, skip_swap

    sw $t7, 0($t5)
    sw $t6, 4($t5)

skip_swap:
    addi $t3, $t3, 1
    j inner_loop

inner_loop_end:
    addi $t2, $t2, 1
    j outer_loop

outer_loop_end:
    la $t0, array
    li $t1, 0

verify_loop:
```

```
60      lw $t2, length
61      slt $t3, $t1, $t2
62      beq $t3, $zero, done
63
64      sll $t4, $t1, 2
65      add $t4, $t4, $t0
66      lw $t5, 0($t4)
67
68      addi $t1, $t1, 1
69      j verify_loop
70
71  done:
```

Below is the result of sorted data stored in registers.

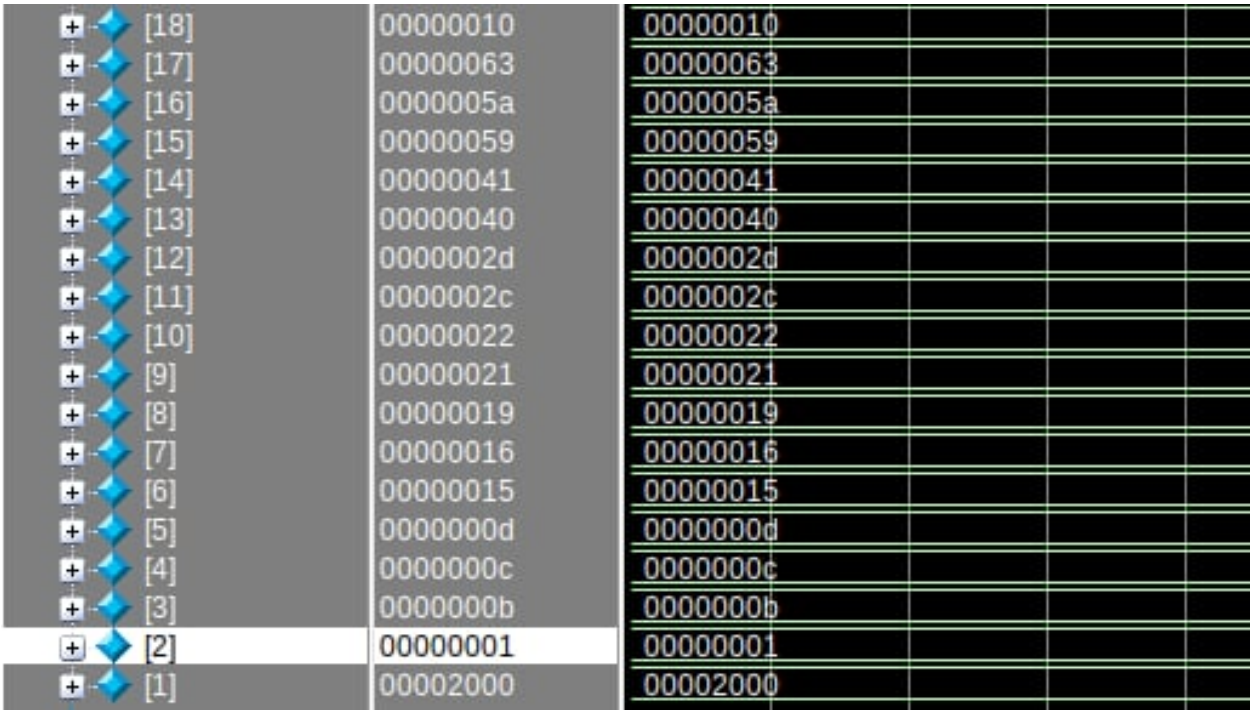


Figure 69: Sorted Data in Hexadecimal format stored in registers [2-18]. initial data to be sorted is: [64, 34, 25, 12, 22, 11, 90, 45, 33, 21, 65, 89, 44, 1, 99, 13]

Below is the result of Signal Tap where we observe registers [2-18].

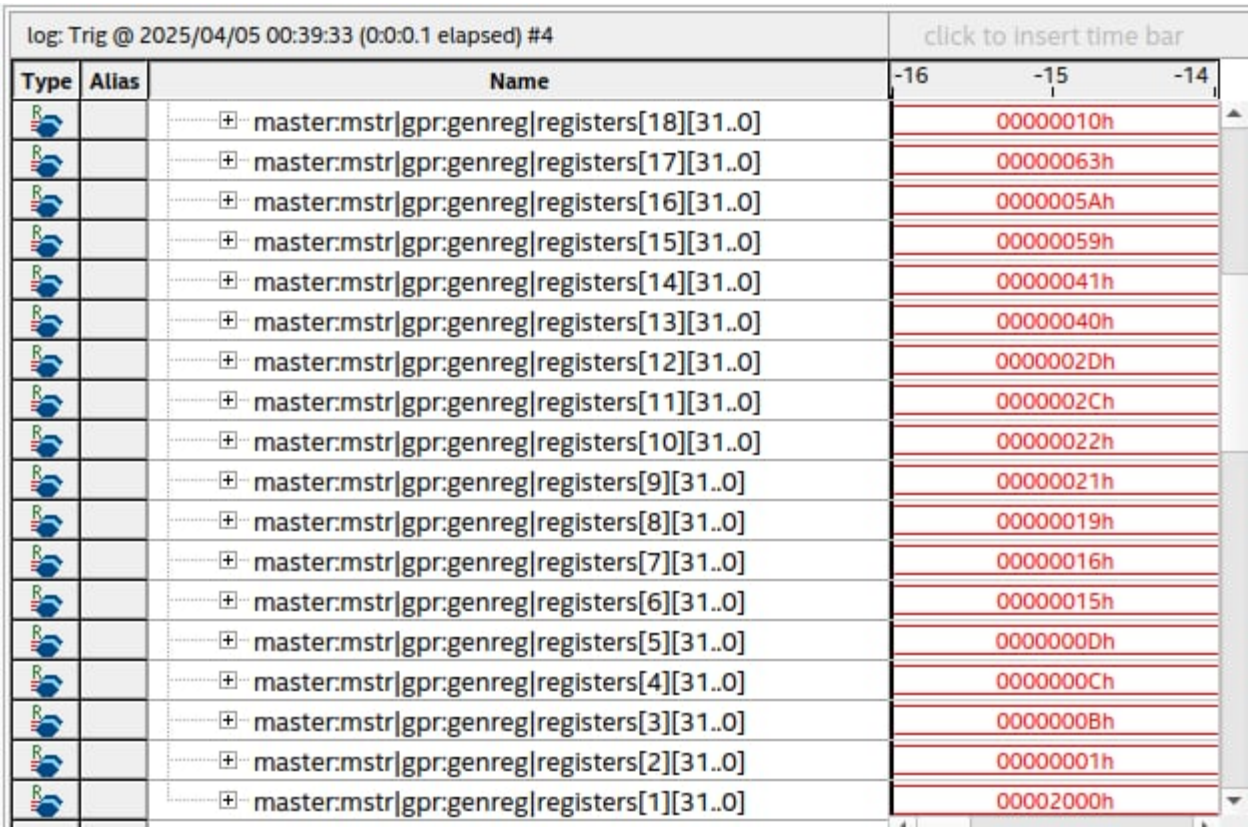


Figure 70: Results on Board observed from Signal Tap

The sorting happens in memory. However, we're unable to tap into memory with Signal Tap Logic Analyzer, hence we've added a few instructions to load the result of the sort in memory into registers at the end.

```
1     memory[0]  = 32'h20082000;  // addi $t0, $zero, 0x2000
2     memory[1]  = 32'h8c092040;  // lw $t1, 0x2040($zero)
3     memory[2]  = 32'h240a0000;  // addiu $t2, $zero, 0
4     memory[3]  = 32'h212bffff;  // addi $t3, $t1, -1
5     memory[4]  = 32'h014b602a;  // slt $t4, $t2, $t3
6     memory[5]  = 32'h11800011;  // beq $t4, $zero, 0x17
7     memory[6]  = 32'h240b0000;  // addiu $t3, $zero, 0
8     memory[7]  = 32'h012a6022;  // sub $t4, $t1, $t2
9     memory[8]  = 32'h218cffff;  // addi $t4, $t4, -1
10    memory[9]  = 32'h016c682a;  // slt $t5, $t3, $t4
11    memory[10] = 32'h11a0000a; // beq $t5, $zero, 0x15
12    memory[11] = 32'h000b6880; // sll $t5, $t3, 2
13    memory[12] = 32'h01a86820; // add $t5, $t5, $t0
14    memory[13] = 32'h8dae0000; // lw $t6, 0($t5)
15    memory[14] = 32'h8daf0004; // lw $t7, 4($t5)
16    memory[15] = 32'h01eec02a; // slt $t8, $t7, $t6
17    memory[16] = 32'h13000002; // beq $t8, $zero, 0x13
18    memory[17] = 32'hadaf0000; // sw $t7, 0($t5)
19    memory[18] = 32'hadae0004; // sw $t6, 4($t5)
20    memory[19] = 32'h216b0001; // addi $t3, $t3, 1
21    memory[20] = 32'h08000007; // j 0x7
22    memory[21] = 32'h214a0001; // addi $t2, $t2, 1
23    memory[22] = 32'h08000003; // j 0x3
24    memory[23] = 32'h20082000; // addi $t0, $zero, 0x2000
25    memory[24] = 32'h24090000; // addiu $t1, $zero, 0
26    memory[25] = 32'h8c0a2040; // lw $t2, 0x2040($zero)
```

```
    memory[26] = 32'h012a582a; // slt $t3, $t1, $t2
    memory[27] = 32'h11600005; // beq $t3, $zero, 0x21
    memory[28] = 32'h00096080; // sll $t4, $t1, 2
    memory[29] = 32'h01886020; // add $t4, $t4, $t0
    memory[30] = 32'h8d8d0000; // lw $t5, 0($t4)
    memory[31] = 32'h21290001; // addi $t1, $t1, 1
    memory[32] = 32'h08000019; // j 0x19

    // Code below loads the data in the registers.
    memory[33] = 32'h20082000;  // addi $t0, $zero, 0x2000
    memory[34] = 32'h00080825;  // or $1, $0, $t0
    memory[35] = 32'h8c220000;  // lw $2, 0($1)   - load 1st element
    memory[36] = 32'h8c230004;  // lw $3, 4($1)   - load 2nd element
    memory[37] = 32'h8c240008;  // lw $4, 8($1)   - load 3rd element
    memory[38] = 32'h8c25000c;  // lw $5, 12($1)  - load 4th element
    memory[39] = 32'h8c260010;  // lw $6, 16($1)  - load 5th element
    memory[40] = 32'h8c270014;  // lw $7, 20($1)  - load 6th element
    memory[41] = 32'h8c280018;  // lw $8, 24($1)  - load 7th element
    memory[42] = 32'h8c29001c;  // lw $9, 28($1)  - load 8th element
    memory[43] = 32'h8c2a0020;  // lw $10, 32($1) - load 9th element
    memory[44] = 32'h8c2b0024;  // lw $11, 36($1) - load 10th element
    memory[45] = 32'h8c2c0028;  // lw $12, 40($1) - load 11th element
    memory[46] = 32'h8c2d002c;  // lw $13, 44($1) - load 12th element
    memory[47] = 32'h8c2e0030;  // lw $14, 48($1) - load 13th element
    memory[48] = 32'h8c2f0034;  // lw $15, 52($1) - load 14th element
    memory[49] = 32'h8c300038;  // lw $16, 56($1) - load 15th element
    memory[50] = 32'h8c31003c;  // lw $17, 60($1) - load 16th element
    memory[51] = 32'h8c320040;  // lw $18, 64($1) - load 17th element

    // Data array starting at memory address 0x2000 (8192 decimal)
    memory[2048] = 32'h00000040;
    memory[2049] = 32'h00000022;
    memory[2050] = 32'h00000019;
    memory[2051] = 32'h0000000c;
    memory[2052] = 32'h00000016;
    memory[2053] = 32'h0000000b;
    memory[2054] = 32'h0000005a;
    memory[2055] = 32'h0000002d;
    memory[2056] = 32'h00000021;
    memory[2057] = 32'h00000015;
    memory[2058] = 32'h00000041;
    memory[2059] = 32'h00000059;
    memory[2060] = 32'h0000002c;
    memory[2061] = 32'h00000001;
    memory[2062] = 32'h00000063;
    memory[2063] = 32'h0000000d;
    memory[2064] = 32'h00000010;
    memory[2112] = 32'h00000011;
```

Code 23: Mock Memory initialization with assembly

17 Hardware Implementation Structure

17.0 Project Organization

This version of the project was written after we implemented a pipelined version of the CPU. You'll see many similarities in structure. Even here, without any pipelining, we have the project split into several parts - giving us a nice, organized structure that is easy to navigate.

17.0 The Main `hdl` Folder

```
hdl/
|-- interfaces/       # Communication between stages
|-- master.sv         # The top-level module
|-- master_tb.sv      # Testbench for the whole CPU
|-- parts/
|   |-- decode/       # Instruction decoding stuff
|   |-- execute/      # ALU and execution units
|   |-- memory/       # Memory access components
|   `-- writeback/    # Register writeback logic
`-- utility/          # Helper modules
```

17.1 Design Rationale

In this setting, Each "stage" handles one specific aspect of the CPU's operation, making the code easier to understand and debug. Another advantage is that Individual components (such as different memory implementations) can be swapped without affecting the rest of the design.

17.2 Core Components

17.2.0 Decode Components (`parts/decode/`)

This is where instructions get broken down:

- `i_decoder.v`: Interprets the instruction bits
- `bce.v`: Branch Condition Evaluation unit
- `next_pc_calc.v`: Calculates the next program counter value
- `decode_master.v`: Top-level module for the decode stage

17.2.1 Execute Components (`parts/execute/`)

This is where the actual computation happens:

- `alu.v` and `au.v`: Arithmetic Logic Unit and Arithmetic Unit
- `shifter.v`: Handles bit-shifting operations
- `execute_master.v`: Coordinates the execution stage

17.2.2 Memory System (`parts/memory/`)

We've implemented multiple memory approaches:

- `memory_master_mock.sv`: A simple register-based memory for testing
- `memory_master.sv`: The actual memory controller that arbitrates between SRAM and LPDDR2
- `lpddr2_memory.sv`: Interface to the external LPDDR2 RAM

17.2.3 General-Purpose Registers (`parts/writeback/`)

- `gpr.sv`: The general-purpose register file
- `writeback_master.sv`: Contains logic for register writing (e.g. choosing input)

17.3 Interface Communication

In `interfaces/`, we have some additional, helper modules.

- `decoder_concat.sv` and `decoder_deconcat.sv`: Pack and unpack control signals between stages

These modules are essential for passing signals between components in a clean way.

17.4 Utility Components

In `utility/`, there are helpful modules like:

- `clock_div.sv`: Divides the clock signal
- `hex_display.sv`: Drives the hex displays for debugging
- `splitter.sv`: Breaks instructions into their component fields

17.5 Memory Options

One notable feature of our design is the flexible memory system. The `memory_master.sv` module switches between:

- On-chip SRAM (the first 2^{12} bytes = 4 KB)
- External LPDDR2 RAM (for addresses beyond 4 KB)

This provides fast access for boot code in SRAM and extensive storage in LPDDR2.

17.6 Sequential vs. Pipelined Operation

Despite resembling a pipelined design, our CPU operates sequentially using an E-bit that toggles between fetch and execute:

- When E=0, we fetch an instruction from memory
- When E=1, we execute the fetched instruction

This two-cycle-per-instruction approach significantly simplifies the design compared to a full pipeline, while still maintaining good organization.

The entire execution is coordinated by the `master.sv` file, which integrates all components and manages the timing of fetch and execute cycles.

18 Conclusion

In this document, we aimed to explain the ideas behind the final design of the CPU. We detailed all the modules used in the Verilog implementation and highlighted the advantages of a hierarchical and parameterized design.

Many aspects of the design can be modified, making it a valuable exercise for students (e.g., changing adders, adding multipliers, etc.). We also provide waveforms after each important section to showcase the modules in action. Some modules do not have hardware counterparts; they were included to make the code cleaner.

Based on our work, other students can build upon it to develop Memory Management Units, an OS kernel, and an entire system, which are currently being explored in other theses. We have put significant effort into testing; in addition to providing waveforms after each chapter, we also

include a separate chapter dedicated to further testing. Many parts of this work can be incorporated into hardware labs for undergraduate students.

The entire design is available on [3] for further details.

19 Appendix - TestBench Codes

```
module adder_tb;

// Parameters
parameter n = 16;

// Inputs
reg [n-1:0] a;
reg [n-1:0] b;
reg cin;

// Outputs
wire [n-1:0] s;
wire cout;

// Instantiate the Unit Under Test (UUT)
adder #(n) uut (
.a(a),
.b(b),
.cin(cin),
.s(s),
.cout(cout)
);

initial begin
// Initialize Inputs
a = 0;
b = 0;
cin = 0;

#10;

// Test cases
a = 16'b0000000000000001; b = 16'b0000000000000001; cin = 1'b0;
#10; // Wait for 10 ns

a = 16'b1111111111111111; b = 16'b0000000000000001; cin = 1'b0;
#10;

a = 16'b0000111111111111; b = 16'b0001000000000000; cin = 1'b1;
#10;

a = 16'b1010101010101010; b = 16'b0101010101010101; cin = 1'b0;
#10;

a = 16'b1000000000000000; b = 16'b1000000000000000; cin = 1'b0;
#10;

// Additional test cases can be added here

$stop;
end

```

```
endmodule
```

Code 24: testbench for adder module

```
module au_tb;

// Parameters
parameter n = 32;

// Inputs
reg [n-1:0] a;
reg [n-1:0] b;
reg sub;
reg u;

// Outputs
wire [n-1:0] s;
wire neg;
wire ovf;

// Instantiate the Unit Under Test (UUT)
au #(n) uut (
.a(a),
.b(b),
.sub(sub),
.u(u),
.s(s),
.neg(neg),
.ovf(ovf)
);

initial begin
// Initialize Inputs
a = 0;
b = 0;
sub = 0;
u = 0;

// Wait 10 ns for global reset to finish
#10;

// Test cases
a = 32'b00000000000000000000000000000001;
b = 32'b00000000000000000000000000000001;
sub = 1'b0;
u = 1'b0;
#10; // Wait for 10 ns

a = 32'b11111111111111111111111111111111;
b = 32'b00000000000000000000000000000001;
sub = 1'b0;
u = 1'b0;
#10;

a = 32'b00001111111111111111111111111111;
b = 32'b00010000000000000000000000000000;
sub = 1'b1;
u = 1'b0;
#10;

a = 32'b10101010101010101010101010101010;
b = 32'b01010101010101010101010101010101;
sub = 1'b0;
u = 1'b0;
#10;
```

```
62
63      a = 32'b10000000000000000000000000000000;
64      b = 32'b10000000000000000000000000000000;
65      sub = 1'b0;
66      u = 1'b0;
67      #10;
68
69
70
71      // Finish simulation
72
73      end
74
75      endmodule
```

Code 25: testbench for arithmetic unit module

```
1       module alu_tb;
2
3       // Parameters
4       parameter n = 32;
5
6       // Inputs
7       reg [n-1:0] a;
8       reg [n-1:0] b;
9       reg [3:0] af;
10      reg i;
11
12      // Outputs
13      wire [n-1:0] alures;
14      wire ovfalu;
15
16      // Instantiate the Unit Under Test (UUT)
17      alu #(n) uut (
18      .a(a),
19      .b(b),
20      .af(af),
21      .i(i),
22      .alures(alures),
23      .ovfalu(ovfalu)
24      );
25
26      initial begin
27      // Initialize Inputs
28      a = 0;
29      b = 0;
30      af = 4'b0000;
31      i = 0;
32
33      // Wait 10 ns for global reset to finish
34      #10;
35
36      // Test cases
37      a = 32'b00000000000000000000000000000001;
38      b = 32'b00000000000000000000000000000001;
39      af = 4'b0000;
40      i = 1'b0;
41      #10; // Wait for 10 ns
42
43      a = 32'b11111111111111111111111111111111;
44      b = 32'b00000000000000000000000000000001;
45      af = 4'b0001;
46      i = 1'b0;
47      #10;
48
```

```
    a = 32'b00001111111111111111111111111111;
    b = 32'b00010000000000000000000000000000;
    af = 4'b0010;
    i = 1'b0;
    #10;

    a = 32'b00001111111111111111111111111111;
    b = 32'b00010000000000000000000000000000;
    af = 4'b0011;
    i = 1'b0;
    #10;

    a = 32'b10101010101010101010101010101010;
    b = 32'b01010101010101010101010101010101;
    af = 4'b0100;
    i = 1'b0;
    #10;

    a = 32'b10000000000000000000000000000000;
    b = 32'b10000000000000000000000000000000;
    af = 4'b0101;
    i = 1'b0;
    #10;

    a = 32'b11111111111111110000000000000000;
    b = 32'b00000000000000001111111111111111;
    af = 4'b0110;
    i = 1'b0;
    #10;

    a = 32'b11111111111111110000000000000000;
    b = 32'b00000000000000001111111111111111;
    af = 4'b0111;
    i = 1'b1;
    #10;

    a = 32'b11111111111111110000000000000000;
    b = 32'b00000000000000001111111111111111;
    af = 4'b0111;
    i = 1'b0;
    #10;

    a = 32'b00000000000000000000000000000001;
    b = 32'b00000000000000000000000000000001;
    af = 4'b1010;
    i = 1'b0;
    #10;

    a = 32'b00000000000000000000000000000001;
    b = 32'b00000000000000000000000000000010;
    af = 4'b1011;
    i = 1'b0;
    #10;



    // Finish simulation
    $stop;
    end

    endmodule
```

Code 26: testbench for ALU module

```
    module srlfix_tb;
```

```
2
3
4       parameter i = 16;
5
6       reg [31:0] a;
7       reg s;
8
9       wire [31:0] res;
10
11      srlfix #(i) uut (
12      .a(a),
13      .s(s),
14      .res(res)
15      );
16
17      initial begin
18      // Initialize Inputs
19      a = 0;
20      s = 0;
21
22      #10;
23
24      // Test cases
25      a = 32'b00000000000000001111111111111111;
26      s = 1'b1;
27      #10; // Wait for 10 ns
28
29      a = 32'b11111111111111110000000000000000;
30      s = 1'b1;
31      #10;
32
33      a = 32'b10101010101010101010101010101010;
34      s = 1'b0;
35      #10;
36
37
38
39      // Finish simulation
40      $stop;
41      end
42
43      endmodule
```

Code 27: testbench for srlfix.v

```
1       module srl_tb;
2
3       // Inputs
4       reg [31:0] a;
5       reg [4:0] b;
6
7       // Outputs
8       wire [31:0] res;
9
10      // Instantiate the Unit Under Test (UUT)
11      srl uut (
12      .a(a),
13      .b(b),
14      .res(res)
15      );
16
17      initial begin
18      // Initialize Inputs
19      a = 0;
20      b = 0;
```

```
21
22      #10;
23
24      // Test cases
25      a = 32'b00000000000000000111111111111111;
26      b = 5'b00001;
27      #10; // Wait for 10 ns
28
29      a = 32'b11111111111111110000000000000000;
30      b = 5'b00010;
31      #10;
32
33      a = 32'b11111111111111110000000000000000;
34      b = 5'b00110;
35      #10;
36
37
38
39      // Finish simulation
40      $stop;
41      end
42
43      endmodule
```

Code 28: testbench for srlfix.v

```
1       module bce_tb;
2
3       // Inputs
4       reg [31:0] a;
5       reg [31:0] b;
6       reg [3:0] bf;
7
8       // Outputs
9       wire bcres;
10
11      // Instantiate the Unit Under Test (UUT)
12      bce uut (
13      .a(a),
14      .b(b),
15      .bf(bf),
16      .bcres(bcres)
17      );
18
19      initial begin
20      // Initialize Inputs
21      a = 32'b00000000000000000000000000000000;
22      b = 32'b00000000000000000000000000000000;
23      bf = 4'b0000;
24
25      // Wait 10 ns for global reset to finish
26      #10;
27
28      // Test case 1: bf = 0010, [a] < 0
29      a = 32'b10000000000000000000000000000000; // Negative number
30      b = 32'b00000000000000000000000000000000; // Not used in this case
31      bf = 4'b0010;
32      #10;
33
34      // Test case 2: bf = 0011, [a] >= 0
35      a = 32'b00000000000000000000000000000001; // Positive number
36      b = 32'b00000000000000000000000000000000; // Not used in this case
37      bf = 4'b0011;
38      #10;
39
```

```
    // Test case 3: bf = 1000, a == b
    a = 32'b00000000000000000000000000000101;
    b = 32'b00000000000000000000000000000101;
    bf = 4'b1000;
    #10;

    // Test case 4: bf = 1010, a != b
    a = 32'b00000000000000000000000000000101;
    b = 32'b00000000000000000000000000000110;
    bf = 4'b1010;
    #10;

    // Test case 5: bf = 1100, [a] <= 0
    a = 32'b10000000000000000000000000000000; // Negative number
    b = 32'b00000000000000000000000000000000; // Not used in this case
    bf = 4'b1100;
    #10;

    // Test case 6: bf = 1110, [a] > 0
    a = 32'b00000000000000000000000000000001; // Positive number
    b = 32'b00000000000000000000000000000000; // Not used in this case
    bf = 4'b1110;
    #10;


    // Finish simulation
    $stop;
    end

    endmodule
```

Code 29: testbench for bce.v

```

module master_tb();

// Test bench signals
reg external_clk;
reg rst;
wire [31:0] debug_hex_display;
wire clock_led;

// Instantiate the master module
master uut (
    .external_clk(external_clk),
    .ena(1),
    .rst(rst),
    .debug_hex_display(debug_hex_display),
    .clock_led(clock_led)
);

initial begin
    #5 external_clk = 0;
    forever #5 external_clk = ~external_clk;
end

initial begin
    // Initial values
    rst = 0;

    // Apply reset
    #10;
    rst = 1;
    #20;
    rst = 0;
```

```
33 end
34
35 // Monitor important signals
36 always @(posedge external_clk) begin
37     if (uut.E) begin
38         $display("Time=%0t PC=%h Instruction=%h",
39                  $time, uut.pc, uut.mem_out);
40     end
41 end
42
43 // Additional monitoring for debug
44 initial begin
45     $monitor("Time=%0t rst=%b E=%b pc=%h debug_display=%h",
46              $time, rst, uut.E, uut.pc, debug_hex_display);
47 end
48
49 endmodule
```

Code 30: testbench for master.v

19 References

[1] Intel Corporation, “Intel Cyclone V FPGA,” Intel, San Jose, CA, USA. [Online]. Available: https://www.intel.com/content/www/us/en/products/details/fpga/cyclone/v.html. [Accessed: Apr. 25, 2026].

[2] Intel Corporation, “External Memory Interface Handbook Volume 2,” doc. 683385, 2017. [Online]. Available: https://www.intel.com/programmable/technical-pdfs/683385.pdf. Archived copy: https://web.archive.org/web/20250227145518/https://cdrdv2-public.intel.com/666390/emi_plan-683385-666390.pdf. [Accessed: Apr. 25, 2026].

[3] S. Lortkipanidze, G. Bakradze, and L. Turmanidze, “MIPS-CPU-Verilog,” GitHub repository, 2026. [Online]. Available: https://github.com/sandrolort/MIPS-CPU-Verilog. [Accessed: Apr. 25, 2026].

[4] Missouri State University, “MARS MIPS Assembler and Runtime Simulator.” [Online]. Available: https://computerscience.missouristate.edu/mars-mips-simulator.htm. [Accessed: Apr. 25, 2026].

[5] W. J. Paul, C. Baumann, P. Lutsyk, and S. Schmaltz, *System Architecture: An Ordinary Engineering Discipline*. Cham, Switzerland: Springer, 2016.

[6] P. Sanderson and K. Vollmar, “MARS: An education-oriented MIPS assembly language simulator,” in *Proc. 37th SIGCSE Tech. Symp. Computer Science Education*, Houston, TX, USA, 2006, pp. 239–243.